

OaldresPuzzle_Cryptic Type-2 分组密码技术细节论文

基于 /OOP/BlockCipher/ 当前实现的中文学术技术论文

Twilight-Dream Sparkle-Magic

2026-06-10

摘要

我们提出并形式化整理 OaldresPuzzle_Cryptic Type-2 状态化分组密码构造。该构造以 /OOP/BlockCipher/ 实现为准，把主密钥字、初始化向量、矩阵状态、轮密钥矩阵、字级可逆轮函数和静态字节替换层连接为一条端到端加解密路径。我们设计该结构的目标不是仅扩大密钥空间、明文空间或轮数，而是在同一算法内耦合 $\mathbb{F}_p$ 上的 SIS/Ajtai 形式矩阵像、白盒海绵 hardening、 $\mathbb{Z}/2^{32}\mathbb{Z}$ 与 $\mathbb{Z}/2^{64}\mathbb{Z}$ 上的字运算、 $\mathbb{F}_2$ 上的 BitMatrix 扩散、状态相关矩阵选址、self-masking 与模加 carry，使传统差分、线性、代数分析以及量子攻击电路的 oracle 构造同时面对多域语义和状态轨道。

我们首先定义主子密钥硬化映射

$$H_M(x) = Mx + \text{TDOM-Sponge}(Mx) \pmod{p}, \quad p = 2^{64} - 59,$$

其中 $Mx \bmod p$ 形成 SIS/Ajtai-form 素域矩阵核心，TDOM-Sponge 是由公开轮常量、公开置换表和 χ -like 布尔层组成的白盒海绵，而不是黑盒随机预言机。硬化后的材料经过 32 位扩展、矩阵初始化、NLFSR/SDP 驱动的状态更新和 Kronecker 外积窄注入形成 A_t, T_t, G_t, π_t 。轮密钥层再通过非交换矩阵表达式、key-state whitening 和 $M_{\text{bit}} = D \otimes I_{64}$ 的 bit-uniform parity 投影生成数据轮所需的状态相关轮材料。数据轮以 CrazyTransformAssociatedWord 作为 Lai-Massey 的 F 函数，在矩阵选址、mask 删除、双通道子密钥混合和 $\mathbb{Z}/2^{32}\mathbb{Z}$ 模加 carry 中引入非线性与状态依赖，并由静态 byte substitution 层完成 macro-round 末端扰动。

我们给出该构造的实现语义、数学定义和局部可证明性质：SIS/Ajtai 核心的碰撞、差分与线性约束；白盒海绵的吸收、置换和挤出路径；Kronecker 注入 $T = A(\delta v u) = \delta(Av)u$ 的 collapsed / outer-product / rank-at-most-one-style 窄映射解释；BitMatrix 的行列度与单比特传播性质；PHT 与 Lai-Massey 字级置换的可逆性；以及 byte substitution 正逆表和 $16 \times N^2$ macro-round 调度。我们同时给出可证明性质和后续分析对象：本文不给出完整 IND-CPA/PRP 归约，也不把整个分组密码归约为 SIS；本文把复杂度拆成可审计的数学对象，并说明这些对象如何提高传统分析和量子攻击电路构造的建模负担。相关背景包括 SIS/Ajtai 哈希、海绵结构、S-box 分析、Lai-Massey 结构和量子攻击模型 [8, 29, 7, 2, 4, 5, 19, 6, 13, 24, 11, 12, 9]。

关键词：OaldresPuzzle_Cryptic；Type-2 分组密码；SIS/Ajtai 哈希；Montgomery 素域；海绵函数；Lai-Massey；BitMatrix；状态化 key schedule；后量子对称密码。

目录

1 总论：复杂性设计、量子门电路成本与传统分析阻力	6
1.1 我们给出密钥空间、明文空间与密文空间	6
1.2 我们首先声明现实量子工程目标	7
1.3 我们面对的量子 oracle 构造问题	7
1.4 我们把基本运算翻译为量子门成本	8
1.5 我们把 key schedule 写成量子电路的痛点	8
1.6 我们让数据相关矩阵访问变成 oracle 构造灾难	9
1.7 我们把 BitMatrix 写成宽 CNOT 网络而不是 word 表	9
1.8 复杂性设计的结构化收益	10
1.9 复杂性不是安全幻觉的局部证据	10
1.10 总-分-总的论文组织逻辑	10
2 研究对象、算法边界与写作原则	10
3 总体结构、总伪代码与复杂性设计动机	11
3.1 整体模块链	11
3.2 端到端总伪代码	11
3.3 为什么复杂性不是无用堆叠	12
3.4 总-分-总的正文组织	12
4 缩写、数学符号与运算语义	12
4.1 缩写定义	13
4.2 数学符号定义	13
4.3 代数空间和运算语义	14
4.4 缩写、代码名和数学符号的排版约定	14
5 公共状态对象：Modules_0aldresPuzzle_Cryptic.hpp	15
5.1 我们定义的实现对象	15
5.2 我们设计这一层的灵感	15
5.3 我们使用的数学定义	15
5.4 我们能够直接证明的性质	15
6 Montgomery 素域层：ExtraIncludes/MontgomeryScalarEigen.hpp	15
6.1 我们定义的实现对象	15
6.2 我们设计这一层的灵感	16
6.3 我们使用的数学定义	16
7 自定义海绵哈希：CustomSecureHash.hpp 与 TDOM_HashModule --- SecureCustomHash.md	16
7.1 我们定义的白盒海绵实现对象	16
7.2 我们设计这一层的灵感：为什么我们把 SIS/LWE 空间再送入白盒海绵	16
7.3 常量与固定表的来源：SecureCustomHash.md	19
7.4 Padding、吸收和挤出：实现语义	20
7.5 一轮状态变换 $\mathcal{P}_h$ 的完整数学展开	20

7.6 我们给出的数学解释：白盒语义与标准海绵语义的边界	21
8 主子密钥硬化：Module_SecureSubkeyGeneratation.*	22
8.1 我们定义的实现对象	22
8.2 我们设计这一层的灵感	22
8.3 我们使用的数学定义	22
8.4 SIS/Ajtai 裸核的碰撞困难性	22
8.5 与扭曲空间推导的连接	23
8.6 与后续矩阵状态的接口	23
9 32 位门级扩展：Module_MixTransformationUtil.*	23
9.1 我们定义的实现对象	23
9.2 我们设计这一层的灵感	23
9.3 bit restructure	23
9.4 12-word expansion 的完整公式	23
9.5 为什么这不是随便扩展	24
10 矩阵初始化、IV 注入和状态更新：Module_SubkeyMatrixOperation.*	24
10.1 我们定义的实现对象	24
10.2 我们设计这一层的灵感	24
10.3 初始化数学	24
10.4 IV 注入数学	24
10.5 NLFSR 仿射广播变异	25
10.6 Kronecker 外积注入	25
10.7 为什么窄映射在设计中可行	26
11 轮密钥矩阵、whitening 和 BitMatrix：Module_SecureRoundSubkeyGeneratation.*	26
11.1 我们定义的实现对象	26
11.2 轮密钥矩阵变换：从矩阵状态到 G_t	26
11.3 我们设计非交换矩阵表达式的灵感	26
11.4 轮密钥向量 whitening：key-state 合成而非数据 whitening	27
11.5 BitMatrix 的构造位置：hard-coded XOR ledger 是压缩表示	27
11.6 我们设计 BitMatrix 的灵感：bit 均匀分布	28
11.7 当前常量的行集合与设计判据	28
12 PHT 正逆变换与 Lai-Massey 可逆性	29
12.1 PHT 正向变换	29
12.2 PHT 逆向变换	29
12.3 Lai-Massey 字级结构	29
13 状态相关 F 函数：CrazyTransformAssociatedWord	30
13.1 我们定义的实现对象	30
13.2 我们设计这一层的灵感	30
13.3 完整数学定义	30
13.4 为什么这个映射变态	31

14 静态字节替换层: OaldresPuzzle_Cryptic.hpp/.cpp	31
14.1 我们定义的实现对象	31
14.2 我们设计这一层的灵感	32
14.3 我们给出的 S-box 生成公式与逆表公式	32
14.4 我们如何从代码得到数学实现	32
14.5 S-box 指标的计算定义与分析范围	33
14.6 这一层与当前 Type-2 core 的边界	33
15 数据块轮函数: OaldresPuzzle_Cryptic.cpp	34
15.1 我们定义的实现对象	34
15.2 我们使用的数学定义	34
15.3 我们设计这一层的灵感	34
16 Worker 层与核心范围: OPC_MainAlgorithm_Worker.*	34
16.1 我们定义的实现对象	34
16.2 设计灵感和边界	34
16.3 核心路径定义	35
16.4 Stateful 性质	35
17 PRNG 状态材料与内部 CSPRNG 子系统: Includes/PRNGs.hpp	35
17.1 本节在实现中的位置	35
17.2 LFSR 实现: 128 位状态、右移反馈和输出累积	35
17.2.1 代码对应	35
17.2.2 数学定义	36
17.2.3 设计灵感	36
17.2.4 可证明性质	36
17.3 NLFSR 实现: 九个反馈多项式槽位、四路门控推进和两级组合函数	36
17.3.1 代码对应	36
17.3.2 数学定义: 反馈槽位	37
17.3.3 数学定义: 组合函数	37
17.3.4 设计灵感	37
17.3.5 可证明性质	37
17.4 SDP-CSPRNG 实现: 双摆初始化、连续系统推进和 64 位抽样	38
17.4.1 代码对应	38
17.4.2 数学定义: 初始化映射	38
17.4.3 数学定义: 双摆动力系统	38
17.4.4 数学定义: 输出抽样	39
17.4.5 设计灵感	39
17.4.6 可证明性质	39
17.5 PRNG 三源在 Type-2 中的分工	39
17.6 为什么这不是把安全性推给 PRNG	40
18 核心子程序的详细 breakablealgorithm 伪代码	40
18.1 主子密钥硬化	40

18.2 白盒海绵	40
18.3 矩阵状态推进与 Kronecker 外积注入	41
18.4 轮密钥生成	41
18.5 状态相关 F 函数	41
18.6 数据轮与解密逆序	42
19 模块级 breakablealgorithm 伪代码完整展开	42
19.1 参数、状态与数据表示	42
19.2 主子密钥硬化与白盒海绵	43
19.3 32 位扩展与矩阵初始化	45
19.4 状态材料发生器	47
19.5 矩阵状态、轮密钥与 BitMatrix	48
19.6 数据轮函数与可逆层	49
19.7 worker 级状态化调度	52
20 端到端正确性与实现—数学对应关系	52
21 设计动机、分析边界与后续工作	53
22 结论	53

1 总论：复杂性设计、量子门电路成本与传统分析阻力

我们把 Type-2 OPC 的复杂性设计放在正文最前面讨论，因为本构造不是“普通分组密码加若干复杂附件”。我们设计该算法时面对的是双重攻击者：一类攻击者使用传统差分、线性、代数、相关密钥和状态恢复技术；另一类攻击者尝试把完整算法嵌入量子 `oracle`、可逆量子线路或相关密钥查询框架，从而使用 Grover 型搜索、Simon 型周期搜索、量子碰撞或量子相关密钥技术 [24, 11, 12, 9, 19]。因此，我们的复杂性目标不是让算法“看起来复杂”，而是让攻击者在真正构造分析对象时必须同时承担多域语义、多状态轨道、多层查表、多处 carry 链、动态轮密钥依赖和可逆反算成本。

我们知道一个复杂算法最容易被质疑为“为了复杂度而复杂度”。我们给出的回答不是口号，而是把每一层复杂性变成可命名、可追踪、可推导的数学对象：SIS/Ajtai 素域核负责把主子密钥差分压入短关系约束；TDOM 白盒海绵负责把素域像送入公开但非线性的状态置换；32-bit expansion 负责破坏 word 局部性；矩阵状态轨道负责让轮密钥不再是静态表；Kronecker 外积注入负责把 SDP 材料压成方向性窄映射；OPC 矩阵变换负责引入非交换矩阵传播；key-state whitening 负责合成旧状态与新矩阵像；BitMatrix 负责 bit-level 均匀 parity 分布；CrazyTransformAssociatedWord 负责状态相关矩阵访问、self-masking 和模加 carry 注入；Lai-Massey 负责把不可逆 F 函数放入可逆字级置换；ByteSubstitution 负责在 macro-round 末尾加入静态字节置换扰动。这样得到的复杂性不是孤立堆叠，而是一条从 key schedule 到数据轮的职责链。

1.1 我们给出密钥空间、明文空间与密文空间

我们首先把 Type-2 OPC 的三个基础空间写清楚。当前 C++ 核心没有把分组大小和密钥块大小写死在轮函数内部，而是在 Common-StateData 中以 64-bit quadword 为单位给出两个参数：

$$B = \text{OPC_QuadWord_DataBlockSize}, \quad K = \text{OPC_QuadWord_KeyBlockSize}.$$

源码约束为

$$B \geq 2, \quad B \equiv 0 \pmod{2}, \quad K \geq 4, \quad K \equiv 0 \pmod{4}, \quad K > B, \quad K \equiv 0 \pmod{B}.$$

因此，单个明文分组和单个密文分组的比特长度为

$$n = 64B,$$

单个主密钥块的比特长度为

$$\kappa = 64K.$$

于是，Type-2 OPC 的单分组明文空间、单分组密文空间和单主密钥块空间分别为

$$\mathcal{P}_B = \{0, 1\}^{64B}, \quad \mathcal{C}_B = \{0, 1\}^{64B}, \quad \mathcal{K}_K = \{0, 1\}^{64K}.$$

它们的基数为

$$|\mathcal{P}_B| = |\mathcal{C}_B| = 2^{64B}, \quad |\mathcal{K}_K| = 2^{64K}.$$

若消息在 padding 与 packing 后包含 m 个数据分组，则消息级明文/密文空间为

$$\mathcal{P}_B^{(m)} = \mathcal{C}_B^{(m)} = \{0, 1\}^{64Bm}, \quad |\mathcal{P}_B^{(m)}| = |\mathcal{C}_B^{(m)}| = 2^{64Bm}.$$

若外部传入 r 个主密钥块，则原始主密钥材料空间为

$$\mathcal{K}_K^{(r)} = \{0, 1\}^{64Kr}, \quad |\mathcal{K}_K^{(r)}| = 2^{64Kr}.$$

这种写法区分了“单次 key-schedule 处理的密钥块空间”和“外部调用层一次性输入的总密钥材料空间”，避免把可变长度输入误写成固定密钥长度。

在当前 C API 上下文和测试配置中，默认参数为

$$B = 16, \quad K = 32.$$

于是默认单分组长度为

$$n = 64 \cdot 16 = 1024 \text{ bits},$$

默认单主密钥块长度为

$$\kappa = 64 \cdot 32 = 2048 \text{ bits}.$$

因此，默认实例的核心空间为

$$|\mathcal{P}_{16}| = |\mathcal{C}_{16}| = 2^{1024}, \quad |\mathcal{K}_{32}| = 2^{2048}.$$

参数	符号	默认值	单位	实现来源
数据分组大小	B	16	64-bit words	OPC_QuadWord_DataBlockSize
主密钥块大小	K	32	64-bit words	OPC_QuadWord_KeyBlockSize
矩阵阶数	N	64	rows/columns	$N = 2K$
明文分组长度	$64B$	1024	bits	block definition
密文分组长度	$64B$	1024	bits	block definition
主密钥块长度	$64K$	2048	bits	key material
轮密钥向量长度	N^2	4096	64-bit words	$\text{vec}(G_t)$
宏轮数	R	16	macro-rounds	RoundFunction
单块字级更新数	RN^2	65536	LM word updates	$16 \cdot 64^2$

表中数值说明默认实例的基本规模。默认单块不是 128-bit 小分组, 而是 1024-bit 分组; 默认主密钥块不是 256-bit 小密钥, 而是 2048-bit key words。我们在后续量子电路资源账本中使用同一组默认参数, 避免把抽象公式和实现规模割裂。最小合法参数实例由 $B = 2, K = 4$ 给出, 此时分组空间为 2^{128} , 单主密钥块空间为 2^{256} ; 默认实例则采用 1024-bit 分组和 2048-bit 主密钥块。我们采用默认实例作为论文主体, 是因为该配置已经明显超过经典 128-bit 分组和 256-bit 密钥的最低安全边界, 并且与后续 $N = 2K = 64$ 的矩阵状态规模、轮密钥矩阵规模和量子 oracle 资源账本自然闭合。

初始化向量和 PRNG 种子不在本文中被混同为主密钥空间。若初始化向量被作为秘密状态材料使用, 则其字节长度必须满足

$$|IV|_{\text{bytes}} \equiv 0 \pmod{8B},$$

并可额外引入 $8|IV|_{\text{bytes}}$ 比特的状态参数; LFSR 与 NLFSR 种子分别排除零种子, SDP 种子还需要满足源码中的下界约束。本文把这些对象称为状态化参数空间或初始化状态空间, 而不把它们无条件并入 $\mathcal{K}_K$, 因为主密钥空间、初始化状态空间和消息空间在实现中承担不同职责。

1.2 我们首先声明现实量子工程目标

我们不把本文写成对“神谕式理论量子计算机”的绝对防御承诺。若攻击者被允许拥有无界容错深度、无代价大规模 qRAM、无成本可逆 RAM、无成本反算和可直接调用理想加密 oracle 的抽象模型, 则任何具体工程算法都无法用实现复杂度本身作为绝对屏障。我们讨论的对象是更接近现实工程的容错量子攻击: 攻击者必须把算法写成具体可逆线路, 必须为 Toffoli/T 门、CNOT 门、受控查表、可逆乘法、模加 carry、矩阵状态寄存器、garbage 清理和 oracle 反算付出资源成本。我们设计 Type-2 OPC 的态度是: 不把安全性建立在“攻击者看不懂”上, 而是让攻击者即使看懂, 也必须面对一个难以压扁成低深度、低 ancilla、低查表成本的完整 key-schedule oracle。

我们知道传统对称密码常见增强方式是扩大密钥空间、扩大分组空间或增加轮数; 这些方式对 Grover 型搜索仍然有效, 因为查询次数从 2^κ 降到 $2^{\kappa/2}$ 后, 密钥长度仍然是第一道资源因素。然而 Type-2 OPC 不满足于只把 κ 加大。我们进一步让一次 oracle 查询本身变重: 攻击者每进行一次 Grover 迭代或相关密钥查询, 都需要构造并反算一条包含 SIS/Ajtai 矩阵像、白盒海绵、矩阵状态更新、动态轮密钥和数据轮函数的可逆线路。也就是说, 我们不只增加“搜索空间”, 还增加“每次搜索查询的电路痛苦”。

1.3 我们面对的量子 oracle 构造问题

我们把一次量子攻击中需要构造的理想加密 oracle 写成

$$U_E : |K, X, Y\rangle \mapsto |K, X, Y \oplus E_K(X)\rangle.$$

对普通静态轮函数而言, 攻击者可以把 E_K 展开为重复的 S-box、线性层和轮密钥异或。Type-2 OPC 的 oracle 不是这种低语义层数的静态 SPN。若把初始状态也纳入 oracle, 则可逆嵌入至少需要处理

$$\tilde{U}_{\text{OPC}} : |K, IV, X, Y, S_0, 0^a\rangle \mapsto |K, IV, X, Y \oplus E_{K, IV, S_0}(X), S_t, G_t, 0^{a'}\rangle,$$

其中 $S_t = (A_t, T_t, \pi_t, \text{PRNG}_t)$, G_t 是动态生成的轮密钥矩阵。若攻击者希望得到干净 oracle, 还必须通过 Bennett 式 compute-copy-uncompute 清理中间状态:

$$U_{\text{clean}} = U_{\text{KS}}^\dagger U_{\text{Round}} U_{\text{KS}} \quad \text{或更细地写成} \quad U_f^\dagger \text{CNOT}_{f \rightarrow Y} U_f.$$

因此, 若 $C_Q(F)$ 表示可逆实现 F 的门级成本, 则 clean oracle 的主要代价满足

$$C_Q(U_{\text{clean}}) \geq 2C_Q(U_{\text{KS}}) + C_Q(U_{\text{Round}}) + C_Q(\text{CopyOut}),$$

并且 ancilla 寄存器不能在中途随意丢弃。我们设计这条路径, 就是为了让量子攻击者不能只把数据轮函数单独抽出来作为低成本 oracle。

1.4 我们把基本运算翻译为量子门成本

我们知道“量子电路很难搭”不能只停留在感受层面，所以先给出可逆门级翻译。表1 给出本文使用的成本模型。这里的 n 表示 word bit 宽，Type-2 的主要 word 宽为 32 和 64； w 表示被查表字宽； m 表示候选表项数量。成本可以分别按 Toffoli 数、T-count、CNOT 数、深度和 clean ancilla 数评价。本文只使用量级和结构下界，不声称给出最优量子综合。

表 1: Type-2 OPC 基本运算在可逆量子线路中的门级语义

经典/字级运算	可逆门级实现语义	对攻击电路的影响
$x \oplus y$	bitwise CNOT; 若覆盖写入, 需要保留可逆输入或额外输出寄存器。	线性, 但数量大时形成宽 CNOT 网络。
$\sim x$	Pauli- X 门逐 bit 翻转。	成本低, 但经常与 AND/OR 共同形成非线性门。
$x \& y$	Toffoli 产生目标 bit $t \leftarrow t \oplus xy$ 。	布尔二次项的基本成本来源。
$x y$	由 $x y = \sim(\sim x \& \sim y)$ 实现, 包含 X 与 Toffoli。	OR 不是免费线性操作, 进入 T/Toffoli 成本。
常量 rotation / shift	线重命名或固定 SWAP 网络。	常量旋转成本低, 利于 diffusion。
数据相关 rotation	controlled barrel shifter 或多级受控 SWAP。	成本约为 $\Omega(n \log n)$ 受控操作; 无 fanout 优化时更高。
n -bit 加法	ripple-carry 或 carry-lookahead reversible adder。	至少携带 carry 链; Toffoli/T-depth 不可忽略。
n -bit 乘法	partial products + reversible adders。	约为 $\Omega(n^2)$ 级 Toffoli/AND 路径。
模 p 乘法	乘法 + 条件约减 + 比较/减法。	Montgomery/素域运算带来额外 carry 和条件选择。
静态 S-box 查表	小规模 reversible lookup 或预综合 S-box 网络。	对 8-bit 表可实现, 但每个 byte 都要付门成本。
数据相关矩阵读取	若无 qRAM, 需 controlled select / multiplexer。	N^2 候选项的选择不能当作免费数组访问。
BitMatrix $D \otimes I_{64}$	CNOT parity network。	线性但宽; 每个 32-word chunk 至少 $32 \cdot 15 \cdot 64$ 个 XOR 合成操作。

我们因此把 Type-2 的量子成本写成模块和门级成本的组合，而不是只写 $O(2^{\kappa/2})$ 。Grover 只降低查询次数，不会消除一次查询内部的 key schedule 展开、矩阵访问和反算成本。

1.5 我们把 key schedule 写成量子电路的痛点

设 $N = 2K$ 。Type-2 的主子密钥硬化阶段包含 $\mathbb{F}_p$ 上的矩阵-向量乘法和白盒海绵：

$$H_M(x) = Mx + \text{TDOM-Sponge}(Mx) \pmod p.$$

若把 M 和 x 视为量子寄存器中的数据，则矩阵-向量乘法至少包含 N^2 次 64-bit 素域乘法和相应累加：

$$C_Q(Mx \bmod p) \gtrsim N^2 C_Q(\text{MontMul}_{64,p}) + N(N-1) C_Q(\text{Add}_{64,p}).$$

白盒 TDOM 进一步加入 rotation schedule、index schedule、round constants、相邻 word mixing、半状态扩散和 χ -like 二次布尔层。若把 $a_{i+1} = a_i \oplus ((\sim b_i) \& c_i)$ 作为典型 χ -like bit 更新，则每个 bit 至少引入一个 Toffoli 型二次项；状态宽度越大，Toffoli/T-count 与 ancilla 压力越直接。

矩阵状态轨道更难被压扁。我们把 $A_t, T_t, G_t \in \mathbb{Z}/2^{64}\mathbb{Z}^{N \times N}$ 作为量子寄存器时，仅保存三张矩阵就需要至少

$$A_Q(\text{matrix state}) \geq 3 \cdot 64N^2$$

个逻辑量子 bit，不包括 PRNG 状态、海绵状态、临时乘法器、carry 寄存器和反算所需历史信息。轮密钥矩阵变换

$$G \leftarrow G + (T^\top - A)(A^\top + T)AT$$

包含转置、加减和至少两到三次矩阵乘法语义。以普通 schoolbook 乘法估算，单次 $N \times N$ word 矩阵乘法包含 N^3 个 64-bit word 乘加项：

$$C_Q(\text{MatMul}_{N,64}) \gtrsim N^3 (C_Q(\text{Mul}_{64}) + C_Q(\text{Add}_{64})).$$

我们知道该估计不是最优量子综合，但它足够说明一个事实：攻击者不能把动态轮密钥生成视为免费 **classical preprocessing**，因为密钥和状态在量子查询中必须 **coherent** 地参与 **oracle**。

在默认参数 $B = 16, K = 32, N = 64$ 下，上述成本可以写成更直接的资源账本。

对象或过程	默认规模	对量子攻击电路的含义
单张矩阵 A_t, T_t, G_t	$64^2 = 4096$ 个 64-bit word	每张矩阵至少 262144 个逻辑 bit
三张矩阵状态	$3 \cdot 4096$ 个 64-bit word	至少 786432 个逻辑 bit ，不含临时寄存器
素域 GEMV Mx	4096 次 64-bit Montgomery 乘法	主子密钥硬化不能视为一次普通 XOR
单次 64×64 word 矩阵乘法	$64^3 = 262144$ 个 word 乘加位置	轮密钥矩阵变换带来立方级乘加账本
轮密钥向量	4096 个 64-bit word	每次 round-subkey generation 输出 262144 bit
单个数据块加密	$16 \cdot 4096 = 65536$ 次 LM 更新	攻击 oracle 需 coherent 地合成大量状态相关 F 调用
BitMatrix 投影	每 32-word chunk 为 $32 \cdot 64 \cdot 15 = 30720$ 个 XOR 合成	宽 CNOT parity network ，不能被写成 word 注释

我们把这些数值放在总论中，是为了把“量子攻击电路会很痛苦”落到资源账本上：攻击者不是只面对 $2^{\kappa/2}$ 次 **Grover** 查询，而是每一次查询都必须实现主子密钥硬化、矩阵状态推进、轮密钥生成、状态相关查表、**Lai-Massey** 字级置换和末端字节替换的可逆组合。

1.6 我们让数据相关矩阵访问变成 **oracle** 构造灾难

CrazyTransformAssociatedWord 中的矩阵访问由数据和 **word key** 共同决定：

$$r = \pi[A \bmod N], \quad c = \pi[B \bmod N], \quad q = G_{r,c}.$$

在经典代码里，这只是一次数组索引；在量子线路里，若 A, B, G, π 处于叠加或依赖密钥的 **coherent** 状态，则读取 $G_{r,c}$ 不能被当作普通内存读。若不假设无代价 **qRAM**，则必须实现可逆选择：

$$|r, c, G, 0\rangle \longmapsto |r, c, G, G_{r,c}\rangle.$$

直接 **multiplexer** 需要比较 (r, c) 与 N^2 个候选坐标，并受控复制 **64-bit** 值；其门级成本至少携带

$$\Omega(N^2 \cdot 64)$$

规模的受控选择痕迹。若再把 $\pi[A \bmod N]$ 与 $\pi[B \bmod N]$ 的 **permutation lookup** 计入，攻击者还需要实现两个额外的受控表读取。我们设计这种状态相关矩阵访问，是为了让 F 函数不是固定 **S-box**，也不是固定 **ARX** 网络，而是一个依赖轮密钥矩阵状态的可审计复杂选择映射。

进一步地， q 不是直接进入输出。算法先从 q 中抽取两个 **bit**，构造左右旋转 **mask**，再形成

$$\tilde{q} = q \ \& \ \sim m,$$

然后让 **masked subkey** 和 **original subkey** 进入双通道输出：

$$F(a, k) = a + (T_1(A, B, \tilde{q}) \oplus T_2(C, D, q)) \pmod{2^{32}}.$$

这一步同时包含数据相关查表、**bit extraction**、旋转 **mask**、布尔混合和模加 **carry**。我们知道这些操作每一项都能单独实现；困难点在于它们必须在同一个 **coherent oracle** 中一起实现并反算，不能把某一步当成 **classical side computation**。

1.7 我们把 **BitMatrix** 写成宽 **CNOT** 网络而不是 **word** 表

BitMatrix 层的真正对象不是 32-word 的记账表，而是

$$\text{bit}(Y) = (D \otimes I_{64}) \text{bit}(X).$$

每个输出 **bit** 是 16 个输入 **bit** 的 **parity**；每个输入 **bit** fan-out 到 16 个输出 **bit**。对一个 32-word chunk 而言，一个输出 **64-bit word** 由 16 个输入 **word** **XOR** 得到，因此每个输出 **bit** 需要 15 次 **CNOT** 合成；全部输出需要

$$32 \cdot 64 \cdot 15 = 30720$$

个 **bit XOR** 合成操作。该层是线性的，所以它不是 **Toffoli** 成本来源；但它非常宽，会增加 **CNOT** 体积、布线压力和与前后非线性层之间的依赖展开。我们设计它的目的不是构造 **MDS** 线性层，而是在 **round-subkey vector** 上形成 **bit-uniform parity distribution**，使单点扰动在 **bit lane** 上均匀进入后续状态相关 F 函数。

1.8 复杂性设计的结构化收益

我们不声称复杂性自动推出安全性。我们声称 Type-2 的复杂性是结构化的：每个复杂部件都能对应到具体数学对象、具体门级成本和具体分析阻力。传统差分分析必须同时携带 $\mathbb{F}_p$ 上的 $M\Delta x$ 、TDOM 的非线性状态差分、 $\mathbb{Z}/2^{64}\mathbb{Z}$ 的矩阵轨道、BitMatrix 的 bit parity 投影和 $\mathbb{Z}/2^{32}\mathbb{Z}$ 模加 carry；传统线性分析必须同时处理素域线性掩码、布尔层掩码、旋转掩码、表查找掩码和 carry 传播；量子攻击者若要构造 oracle，则必须把主子密钥硬化、矩阵状态更新、动态轮密钥生成、状态相关矩阵访问和数据轮加密全部做成可逆线路，并为清理中间垃圾寄存器支付反算成本。

我们知道上述论证仍然不是完整 IND-CPA/PRP 归约，也不是对理想无限资源量子机器的神话式防御声明。本文的可验证结论是更具体的：若攻击者试图在现实容错量子计算机上实现 Type-2 OPC 的攻击 oracle，则 key schedule 和 round function 不会坍缩成低成本静态 S-box 网络；它们会形成包含 $\mathbb{F}_p$ 运算、 $\mathbb{Z}/2^{64}\mathbb{Z}$ 矩阵乘法、可逆查表、宽 CNOT、Toffoli-rich 布尔层、数据相关旋转和模加 carry 的门级组合。我们把这种组合称为“工程上不友好的量子攻击电路目标”：它不保证理论上不可攻击，但它迫使攻击从抽象查询次数进入具体线路资源账本。

1.9 复杂性不是安全幻觉的局部证据

我们用局部数学性质证明每个复杂部件承担明确职责。主子密钥硬化的裸核碰撞满足

$$M(x - x') \equiv 0 \pmod{p},$$

短差分碰撞回到 SIS/Ajtai 型短关系；加入 TDOM 后，差分变为

$$\Delta H = M\Delta x + \text{TDOM-Sponge}(y + M\Delta x) - \text{TDOM-Sponge}(y) \pmod{p},$$

从而不能被压回单一线性核。Kronecker 注入满足

$$K_{\otimes} = vu, \quad \delta = \langle v, u \rangle, \quad T = A(\delta vu) = \delta(Av)u,$$

它确实是 collapsed / outer-product / rank-at-most-one-style 的畸形窄映射，不是普通满熵矩阵随机化；但它的设计位置不是生成满熵矩阵，而是把 SDP 材料注入一个方向性状态轨道，然后交给后续非交换矩阵表达式和 F 函数放大。BitMatrix 满足每个输出 bit 是 16 个输入 bit 的 parity，每个输入 bit fan-out 到 16 个输出 bit；PHT 与 Lai-Massey 保证数据路径可逆；ByteSubstitution 的正逆表保证字节层可逆。我们把这些局部性质合成一条职责链，用数学对象回答“为什么存在这一层”，而不是用复杂度本身替代论证。

1.10 总-分-总的论文组织逻辑

我们先在本节给出设计总体、量子攻击电路成本和传统分析阻力；随后正文逐层展开每个实现文件对应的数学对象、伪代码、设计灵感和可证明性质；最后再回到端到端正确性与后续分析对象。这样安排可以避免两种错误：一是只给源码而没有论文论证，二是只给抽象公式而让实现路径变成黑盒。

2 研究对象、算法边界与写作原则

我们把研究对象固定为当前 /OOP/BlockCipher/ 文件夹中的 Type-2 分组密码实现。我们不把 /Template/ 旧实现、/StreamCipher/ Type-1 小 OPC、C API wrapper、测试文件或外部应用层 KDF 当成本文核心算法的一部分。我们保留这些边界，不是为了删减算法，而是为了防止论文把历史实验路径和当前 Type-2 core 混在一起。

我们知道当前实现由下列源文件共同定义：Modules_OaldresPuzzle_Cryptic.hpp 定义公共状态；CustomSecureHash.hpp 定义海绵哈希；ExtraIncludes/MontgomeryScalarEigen.hpp 定义 $p = 2^{64} - 59$ 上的 Montgomery 素域算术；Module_SecureSubkeyGeneratation.* 定义 SIS/Ajtai 形式主子密钥硬化；Module_MixTransformationUtil.* 定义 32 位门级混合与 expansion；Module_SubkeyMatrixOperation.* 定义矩阵初始化、IV 注入和状态更新；Module_SecureRoundSubkeyGeneratation.* 定义轮密钥矩阵、BitMatrix 扩散、PHT 变换和状态相关 F 函数；OaldresPuzzle_Cryptic.* 定义 Lai-Massey 数据轮和静态字节替换；OPC_MainAlgorithm_Worker.* 定义分块、padding 和 stateful worker 边界；Includes/PRNGs.hpp 给出 LFSR、NLFSR、SDP、ISAAC 等状态材料来源。

本文采用实现路径到数学对象的对应组织。每个核心实现文件对应一个明确的密码学对象：先给出代码位置和执行对象，再给出设计灵感，然后给出数学定义，最后给出可证明性质与分析范围。若同一结构在不同代码路径中重复出现，正文只保留一次完整论述，并在后续位置引用该定义；若某一段旧伪代码只说明了执行顺序而没有数学对象，正文将其改写为公式、定义、命题和证明。这样组织的目标是让复杂性可以被审计，而不是让复杂性作为黑盒存在。

我们采用正文优先的论文写法：源码仍作为核对材料保留，但正文不能把源码列表当作论证。对每一个实现文件，我们把代码中的状态变量、循环、表、常量和返回值翻译成数学对象；对每一个数学对象，我们说明我们设计它的必要性；对每一个可证明性质，我们给出

证明或证明草图；对不能由代码本身证明的密码安全主张，我们明确写成设计动机或待分析问题。这样处理后，正文不依赖外部源码列表才能完成理解，也不会让实现细节脱离数学定义和设计论证。

表 2: 当前 /OOP/BlockCipher/ 源码到论文对象的覆盖表

源文件	代码对象	论文中的数学对象
Modules_OaldresPuzzle_Cryptic.hpp	CommonStateData	参数 B, K, N ，矩阵状态 A, T ，shuffle index π ，PRNG 指针。
ExtraIncludes/MontgomeryScalarEigen.hpp	Montgomery64	$\mathbb{F}_p$ 上的无除法矩阵-向量乘法。
CustomSecureHash.hpp	CustomSecureHash	rate/capacity 白盒海绵、固定表、 χ -like 状态变换、吸收/挤出。
TDOM_HashModule --- SecureCustomHash.md	常量来源	4096-bit prime、平方根小数、十六进制常量链。
Module_SecureSubkeyGeneratation.*	TDOM_HashModule, GenerationSubkeys	$H_M(x) = Mx + \text{Sponge}(Mx) \bmod p$ 。
Module_MixTransformationUtil.*	WordBitRestruct, Word32Bit_ExpandKey	32 位 bit permutation、六变量 χ -like 层、PHT expansion。
Module_SubkeyMatrixOperation.*	InitializationState, UpdateState	矩阵填充、NLFSR 仿射广播、 $T = A(\delta vu)$ 。
Module_SecureRoundSubkeyGeneratation.*	OPC_MatrixTransformation, GenerationRoundSubkeys	$G \leftarrow G + (T^\top - A)(A^\top + T)AT$ 、whitening、 $D \otimes I_{64}$ 。
Module_SecureRoundSubkeyGeneratation.*	ForwardTransform, BackwardTransform, CrazyTransformAssociatedWord	PHT 正逆变换、状态相关 F 函数。
OaldresPuzzle_Cryptic.*	LaiMasseyFramework, ByteSubstitution, RoundFunction	字级 Lai–Massey 置换、静态 S-box 层、 $16 \times N^2$ word update。
OPC_MainAlgorithm_Worker.*	EncrypterMain, SplitDataBlockToEncrypt	stateful worker、padding、byte/word packing、core 调用边界。
Includes/PRNGs.hpp	LFSR/NLFSR/SDP/ISAAC 等	状态材料、索引洗牌、离线常量生成背景。

3 总体结构、总伪代码与复杂性设计动机

我们首先给出算法的总览，然后进入各模块分解，最后再回到端到端闭环。本文后续各节均服从这种“总–分–总”的组织：先说明 Type-2 core 的整体路径，再分别展开主子密钥硬化、白盒海绵、矩阵状态、轮密钥、BitMatrix、状态相关 F 函数、Lai–Massey 可逆层和 worker 边界，最后汇总其正确性与分析范围。这样写的原因是，当前算法不是一个可以用单轮伪代码概括的普通 Feistel 或 SPN，而是一个状态化 key schedule 与可逆数据路径互相咬合的构造。

3.1 整体模块链

我们把 Type-2 core 分成五个连续层级。第一层是输入边界，负责把外部已经给定的 64-bit key words、IV words 和明文块交给 core；第二层是主子密钥硬化，负责把 key words 送入 SIS/Ajtai-form 素域矩阵像与白盒海绵；第三层是矩阵状态轨道，负责把硬化材料推进为 A_t, T_t, G_t, π_t ；第四层是轮密钥生成，负责产生状态相关 round-subkey vector 并通过 whitening 和 BitMatrix 投影；第五层是数据轮函数，负责用 Lai–Massey 框架包住复杂的 F 函数并保证数据路径可逆。

层级	作用
输入层	接收 key words、IV words、message blocks；外部口令派生不属于 Type-2 core。
硬化层	计算 $H_M(x) = Mx + \text{TDOM-Sponge}_h(Mx) \bmod p$ ，把主密钥材料压入 SIS/Ajtai-form 素域结构。
状态层	通过 32-bit expansion、矩阵初始化、NLFSR 变异和 SDP Kronecker 外积注入形成矩阵轨道。
轮密钥层	用 $G \leftarrow G + (T^\top - A)(A^\top + T)AT$ 、whitening 和 $D \otimes I_{64}$ 得到 round-subkey vector。
数据层	对每个 64-bit word 执行状态相关 Lai–Massey update，并在 macro-round 末尾执行静态 byte substitution。

3.2 端到端总伪代码

Algorithm 1 OaldresPuzzle_Cryptic Type-2 core 的端到端总算法

Require: key words $K \in (\mathbb{Z}/2^{64}\mathbb{Z})^K$, initial-vector words IV , 明文 word blocks $P^{(0)}, \dots, P^{(m-1)}$, 参数 $B, K, N = 2K$

Ensure: 密文 word blocks $C^{(0)}, \dots, C^{(m-1)}$ 与推进后的状态 $\mathcal{S}_m$

```
1: 验证尺寸约束:  $B$  为偶数,  $K$  为 4 的倍数,  $K > B$  且  $B \mid K$ 。
2: 将  $K$  输入主子密钥硬化层, 计算  $z = H_M(x) = Mx + \text{TDOM-Sponge}_h(Mx) \bmod p$ 。
3: 对  $z$  执行 32-bit expansion, 填充初始矩阵状态  $A_0, T_0, G_0, \pi_0$ 。
4: 若  $IV$  被使用, 则将  $IV$  展开为 bit 位置扰动并注入  $A_0$ 。
5: for  $b = 0$  to  $m - 1$  do
6:   执行 UpdateState, 得到新的  $A_t, T_t, \pi_t$ 。
7:   执行 GenerationRoundSubkeys, 得到  $g_t \in (\mathbb{Z}/2^{64}\mathbb{Z})^{N^2}$ 。
8:    $X \leftarrow P^{(b)} \in (\mathbb{Z}/2^{64}\mathbb{Z})^B$ 。
9:   for  $r = 0$  to 15 do
10:    for  $i = 0$  to  $N^2 - 1$  do
11:       $j \leftarrow i \bmod B$ 。
12:       $X_j \leftarrow \text{LM}_{g_t, i}(X_j)$ 。
13:    end for
14:     $X \leftarrow \text{ByteSubstitution}(X)$ 。
15:  end for
16:   $C^{(b)} \leftarrow X$ 。
17: end for
18: return  $C^{(0)}, \dots, C^{(m-1)}$  与最终状态。
```

3.3 为什么复杂性不是无用堆叠

我们知道本算法的复杂性不是为了掩盖结构, 而是为了同时增加传统对称分析和量子攻击电路建模的阻力。传统分组密码常见的增强手段是扩大密钥空间、扩大明文/密文空间、增加轮数或替换更强 S-box; 这些手段仍然有价值, 但在量子攻击背景下, 攻击者还必须把目标算法嵌入可查询 oracle 或量子线路, 才能执行 Simon 型、Grover 型、量子碰撞或相关量子攻击框架 [24, 11, 12, 9, 19]。因此, 本文的设计目标不止是“对称加密解密正确”, 也不止是“把 key space 放大”。我们还希望让攻击者在构造量子攻击电路、展开 key schedule、固定 oracle 接口、追踪状态矩阵和分析轮密钥依赖时面对高度不友好的结构。

这种不友好不是不可分析的黑盒复杂度, 而是由不同代数域和不同依赖路径共同形成。SIS/Ajtai 层把主密钥差分推入素域短关系约束; 白盒海绵把素域像送入公开但非线性的状态机; 32-bit expansion 把 word 局部性打散; 矩阵状态层使用 $\mathbb{Z}/2^{64}\mathbb{Z}$ 溢出语义和非交换矩阵表达式; Kronecker 外积注入把 SDP 材料压成 collapsed direction; BitMatrix 在 bit 层实现 16-of-32 parity distribution; 状态相关 F 函数又把数据、word key、轮密钥矩阵单元、自 mask 和模加 carry 绑在一起。攻击者若要把该结构压成量子 oracle, 就必须同时保留这些状态依赖、条件选择、矩阵更新、查表、位掩码和 carry 链, 而不能只把算法写成固定低深度 S-box 网络或静态 SPN。

我们并不把“量子电路难搭”写成完整安全证明。本文能够给出的严谨结论是: 当前设计显著增加了 oracle 展开所需的状态变量、门级依赖层、word-ring/bit-vector/prime-field 混合边界和轮密钥状态轨道; 这些结构使得传统差分、线性、代数和量子电路分析都必须面对多域耦合, 而不是只面对一个静态轮函数。完整的量子查询下界、T-count 成本、Toffoli 深度、相关密钥模型和状态恢复复杂度仍属于后续分析对象。

3.4 总-分-总的正文组织

我们在后文采用三段式展开。第一组章节给出全局边界、符号、总体伪代码和可证明性质与分析对象; 第二组章节逐层展开各实现文件的数学对象和详细伪代码; 第三组章节回到端到端正确性、复杂性设计哲学和可分析边界。这样安排可以避免两个极端: 一边是只给总图却隐藏实现细节, 另一边是把 C++ 源码直接粘贴进论文而没有数学解释。本文采用的是中间路径: 每个关键代码对象都有伪代码, 每个伪代码都有数学定义, 每个数学定义都说明其设计角色。

4 缩写、数学符号与运算语义

我们先给出全文使用的缩写和数学符号。我们这样处理, 是因为 /OOP/BlockCipher/ 当前实现同时包含素域矩阵哈希、word-ring 矩阵状态、bit-vector 布尔代数、状态化轮密钥矩阵和分组密码轮函数; 若不先定义缩写和符号, 后续的 SIS/Ajtai、BitMatrix、Kronecker 外积注入和 Lai-Massey 可逆性会被误归入同一类运算。

4.1 缩写定义

表 3: 本文使用的主要缩写

缩写	全称或代码对象	在本文中的含义
OPC	OaldresPuzzle_Cryptic	本文研究的 Type-2 分组密码核心。
Type-2	Type-2 block cipher core	当前 /OOP/BlockCipher/ 路径中的状态化大 OPC 分组密码构造。
OOP	Object-Oriented Programming implementation	当前论文只对应面向对象实现路径，不对应 Template 旧路径。
SIS	Short Integer Solution	用于说明 $Mx \equiv 0 \pmod{p}$ 的短向量碰撞约束。
ISIS	Inhomogeneous Short Integer Solution	用于说明指定输出差分 $M\Delta x = \eta \pmod{p}$ 时的非齐次短解约束。
LWE	Learning With Errors	在噪声和错误中学习；用于说明线性像加误差后的高维离散点云与抗量子格问题背景。
Ajtai	Ajtai-form matrix hash	指 $x \mapsto Mx \bmod p$ 的素域矩阵像哈希核心。
TDOM	TDOM_HashModule	主子密钥硬化中承载 SIS/Ajtai core 与白盒海绵的模块。
Sponge	CustomSecureHash	白盒自定义海绵函数，不是未知 random oracle。
PHT	Pseudo-Hadamard Transform	32 位字对上的加法型可逆混合变换。
LM	Lai-Massey	数据字轮函数采用的可逆分组密码框架。
S-box	Substitution box	OaldresPuzzle_Cryptic 中的静态 byte substitution 表。
IV	Initial Vector	可选状态注入材料，不等同于外部 KDF。
PRNG	Pseudo-Random Number Generator	LFSR、NLFSR、SDP、ISAAC 等状态材料来源。
CSPRNG	Cryptographically Secure PRNG	文中仅对代码角色进行说明，不把整个 Type-2 安全性外包给单个生成器。
LFSR	Linear Feedback Shift Register	线性、低成本、可分析的状态材料生成器。
NLFSR	Nonlinear Feedback Shift Register	非线性门控反馈状态材料生成器。
SDP	Simulated Double Pendulum	双摆连续系统采样源，用于构造扰动向量和矩阵材料。
BitMatrix	$M_{\text{bit}} = D \otimes I_{64}$	作用于 2048-bit 展平向量的 bit-uniform parity distribution layer。
KDF	Key Derivation Function	外部口令派生不属于本文 Type-2 core。

4.2 数学符号定义

表 4: 本文使用的数学符号

符号	定义	说明
$\mathbb{F}_2$	$\mathbb{F}_2$	bit 级线性空间；XOR 是加法。
$\mathbb{Z}/2^{32}\mathbb{Z}$	$\mathbb{Z}/2^{32}\mathbb{Z}$	32 位 unsigned word 溢出环。
$\mathbb{Z}/2^{64}\mathbb{Z}$	$\mathbb{Z}/2^{64}\mathbb{Z}$	64 位 unsigned word 溢出环。
$\mathbb{F}_p$	$\mathbb{F}_p, p = 2^{64} - 59$	Montgomery 素域层使用的代数空间。
q	$q = p$ 或一般格问题模数	SIS/LWE 空间定义中的整数模数。
$\Lambda_q^\perp(M)$	$\{z \in \mathbb{Z}^N : Mz \equiv 0 \pmod{q}\}$	由矩阵 M 定义的 q -ary 正交格。
χ	错误分布	LWE 中采样误差向量的分布。
B	OPC_QuadWord_DataBlockSize	一个数据块包含的 64 位 word 数。
K	OPC_QuadWord_KeyBlockSize	一个 key block 包含的 64 位 word 数。
N	$2K$	key matrix 的维度。
A_t	RandomQuadWordMatrix	主矩阵状态，元素在 $\mathbb{Z}/2^{64}\mathbb{Z}$ 中解释。
T_t	TransformedSubkeyMatrix	经状态更新得到的变换子密钥矩阵。

符号	定义	说明
G_t	<code>GeneratedRoundSubkeyMatrix</code>	当前轮密钥矩阵。
g_t	<code>vec(G_t)</code>	轮密钥矩阵展平后的 64 位 word 向量。
π_t	<code>MatrixOffsetWithRandomIndices</code>	被洗牌的矩阵行列索引映射。
M	$M \in \mathbb{F}_p^{N \times N}$	SIS/Ajtai 主子密钥硬化矩阵。
x	$x \in \mathbb{F}_p^N$	进入主子密钥硬化的 key-derived 向量。
y	$Mx \bmod p$	裸 Ajtai 矩阵像。
$H_M(x)$	$Mx + \text{Sponge}(Mx) \bmod p$	当前实现中的白盒硬化矩阵哈希。
u	$u \in \mathbb{Z}/2^{64}\mathbb{Z}^{1 \times N}$	SDP 生成的行向量。
v	$v \in \mathbb{Z}/2^{64}\mathbb{Z}^{N \times 1}$	SDP 生成的列向量。
δ	$\langle v, u \rangle = \sum_i v_i u_i$	Kronecker 外积注入中的内点积标量。
D	$D \in \mathbb{F}_2^{32 \times 32}$	hard-coded XOR ledger 的 word-index incidence matrix。
M_{bit}	$D \otimes I_{64}$	真实作用在 bit vector 上的 2048×2048 BitMatrix。
$F(a, k)$	<code>CrazyTransformAssociatedWord</code>	Lai-Massey 调用的状态相关 F 函数。
H, H^{-1}	PHT 正逆变换	32 位字对的可逆混合层。

4.3 代数空间和运算语义

我们固定四种运算语义。第一, XOR、AND、OR、NOT 和 bit 选择属于 Boolean bit-vector 语义; 在 $\mathbb{F}_2$ 上, XOR 是加法, 但 AND/OR/NOT 不是 $\mathbb{F}_2$ 线性映射。第二, `uint32_t` 加减、PHT 和 `CrazyTransformAssociatedWord` 的最终加法属于 $\mathbb{Z}/2^{32}\mathbb{Z}$ 。第三, 矩阵状态 A_t, T_t, G_t 的加、减、乘属于 $\mathbb{Z}/2^{64}\mathbb{Z}$ 的 `unsigned overflow` 语义; 此处没有不可约多项式约简, 因此不是 $GF(2^{64})$ 乘法。第四, `MontgomeryScalarEigen.hpp` 只在 $p = 2^{64} - 59$ 上实现 $\mathbb{F}_p$ 运算, 服务于 $Mx \bmod p$ 这一主子密钥硬化核心。

定义 4.1 (Type-2 参数). 我们令

$$B = \text{OPC_QuadWord_DataBlockSize}, \quad K = \text{OPC_QuadWord_KeyBlockSize}, \quad N = 2K.$$

其中 B 是一个数据块包含的 64 位 word 数, K 是一个 key block 包含的 64 位 word 数, N 是 key matrix 的行列数。代码要求 $B \geq 2$ 且 B 为偶数; 要求 $K \geq 4$ 且 $K \equiv 0 \pmod{4}$; 并要求 $K > B$ 且 $K \equiv 0 \pmod{B}$ 。

定义 4.2 (公共状态). 我们把 `CommonStateData` 的核心状态写成

$$A_t \in \mathbb{Z}/2^{64}\mathbb{Z}^{N \times N}, \quad T_t \in \mathbb{Z}/2^{64}\mathbb{Z}^{N \times N}, \quad \pi_t \in S_N,$$

其中 A_t 对应 `RandomQuadWordMatrix`, T_t 对应 `TransformedSubkeyMatrix`, π_t 对应 `MatrixOffsetWithRandomIndices`。轮密钥模块还维护

$$G_t \in \mathbb{Z}/2^{64}\mathbb{Z}^{N \times N}, \quad g_t = \text{vec}(G_t) \in \mathbb{Z}/2^{64}\mathbb{Z}^{N^2}.$$

我们知道这些对象的混合层级不同。 π_t 是 NLFSR 洗牌产生的索引置换; A_t, T_t, G_t 的矩阵加减乘在代码层是 `uint64` 溢出语义; 只有 `TDOM_HashModule::SecureHash` 通过 `Montgomery64` 把矩阵-向量乘法提升到 $\mathbb{F}_p$ 。因此本文凡是写 $+$ 、 $-$ 、 $\cdot$ 、 $\oplus$ 的地方都会说明所在空间: $\mathbb{F}_p$ 中的 $+$ 是素域加法, $\mathbb{Z}/2^{64}\mathbb{Z}$ 中的 $+$ 是 `unsigned carry` 链, $\mathbb{F}_2$ 中的 $+$ 就是 XOR。

说明 4.1 (为什么必须这样分空间). 我们知道当前算法的设计灵感之一, 是把不同代数结构的弱点互相打断。SIS/Ajtai 层依赖素域线性像, BitMatrix 层依赖 $\mathbb{F}_2$ 上的 parity, CrazyTransform 层使用 Boolean 门和 $\mathbb{Z}/2^{32}\mathbb{Z}$ 模加, Lai-Massey 层依赖 XOR 不变量。若把它们全部合并成一个域, 论文会变短, 但代码中的乘法、XOR、rotate 会被误归入同一个代数对象; 这正是当前论文必须主动澄清的地方。

4.4 缩写、代码名和数学符号的排版约定

我们在全文中把代码名字、路径和函数名统一写成等宽字体, 例如 `Module_SecureRoundSubkeyGeneratation.cpp`、`CrazyTransformAssociatedWord`、`Word32Bit_ExpandKey`。我们这样处理, 是因为代码名中的下划线不是数学下标, 若把它们直接放进正文, LaTeX 会把 `_` 解释成下标符号, 从而破坏排版和语义。代码名只表示实现对象, 数学符号才表示代数对象。

我们把数学符号分成四个层次。第一层是 bit 向量空间 $\mathbb{F}_2^n$, 用于 BitMatrix、XOR ledger、Boolean ANF 和单 bit 传播证明。第二层是 word 环 $\mathbb{Z}/2^{32}\mathbb{Z}$ 与 $\mathbb{Z}/2^{64}\mathbb{Z}$, 用于 C++ `unsigned overflow`、模加、模减、旋转和 word-ring 矩阵状态。第三层是素域 $\mathbb{F}_p$, 仅用于 Montgomery 标量、SIS/Ajtai 矩阵哈希和 $p = 2^{64} - 59$ 上的向量计算。第四层是实现状态空间 $\mathcal{S}$, 它把 A_t, T_t, G_t, π_t 和 PRNG 状态对象组合起来。我们不把这四层合并, 因为合并会让同一个符号同时承担 XOR 线性、word 溢出和素域乘法三种不兼容语义。

定义 4.3 (符号生命期). 我们称一个符号具有生命期, 是指它在算法路径中从生成、使用到擦除或覆盖的范围。主子密钥向量 x 的生命期止于 $H_M(x)$ 与后续 **expansion**; 矩阵状态 A_t, T_t 的生命期跨越多次轮密钥生成; 轮密钥向量 g_t 的生命期止于当前数据块或当前轮函数调度; 数据字 X_i 的生命期跨越 Lai-Massey macro-round; PRNG 输出材料只作为状态注入材料, 不作为直接 **keystream** 输出。

我们用这种符号生命期区分核心与外框架。外部 **password-to-key** 或 **KDF** 若存在, 只负责把人类输入变成 **Type-2 core** 接收的 64 位 **key words**; 从 $x \in \mathbb{F}_p^N$ 进入 H_M 开始, 才属于本文讨论的 **Type-2** 密码核。这样写以后, 论文不会把外部工程经验误写成密码原语, 也不会把 **Type-2 core** 的状态化路径切断。

5 公共状态对象: Modules_OaldresPuzzle_Cryptic.hpp

5.1 我们定义的实现对象

我们在 **CommonStateData** 中看到, **Type-2 core** 初始化时同时建立三类对象: 第一类是尺寸参数 B, K, N ; 第二类是 PRNG 指针, 包括 **LFSR**、**NLFSR** 和 **SDP**; 第三类是算法状态, 包括 A_t 、 T_t 、 π_t 、初始向量和主密钥 **word vector**。构造函数把 N 固定为 $2K$, 把 A_t, T_t 初始化为 $N \times N$ 零矩阵, 把 π_t 初始化为 $0, 1, \dots, N-1$ 。

5.2 我们设计这一层的灵感

我们设计公共状态时, 不把分组密码写成“同一个主密钥派生出固定轮密钥表”的形式。我们希望每次加密或解密调用都推动内部 **key state**, 因此 **worker** 构造器中特别提示: 调用加密或解密后, 内部 **key state** 会改变; 若要反向操作, 应销毁当前实例并重建。我们这样做的动机, 是让穷举者面对的不是静态 **key schedule**, 而是一个由主密钥、状态矩阵、PRNG 材料和数据处理顺序共同决定的状态化路径。

5.3 我们使用的数学定义

我们把 **CommonStateData** 看成状态空间

$$\mathcal{S} = \mathbb{Z}/2^{64}\mathbb{Z}^{N \times N} \times \mathbb{Z}/2^{64}\mathbb{Z}^{N \times N} \times S_N \times \{0, 1\}^{32m} \times \mathbb{Z}/2^{64}\mathbb{Z}^K,$$

其中 m 是初始字节向量转换成 32 位 **word** 后的长度。一次状态更新不是单独作用在 A_t 上, 而是作用在 (A_t, T_t, π_t) 上:

$$(A_t, T_t, \pi_t) \mapsto (A_{t+1}, T_{t+1}, \pi_{t+1}).$$

这里 π_{t+1} 来自 **ShuffleMatrixOffsetWithRandomIndices**, 也就是用 **NLFSR** 对 $0, \dots, N-1$ 做洗牌。

5.4 我们能够直接证明的性质

命题 5.1 (索引洗牌的作用范围). 只要 π_t 是 $\{0, \dots, N-1\}$ 的排列, 则 **CrazyTransformAssociatedWord** 中

$$\text{row} = \pi_t(A \bmod N), \quad \text{col} = \pi_t(B \bmod N)$$

永远访问 G_t 的合法行列。

证明. $A \bmod N$ 与 $B \bmod N$ 属于 $\{0, \dots, N-1\}$ 。若 π_t 是该集合上的排列, 则输出仍在该集合内。因此 $G_t[\text{row}, \text{col}]$ 的索引合法。代码中 π_t 初始为完整顺序数组, 之后只通过 **shuffle** 改变顺序, 不改变元素集合。□

6 Montgomery 素域层: ExtraIncludes/MontgomeryScalarEigen.hpp

6.1 我们定义的实现对象

我们在 **MontgomeryScalarEigen.hpp** 中看到, **SIS/Ajtai** 层并不是在 **uint64** 溢出环里做 Mx , 而是用 **MontgomeryPrimeField-Context**、**MontgomeryComputationScope** 和 **Montgomery64** 给 **Eigen** 注入自定义标量。代码先计算 $n' = -p^{-1} \bmod 2^{64}$ 和 $R^2 \bmod p$, 其中 $R = 2^{64}$, 再用 **REDC** 把 128 位乘积规约回 $[0, p)$ 。

6.2 我们设计这一层的灵感

我们知道 SIS/Ajtai 形式的核心是素域矩阵-向量乘法。若直接使用 `uint64` 溢出乘法, Mx 会落在 $\mathbb{Z}/2^{64}\mathbb{Z}$ 而不是 $\mathbb{F}_p$, 这会破坏“短整数解”语境中的模素数线性结构。我们因此把 Mx 独立放在 Montgomery 素域层, 让 Eigen 的矩阵乘法仍然可用, 但每一个标量乘加都变成 $\mathbb{F}_p$ 运算。

6.3 我们使用的数学定义

我们令 $p = 18446744073709551557 = 2^{64} - 59$ 。Montgomery 表示把普通余数 $x \in \mathbb{F}_p$ 映为

$$\tilde{x} = xR \bmod p, \quad R = 2^{64}.$$

两个 Montgomery 元素相乘时, 代码计算

$$\text{REDC}(\tilde{x}\tilde{y}) = xyR \bmod p,$$

因此乘法结果仍保持 Montgomery 表示。出域时再计算 $\text{REDC}(\tilde{x}) = x$ 。

命题 6.1 (Montgomery 乘法保持素域语义). 若 $\tilde{x} = xR \bmod p$ 且 $\tilde{y} = yR \bmod p$, 则 Montgomery64 的乘法返回 $xyR \bmod p$ 。

证明. Montgomery 规约满足 $\text{REDC}(z) = zR^{-1} \bmod p$ 。令 $z = \tilde{x}\tilde{y} = xyR^2 \bmod p$, 则

$$\text{REDC}(z) = xyR \bmod p.$$

因此结果仍是 xy 的 Montgomery 表示。加法和减法只在同一表示中做条件规约, 所以同样保持 $\mathbb{F}_p$ 语义。 □

7 自定义海绵哈希: CustomSecureHash.hpp 与 TDOM_HashModule --- SecureCustomHash.md

7.1 我们定义的白盒海绵实现对象

我们知道这里最容易被写错: 自定义海绵哈希不是一个只用外部安全假设支撑的未展开对象 `Sponge(·)`。在当前实现中, `CustomSecureHash.hpp` 明确给出状态长度、rate/capacity、padding、吸收、挤出、状态变换、固定索引表、旋转量表和轮常量; `TDOM_HashModule --- SecureCustomHash.md` 则给出 `HASH_ROUND_CONSTANTS` 的来源过程。因此论文中出现的 `Sponge` 只是为了在主公式中避免重复写长状态机; 它的含义必须展开为本节定义的白盒函数 `TDOM-Spongeh`, 而不是“未知安全 oracle”。

我们令目标哈希位数为 $h = \text{HashBitSize}$ 。当前 Type-2 主子密钥模块中, 构造器设置

$$h = 32N, \quad N = \text{OPC_KeyMatrix_Rows},$$

因此 rate 以 64-bit word 对齐。CustomSecureHash 的几何参数为

$$b = 2h + 64, \quad r = h, \quad c = b - r = h + 64,$$

其中 b 是状态比特数, r 是吸收/挤出的 rate, c 是 capacity。以 64-bit word 表示时, 令

$$W = \frac{b}{64}, \quad R = \frac{r}{64}, \quad C = \frac{c}{64}, \quad H = \left\lfloor \frac{W}{2} \right\rfloor.$$

状态是

$$S = (S_0, S_1, \dots, S_{W-1}) \in (\mathbb{Z}/2^{64}\mathbb{Z})^W.$$

我们在这里故意同时保留 $\mathbb{Z}/2^{64}\mathbb{Z}$ 和 $\mathbb{F}_2^{64}$ 两种视角: word 的存储和轮常量是 64-bit word; XOR、NOT、AND、固定 rotation 和固定 index permutation 可以解释为 64 条 bit lane 上的布尔网络。这个区分很重要, 因为它说明该哈希不是“把字当整数随便搅”, 而是一个 word-parallel Boolean sponge state machine。

7.2 我们设计这一层的灵感: 为什么我们把 SIS/LWE 空间再送入白盒海绵

我们把这一层的设计问题严格化为一个空间组合问题: 如果能够产生一个来自 SIS/Ajtai 与 LWE 语境的高维离散格点空间, 并将这个空间定义和离散格哈希化, 会发生什么? 进一步地, 如果输入先落入原来的格结构, 再以对称密码的白盒置换方式继续搅拌, 那么输出空间是否仍然只是一个普通线性像, 还是会变成一个被 twist 扭曲过的高维离散图像空间? 我们采用这个结构, 不是因为“矩阵乘法加哈希”在文字上显得复杂, 而是因为它把两个在分析坐标上差异很大的空间放到同一条可实现路径中: 前向构造只需要矩阵乘法、白盒 sponge 和模加; 反向分析却必须同时处理离散格、素域线性像、word-parallel 布尔状态机、模加扰动和图像空间。

我们先把 SIS/Ajtai 空间写成裸骨架。令

$$q = p = 2^{64} - 59, \quad M \in \mathbb{Z}_q^{N \times N}, \quad x \in \mathbb{Z}_q^N,$$

并定义 Ajtai-form matrix hash core

$$h_M(x) = Mx \bmod q.$$

若输入限制在有界整数盒

$$\mathcal{B}_\beta = \{x \in \mathbb{Z}^N : \|x\|_\infty \leq \beta\},$$

则裸核的碰撞关系为

$$h_M(x) = h_M(x') \iff M(x - x') \equiv 0 \pmod{q}.$$

我们因此得到 q -ary 正交格

$$\Lambda_q^\perp(M) = \{z \in \mathbb{Z}^N : Mz \equiv 0 \pmod{q}\},$$

其中

$$z = x - x' \in \Lambda_q^\perp(M)$$

是由输入界控制的短向量。也就是说，裸 SIS/Ajtai 层把“找碰撞”写成“在 q -ary 格中找短核向量”。这个对象是干净的、线性的、可命名的，也是攻击者最想保留的分析坐标：只要问题停留在 Mx ，所有差分 and 线性掩码都能回到矩阵 M 的核空间、像空间和转置空间。

我们再写 LWE（Learning With Errors，在噪声和错误中学习）给出的空间直觉。一般 LWE 样本为

$$A \in \mathbb{Z}_q^{m \times n}, \quad s \in \mathbb{Z}_q^n, \quad e \leftarrow \chi^m,$$

$$b = As + e \bmod q.$$

若没有误差 e ，点 b 位于线性像空间 $\text{Im}(A)$ ；加入误差后，输出变成围绕该线性像的离散噪声点云

$$\mathcal{C}_{A,\chi}(s) = \{As + e \bmod q : e \in \text{supp}(\chi)^m\}.$$

我们没有把 LWE 样本直接作为 Type-2 的加密原语；我们采用的是同一类空间问题：先得到可命名的高维线性/格结构，再加入一个扰动机制，使攻击者不能只在原来的线性坐标中完成差分、线性或碰撞分析。LWE 的扰动来自随机错误分布 χ ；Type-2 主子密钥硬化的扰动来自公开确定的白盒 TDOM 状态机在 $\text{Im}(M)$ 上诱导的非线性位移场。

我们把“把格空间哈希化”写成可审查的组合映射。定义 SIS/Ajtai 裸核像空间

$$\mathcal{L}_M = \text{Im}(M) = \{Mx : x \in \mathbb{Z}_q^N\} \subseteq \mathbb{Z}_q^N.$$

令

$$\phi : \mathbb{Z}_q^N \rightarrow (\mathbb{Z}/2^{64}\mathbb{Z})^N$$

表示代码中的 64-bit word 序列化，令

$$\psi : (\mathbb{Z}/2^{64}\mathbb{Z})^N \rightarrow \mathbb{Z}_q^N$$

表示把海绵输出重新规约为 q 模向量。白盒海绵诱导确定函数

$$S_h(y) = \psi(\text{TDOM-Sponge}_h(\phi(y))).$$

当前主子密钥硬化层不是单纯输出 Mx ，而是

$$H_M(x) = \tau_h(Mx), \quad \tau_h(y) = y + S_h(y) \bmod q.$$

因此进入后续 key schedule 的空间不是裸线性像 $\mathcal{L}_M$ ，而是被位移场扭曲后的空间

$$\tau_h(\mathcal{L}_M) = \{y + S_h(y) : y \in \mathcal{L}_M\} \subseteq \mathbb{Z}_q^N.$$

我们把这个对象称为 SIS/Ajtai 离散骨架上的白盒 twist。 M 生成一张高维离散格像， TDOM-Sponge_h 在这张格像上给出一个公开、确定、非线性的位移场，最终 τ_h 把线性骨架推成一张扭曲图像。

我们为了让空间展开能够被直接看见，把整个组合分成四层。第一层是输入盒 $\mathcal{B}_\beta$ ，它描述可接受的 key material 坐标；第二层是格核 $\Lambda_q^\perp(M)$ 与像空间 $\mathcal{L}_M$ ，它描述 SIS/Ajtai 裸核；第三层是白盒位移场 $S_h : \mathcal{L}_M \rightarrow \mathbb{Z}_q^N$ ，它描述 TDOM 在格像上的确定搅拌；第四层是图空间

$$\Gamma_{M,h} = \{(y, \tau_h(y)) : y \in \mathcal{L}_M\} = \{(y, y + S_h(y)) : y \in \mathcal{L}_M\}.$$

普通线性哈希只给出 $y = Mx$ ；本层给出的是 y 与 $y + S_h(y)$ 的成对图像。第一坐标让结构保留 SIS/Ajtai 的可审查骨架，第二坐标把该骨架送入白盒对称置换产生的非线性位移。攻击者如果只看第一坐标，会漏掉海绵扰动；如果只看第二坐标，则必须解释第二坐标如何从第一坐标经过 padding、absorb、state permutation、 χ -like 布尔层、固定 rotation/index schedule、round constants 和模加得到。

我们把差分困难性直接融入这个 **twist** 图像，而不是另写成一个孤立小节。给定输入差分 Δx ，令

$$y = Mx, \quad \Delta y = M\Delta x \pmod{q}.$$

裸 SIS/Ajtai 核的差分为

$$\Delta h_M = h_M(x + \Delta x) - h_M(x) = \Delta y.$$

若攻击者要在裸核中制造零差分，则只需要满足

$$M\Delta x = 0 \pmod{q},$$

这正是 $\Delta x \in \Lambda_q^\perp(M)$ 的齐次短关系。困难性来自 SIS 短向量搜索：在给定 M 的情况下寻找非零短 Δx 使 $M\Delta x = 0$ 。

我们把 Mx 再送入白盒海绵以后，差分不再停留在这条齐次线性方程上。硬化函数满足

$$\begin{aligned} \Delta H_M(x; \Delta x) &= H_M(x + \Delta x) - H_M(x) \\ &= M\Delta x + S_h(y + M\Delta x) - S_h(y) \pmod{q}. \end{aligned}$$

令

$$\Delta_S(y, \Delta y) = S_h(y + \Delta y) - S_h(y),$$

则

$$\Delta H_M = \Delta y + \Delta_S(y, \Delta y) \pmod{q}.$$

零差分条件因此从裸核的

$$\Delta y = 0$$

变成

$$\Delta y = -\Delta_S(y, \Delta y) \pmod{q}.$$

换回输入差分，就是

$$M\Delta x = -(S_h(y + M\Delta x) - S_h(y)) \pmod{q}.$$

这条方程显示了 **twist** 的数学含义：攻击目标不再是固定的齐次核，而是一个基点相关的非齐次移动目标。若记

$$\eta_h(y, \Delta x) = -(S_h(y + M\Delta x) - S_h(y)) \pmod{q},$$

则攻击者必须求解

$$M\Delta x = \eta_h(y, \Delta x) \pmod{q}.$$

它类似 ISIS 形式的非齐次方程，但右端不是预先给定的常量向量，而是由 $y = Mx$ 、 Δx 和白盒海绵差分共同决定的移动向量。我们因此把差分困难性的变化说成：裸 SIS/Ajtai 中的“短核向量搜索”被扭成“基点相关的非齐次移动像追踪”。

我们还可以从空间图像解释这个差分方程。裸空间中， x 与 $x + \Delta x$ 的像差就是一条直线位移 $\Delta y = M\Delta x$ ；若 $\Delta y = 0$ ，两点在 $\mathcal{L}_M$ 中重合。扭曲后，两个点分别变成

$$\tau_h(y) = y + S_h(y), \quad \tau_h(y + \Delta y) = y + \Delta y + S_h(y + \Delta y).$$

两点是否重合，不只取决于 Δy ，还取决于位移场在两处的差

$$S_h(y + \Delta y) - S_h(y).$$

也就是说，攻击者不仅要找到格骨架上的短关系，还要让白盒位移场在两个相关格点上产生精确抵消。这个抵消项不是隐藏 **oracle**，而是 CustomSecureHash.hpp 中完全公开的白盒状态机；困难性来自攻击者必须同时满足格方程和状态机差分方程，而不是来自把细节藏起来。

我们把线性困难性也融入同一个空间描述。对任意线性掩码 $\alpha \in \mathbb{Z}_q^N$ ，裸核满足

$$\langle \alpha, h_M(x) \rangle = \langle \alpha, Mx \rangle = \langle M^T \alpha, x \rangle \pmod{q}.$$

裸 SIS/Ajtai 核并不隐藏这个线性结构；它保留矩阵转置掩码，并把碰撞关系压到 $\Lambda_q^\perp(M)$ 。这不是缺陷，而是裸骨架的作用：它给出可分析、可证明、可定位的格结构。

我们加入白盒海绵后，线性掩码变为

$$\begin{aligned} \langle \alpha, H_M(x) \rangle &= \langle \alpha, Mx + S_h(Mx) \rangle \\ &= \langle M^T \alpha, x \rangle + \langle \alpha, S_h(Mx) \rangle \pmod{q}. \end{aligned}$$

第一项是裸线性掩码；第二项是 α 经过白盒位移场以后限制在 $\mathcal{L}_M$ 上的海绵掩码。若 S_h 是一个固定线性矩阵 R ，则

$$\langle \alpha, S_h(Mx) \rangle = \langle R^T \alpha, Mx \rangle = \langle M^T R^T \alpha, x \rangle,$$

整个表达式仍会塌回一个关于 x 的线性掩码。但当前 S_h 不是 $\mathbb{Z}_q$ 上的固定线性矩阵；它由 64-bit word 序列化、rate/capacity sponge、固定索引赋值、rotation、轮常量、相邻 word mixing 与 χ -like 二次布尔层组成。因此线性分析对象不是单个 $M^T\alpha$ ，而是

$$\mathcal{L}_\alpha(x) = \langle M^T\alpha, x \rangle + \langle \alpha, S_h(Mx) \rangle \pmod{q}.$$

我们把这称为裸线性掩码和海绵掩码的耦合：裸项告诉攻击者线性骨架在哪里，海绵项迫使攻击者在同一个掩码下分析白盒状态机在 $\text{Im}(M)$ 上的非线性响应。

我们进一步把该耦合写成“攻击者必须解释的空间展开”。若攻击者尝试建立仿射近似

$$\langle \alpha, H_M(x) \rangle \approx \langle \beta, x \rangle + c,$$

则必须把

$$\langle \alpha, S_h(Mx) \rangle$$

在 $x \mapsto Mx$ 的像空间上近似为低复杂度线性函数。这个任务同时跨过两个坐标： M 的素域线性坐标和 S_h 的 bit-level 白盒坐标。前者允许转置掩码 $M^T\alpha$ ；后者引入布尔二次项、旋转跨位依赖、赋值覆盖、常量注入和模 q 规约。我们由此得到第二个数学效果：线性掩码不再只沿 M^T 传播，而是被限制在扭曲图空间 $\Gamma_{M,h}$ 上。

我们把这种设计理解为 **LWE** 空间直觉的确定性、白盒化版本。**LWE** 把线性像

$$As$$

推成随机误差点云

$$As + e \pmod{q},$$

而 **Type-2** 主子密钥硬化把 **SIS/Ajtai** 像

$$y = Mx$$

推成确定白盒扰动图像

$$y + S_h(y) \pmod{q}.$$

二者并不相同：本文不声称 $S_h(y)$ 是 **LWE** 噪声，也不声称 H_M 继承 **LWE** 标准归约。我们真正使用的是空间组合思想：先构造一个高维离散线性骨架，再把它送入一个对称密码式白盒搅拌层，使输出空间既保留可审查的格来源，又不再允许攻击者只用裸线性坐标描述差分和线性传播。

我们最后给出构造者-分析者不对称的公式化表达。构造路径是

$$x \xrightarrow{M} y \xrightarrow{\text{TDOM-Sponge}_h} S_h(y) \xrightarrow{+} \tau_h(y) = H_M(x),$$

只需要一次素域矩阵乘法、一次白盒海绵和一次向量模加。分析路径则必须同时解释

$$\Lambda_q^\perp(M), \quad \mathcal{L}_M = \text{Im}(M), \quad \Gamma_{M,h},$$

$$M\Delta x = \eta_h(y, \Delta x), \quad \langle M^T\alpha, x \rangle + \langle \alpha, S_h(Mx) \rangle.$$

这些对象分别位于整数格、素域向量空间、word-parallel Boolean sponge、模加环和扭曲图像空间中。我们选择把 **SIS/LWE** 空间再送入白盒海绵，正是为了把“设计者容易构建的前向路径”和“攻击者难以统一坐标的分析空间”分开；复杂性不是被隐藏的细节，而是被写成可审查公式的空间混合。

我们明确该层的可证明内容。第一，裸核碰撞满足 **SIS/Ajtai** 型短核约束；第二，白盒 **hardening** 后的差分满足基点相关非齐次方程；第三，线性掩码分解为裸转置掩码和海绵掩码耦合；第四，**TDOM-Sponge_h** 是可展开白盒状态机而不是黑盒 **oracle**。我们不把这一层写成整个 **Type-2** 分组密码的 **SIS/LWE** 归约；我们把它写成主子密钥硬化层的数学来源：**SIS/Ajtai** 给出高维离散格骨架，**LWE** 给出线性像加扰动的空间直觉，白盒 **TDOM** 给出确定非线性位移场，最终形成 $\tau_h(\mathcal{L}_M)$ 和 $\Gamma_{M,h}$ 这两个可命名、可推导、可审查的扭曲空间。

我们还需要把安全边界钉得更精确。我们不声称 τ_h 对所有 M 都保持标准 **SIS-hardness**，也不声称 H_M 自动继承 **LWE** 的归约结论。我们主张的是一个更具体的结构阻力：敌手若试图沿裸 **SIS** 核构造短差分或低复杂度线性掩码，就必须同时满足由 **TDOM-Sponge_h** 在 $\text{Im}(M)$ 上诱导的非线性位移抵消条件。换言之，裸格核给出可审查的困难性骨架，白盒海绵给出确定但非线性的 **twist** 位移场；攻击者不能只在其中一个空间里完成分析。

7.3 常量与固定表的来源：SecureCustomHash.md

SecureCustomHash.md 不是无关说明文件。它记录了 **HASH_ROUND_CONSTANTS** 的生成思想：先生成一个 4096-bit 大素数 P_{4096} ，再计算其平方根的高精度小数部分，最后把小数点后的十六进制串切成 64-bit words。实现中固化了 64 个 64-bit 常量

$$K_0, K_1, \dots, K_{63} \in \mathbb{Z}/2^{64}\mathbb{Z},$$

也就是 `HASH_ROUND_CONSTANTS`。这种做法的作用不是给出一个秘密参数，而是给出一个可复现、非低复杂度、非全零、非人为短周期的常量表。

同时，`GenerateRandomMoveBitCounts` 和 `GenerateRandomHashStateIndices` 使用固定种子

$$s_0 = 1946379852749613$$

初始化 `ISAAC64`，并分别丢弃 1024 和 2048 个输出后产生两个公开表：

$$\mu = (\mu_0, \dots, \mu_{62}), \quad \mu_i \in \{1, 2, \dots, 63\},$$

以及

$$\sigma = (\sigma_0, \dots, \sigma_{H-1}), \quad \sigma_i \in \{0, 1, \dots, H-1\}.$$

构造器还从自然序列 $(0, 1, \dots, H-1)$ 生成两个半状态索引表：

$$L_i = i + 1 \pmod{H}, \quad R_i = i - 1 \pmod{H}.$$

因此海绵内部不存在隐藏随机源；所有表都是公开、确定、可复现的常量。我们在论文中把它们写成 K, μ, σ, L, R ，是数学抽象，不是省略实现。

7.4 Padding、吸收和挤出：实现语义

我们定义 `u64` 输入序列 $X = (x_0, \dots, x_{m-1})$ 的 **rate** 对齐 **padding** 为

$$\text{pad}_R(X) = \begin{cases} X, & m \equiv 0 \pmod{R}, \\ X \parallel 1 \parallel 0^{R-(m \bmod R)-1}, & m \not\equiv 0 \pmod{R}. \end{cases}$$

这里的 1 是 `PAD_BITSWORD_DATA=1`。字节稿件完全类似，只是 R 换成 `BYTES_RATE`，padding byte 为 `PAD_BYTE_DATA=1`。注意，当前代码在输入已经 **rate-aligned** 时不额外追加 **domain-separation block**；论文必须按实现写出这个条件 **padding** 语义，而不能把它偷换成标准化海绵 **padding**。

对每个 **rate block**

$$B^{(j)} = (b_0^{(j)}, \dots, b_{R-1}^{(j)})$$

吸收阶段执行

$$S_i \leftarrow S_i \oplus b_i^{(j)}, \quad 0 \leq i < R,$$

然后调用

$$S \leftarrow \mathcal{P}_h^W(S),$$

其中 $\mathcal{P}_h$ 是下面定义的一轮白盒状态变换， $W = |S|$ ，因为代码中每吸收一个块调用 `TransfromState(BitsHashState.size())`。

挤出阶段从 **rate** 槽读出

$$(S_0, S_1, \dots, S_{R-1}),$$

如果输出还不够，则再次执行 $S \leftarrow \mathcal{P}_h^W(S)$ 后继续读出。最后 `SpongeHash` 会清理临时缓冲并调用 `Reset` 清空状态。因此，`CustomSecureHash` 在 **Type-2** 中不是长期有记忆的外部状态源，而是每次调用内部吸收、挤出、清零的一次性白盒海绵。

7.5 一轮状态变换 $\mathcal{P}_h$ 的完整数学展开

我们展开 `TransfromState` 的一轮。给定状态

$$S = (S_0, \dots, S_{W-1}),$$

先初始化三个临时缓冲 $A, B, C \in (\mathbb{Z}/2^{64}\mathbb{Z})^W$ 为零。代码不是“概念上某个随机置换”，而是以下五步赋值网络。

第一步：相邻 word 混合。 对 $0 \leq i < H$ ，代码执行

$$A_i = S_{2i \bmod W} \oplus S_{(2i+1) \bmod W},$$

$$A_{i+H} = S_{(2i+2) \bmod W} \oplus S_{(2i+3) \bmod W}.$$

如果 W 为奇数，则未被上述赋值覆盖的缓冲槽保持零。这一点是实现语义的一部分，不能写成理想化的全覆盖矩阵。

第二步：半状态线性旋转扩散。 对 $0 \leq i < H$ ，令 $L_i = i + 1 \pmod{H}$ ， $R_i = i - 1 \pmod{H}$ ，则

$$B_i = A_{R_i} \oplus \text{rotr}(A_{L_i}, 1),$$

$$B_{i+H} = A_{R_i+H} \oplus \text{rotr}(A_{L_i+H}, 1).$$

这一步仍然是 $\mathbb{F}_2$ 上的线性层：rotation 是 bit 置换，XOR 是线性叠加。

第三步：固定表驱动的 bit pseudo-random permutation。 代码先设置

$$C_0 = S_0 \oplus B_0.$$

随后对 $i = 1, \dots, W - 1$ ，令

$$u_i = \sigma_i \bmod H, \quad \tau_i = \begin{cases} u_i, & i < H, \\ u_i + H, & i \geq H, \end{cases}$$

并取旋转量

$$\rho_i = \mu_{q_i} \bmod 63,$$

其中 q_i 是代码中的 StateCurrentCounter 当前值，随后递增。赋值为

$$C_{\tau_i} \leftarrow \text{rotr}(S_i \oplus B_i, \rho_i).$$

这里必须注意：这是赋值，不是 XOR 累积。若不同 i 命中同一 τ_i ，后写入会覆盖前写入；若某个槽未被命中，则该槽保持零。我们保留这个细节，因为这正是白盒实现和理想置换模型的区别。

第四步： χ -like 二次布尔层。 对 $0 \leq i < W$ ，代码执行

$$S'_i = C_i \oplus ((\sim C_{i+1} \bmod W) \& C_{i+2} \bmod W).$$

这一层和 Keccak 风格的 χ 思想相似：它不是查表 S-box，而是由相邻状态 word 形成的 bit-parallel 二次布尔映射。

第五步：双端轮常量注入。 第 t 轮把 64-word 常量表两端注入：

$$S'_0 \leftarrow S'_0 \oplus K_t \bmod 64,$$

$$S'_{W-1} \leftarrow S'_{W-1} \oplus K_{(63-t) \bmod 64}.$$

这一步是仿射层；它本身不给出非线性，但打破全零固定点、轮间平移对称和前后端同构。

7.6 我们给出的数学解释：白盒语义与标准海绵语义的边界

定义 7.1 (TDOM 白盒海绵). 对给定 h ，我们把由公开常量 K 、公开旋转表 μ 、公开索引表 σ 、公开半状态索引 L, R 和上文五步状态变换定义的函数记为

$$\text{TDOM-Sponge}_h(X) = \text{Squeeze}(\mathcal{P}_h^W \circ \text{Absorb}(\text{pad}_R(X))).$$

它是当前代码中的具体白盒哈希函数，而不是未定义 oracle。

命题 7.1 (白盒可展开性). 对固定 h ，CustomSecureHash.hpp 定义的 SpongeHash 是一个完全确定的有限状态函数；它不依赖运行时秘密常量、外部随机 oracle 或未说明的压缩函数。

证明. 状态大小 W 、rate R 、capacity C 由 h 决定； K 在源码中固定； μ 与 σ 由固定种子 ISAAC64 丢弃固定步数后生成； L, R 由自然序列旋转得到；吸收、状态变换和挤出均由 XOR、AND、NOT、rotation、固定表索引和赋值组成。因此给定输入后，整个输出由有限条确定指令唯一决定。□

命题 7.2 (非线性来源). 若把 word rotation、固定索引赋值、XOR 和常量注入都视为 $\mathbb{F}_2$ 上的线性或仿射操作，则 TransfromState 的主要非线性来源是

$$C_i \mapsto C_i \oplus ((\sim C_{i+1}) \& C_{i+2}).$$

该映射在 ANF 中含二次项 $C_{i+1}C_{i+2}$ 。

证明. XOR、rotation 和固定索引重排都是 $\mathbb{F}_2$ 线性操作；常量注入和 NOT 是仿射操作，因为 $\sim x = x \oplus 1$ 。而 $(\sim y) \& z = (1 \oplus y)z = z \oplus yz$ ，含有二次单项式 yz 。故该层把前面线性/仿射网络提升到二次布尔网络。□

命题 7.3 (多轮 hardening 的结构边界). TDOM-Sponge_h 可以为 $H_M(x) = Mx + \text{TDOM-Sponge}_h(Mx) \bmod p$ 给出白盒非线性 hardening, 但它本节本身不构成“标准海绵安全证明”。

证明. 本节已经把代码展开为具体状态机, 所以我们能证明其确定性、表生成方式、二次非线性来源和吸收/挤出路径; 但随机预言机安全、理想置换安全或抗所有哈希攻击需要额外的差分、线性、代数、状态碰撞和 padding 域分离分析。当前论文因此只把它称为 Type-2 主子密钥硬化中的白盒海绵 hardening layer, 而不声称它已经等价于标准化哈希。□

我们把这一层与海绵/哈希和后量子哈希攻击背景联系起来 [6, 19]。这里引用的目的不是把 CustomSecureHash 冒充成 Keccak 或 SHA-3, 而是说明我们使用的吸收–置换–挤出思想属于海绵式构造语境; 当前实现的每个内部步骤则由本节公式白盒给出。

8 主子密钥硬化: Module_SecureSubkeyGeneratation.*

8.1 我们定义的实现对象

我们在 `TDOM_HashModule::SecureHash` 中看到, 主子密钥硬化分三层: 第一层把随机矩阵 M 和输入向量 x 映入 $\mathbb{F}_p$; 第二层计算裸 Ajtai 像 $y = Mx \bmod p$; 第三层把 y 输入自定义白盒海绵得到 $s = \text{Sponge}(y)$, 再输出 $y + s \bmod p$ 。LatticeCryptographyAndHash 随后把输入 key words 与该哈希向量相加, 得到用于矩阵初始化的 WordKeyResistQC。

8.2 我们设计这一层的灵感

我们知道主密钥材料若直接进入矩阵状态, 就会形成可追踪的线性扩展路径; 若只经过普通 hash, 又很难解释其与后量子困难性的结构关系。因此我们先采用 Ajtai 形式的素域矩阵像 $Mx \bmod p$, 让碰撞搜索自然落到 SIS 型约束; 再把该像送入白盒海绵, 使输出不再是裸线性像。这个设计不是把“hash 套在 SIS 上”当成一句口号, 而是把攻击者面对的对象从单个线性方程组变成

$$\text{素域线性像} + \text{白盒非线性像}$$

的组合对象。

我们设计这一层时刻意把 SIS/Ajtai core 放在 master-subkey hardening, 而不是把整个 Type-2 分组密码强行说成 SIS 问题。主子密钥硬化层处理的是 key-material compression、collision-style 描述和子密钥初始化入口; 数据轮函数、BitMatrix、Lai–Massey 和 byte substitution 仍需要独立的差分、线性、代数和相关密钥分析。

8.3 我们使用的数学定义

设 $M \in \mathbb{F}_p^{N \times N}$, $x \in \mathbb{F}_p^N$ 。代码中的主硬化函数为

$$y = Mx \bmod p, \quad s = \text{Sponge}(y), \quad \boxed{H_M(x) = y + s \bmod p.}$$

LatticeCryptographyAndHash 的输出可写为

$$z_i \leftarrow z_i + x_i \bmod |x| + H_M(x)_i \pmod{p}, \quad 0 \leq i < N.$$

随机矩阵 M 的每个元素由 SDP 采样得到 $u \in [0, 2^{64})$, 代码通过 rejection sampling 要求 $u < T$, 再令 $M_{ij} = u \bmod p$, 其中

$$T = 2^{64} - 1 - ((2^{64} - 1) \bmod p).$$

我们这样写, 是为了明确 M 不是 C++ word-ring 矩阵, 而是 $\mathbb{F}_p$ 矩阵; A_t, T_t, G_t 才是 $\mathbb{Z}/2^{64}\mathbb{Z}$ 语义的矩阵状态。

8.4 SIS/Ajtai 裸核的碰撞困难性

命题 8.1 (裸 Ajtai 核的碰撞约束). 若攻击者在裸函数 $A_M(x) = Mx \bmod p$ 上找到 $x \neq x'$ 且 $A_M(x) = A_M(x')$, 则

$$d = x - x' \neq 0, \quad Md = 0 \pmod{p}.$$

若 d 在预设范数界内足够短, 则该碰撞给出一个 SIS 实例的短解。

证明. 由 $Mx = Mx'$ 得 $M(x - x') = 0 \bmod p$. 令 $d = x - x'$. SIS 问题正是给定矩阵后寻找非零短向量 d 使 $Md = 0 \bmod p$. 因此裸核的碰撞结构和 SIS 约束一致。□

说明 8.1 (Ajtai 形式在本文中的准确位置). 我们使用的是 SIS/Ajtai-form matrix hash core。这个 core 的裸碰撞约束确实是 SIS 型; 但本文不把完整 Type-2 分组密码的 PRP 安全性、IND-CPA 安全性或后量子安全性归约到 SIS。这个边界必须保留, 否则论文会把 key schedule 的结构困难性错写成全算法安全证明。

8.5 与扭曲空间推导的连接

我们已经在前面“为什么我们把 SIS/LWE 空间再送入白盒海绵”中把主子密钥硬化层的差分方程、线性掩码耦合和空间图像统一展开。这里不重复书写同一组推导，只保留当前实现到后续矩阵状态的接口：TDOM\HashModule::SecureHash 输出 $H_M(x) = Mx + \text{TDOM-Sponge}_h(Mx) \bmod p$ ，LatticeCryptographyAndHash 再把该向量与输入 key words 相加，形成用于 InitializationState 的 WordKeyResistQC。因此，后续 A_t, T_t, G_t 的状态轨道不是从裸 key words 直接开始，而是从已经经过 SIS/Ajtai 格骨架与白盒 twist 位移场共同硬化的向量开始。

8.6 与后续矩阵状态的接口

我们把这一层与格密码和量子背景联系起来 [8, 29, 7, 2, 4, 5, 19]。我们也知道后量子对称密码的常见策略不是“换成一个格问题就结束”，而是同时增大 key schedule 复杂度、扩大状态空间并避免静态轮密钥暴露 [24, 11, 12, 9, 26, 14, 28, 1, 25]。因此， $H_M(x)$ 的职责到此为止：它生成主子密钥硬化向量，后续矩阵初始化、状态更新、轮密钥矩阵变换和数据轮函数继续承担各自的结构作用。

9 32 位门级扩展：Module_MixTransformationUtil.*

9.1 我们定义的实现对象

我们在 Module_MixTransformationUtil 中看到三类操作：WordBitRestruct 按固定 bit-pair 表交换 32 位 word 的 bit；Word32Bit_KeyWithFunction 使用两个 32 位状态寄存器和四个输入 word 做 NAND/NOR/rotate 混合；Word32Bit_ExpandKey 把每个 32 位输入 word 扩展成 12 个 32 位 word。

9.2 我们设计这一层的灵感

我们知道这一层的目的不是构造最终分组密码轮函数，而是在主密钥进入矩阵前面先破坏局部 bit 位置和字节边界。我们选择 NAND、NOR、XOR、rotate 和 PHT，是因为这些操作在普通 CPU 上成本低、不会依赖查表，并且可以在 $\mathbb{F}_2$ 非线性与 $\mathbb{Z}/2^{32}\mathbb{Z}$ carry 扩散之间来回切换。

9.3 bit restructure

设输入 word 为 $w \in \{0, 1\}^{32}$ 。WordBitRestruct 使用固定交换对

$$(0, 9), (1, 18), (2, 27), (3, 20), (4, 19), (5, 28), (6, 21), (7, 14),$$

$$(8, 23), (10, 24), (11, 25), (12, 30), (13, 31), (15, 16), (17, 29), (22, 26)$$

依次交换 bit。数学上它是 $\mathbb{F}_2^{32}$ 上的置换矩阵 P_{swap} ：

$$w' = P_{\text{swap}}w.$$

9.4 12-word expansion 的完整公式

我们把 w' 分成四类视图：

$$U = w' \gg 16, \quad D = w' \& 0xffff,$$

$$L = (w' \& 0xf0000000) | ((w' \& 0x00f00000) \ll 4) | ((w' \& 0x0000f000) \ll 8) | ((w' \& 0x000000f0) \ll 12),$$

$$R = ((w' \& 0x0f000000) \ll 4) | ((w' \& 0x000f0000) \ll 8) | ((w' \& 0x00000f00) \ll 12) | ((w' \& 0x0000000f) \ll 14).$$

然后初始化六个扩散变量：

$$(a, b, c, d, e, f) = (U \oplus D, L \oplus R, U \oplus L, D \oplus R, U \oplus R, D \oplus L).$$

每一轮 $r = 0, 1$ 先执行六变量 χ -like 层：

$$\begin{aligned} a_0 &= a \oplus ((\sim b) \& c), & d_0 &= d \oplus ((\sim e) \& f), \\ b_0 &= b \oplus ((\sim c) \& d_0), & e_0 &= e \oplus ((\sim f) \& a_0), \\ c_0 &= c \oplus ((\sim d_0) \& e_0), & f_0 &= f \oplus ((\sim a_0) \& b_0). \end{aligned}$$

再执行旋转：

$$(a, b, c, d, e, f) = (\text{rotr}(a_0, 5), \text{rotr}(b_0, 11), \text{rotr}(c_0, 17), \text{rotr}(d_0, 7), \text{rotr}(e_0, 13), \text{rotr}(f_0, 19)).$$

最后对三对变量执行 PHT:

$$(a, b) \leftarrow (a + 2b, b + a), \quad (c, d) \leftarrow (c + 2d, d + c), \quad (e, f) \leftarrow (e + 2f, f + e)$$

其中加法在 $\mathbb{Z}/2^{32}\mathbb{Z}$ 中进行。两轮后输出 12 个扩展 word:

$$(a, b, c, d, e, f, a \oplus c, b \oplus d, c \oplus e, d \oplus f, e \oplus a, f \oplus b).$$

9.5 为什么这不是随便扩展

命题 9.1 (扩展层同时含二次项与 carry 链). Word32Bit_ExpandKey 的一轮包含 Boolean 二次项和 $\mathbb{Z}/2^{32}\mathbb{Z}$ 加法 carry, 因此它既不是 $\mathbb{F}_2$ 上的线性映射, 也不是单纯 ARX。

证明. χ -like 层中的 $((\sim b) \& c)$ 产生二次 Boolean 单项; PHT 中的 $a + 2b$ 和 $b + a$ 在 $\mathbb{Z}/2^{32}\mathbb{Z}$ 上产生 carry 链。二者叠加后, 输出 bit 不是输入 bit 的线性函数。□

我们把这一层与 S-box 设计、bitsliced 非线性和 lightweight symmetric design 背景相连 [18, 27, 21, 16, 17, 10], 但它在本文中的规范位置是 key-state expansion, 不是最终数据轮 S-box。

10 矩阵初始化、IV 注入和状态更新: Module_SubkeyMatrixOperation.*

10.1 我们定义的实现对象

我们在 InitializationState 中看到, 硬化后的 64 位 key vector 先被拆成 byte, 再打包成 32 位 words, 经过 Word32Bit_ExpandKey 和 Word32Bit_KeyWithFunction 得到 Word64Bit_ProcessedKey, 最后填入 A_t 。若 processed key 已用尽, 代码用 LFSR 和 Bernoulli distribution 生成随机 bit 填补矩阵剩余位置。

我们在 ApplyWordDataInitialVector 中看到, 初始向量同样经过 Word32Bit_ExpandKey, 再逆序遍历矩阵槽, 使用旋转、AND、XOR、减法、bit switch 和乘加改变 A_t 。我们把它写成一个规范的 optional state-injection 子过程, 因为它的数学对象已经存在于当前文件中。

我们在 UpdateState 中看到两段状态变换: 第一段使用 NLFSR 生成 row vector u 与 column vector v , 做仿射广播变异; 第二段使用 SDP 重新生成 u, v , 构造 Kronecker 外积和 dot product, 写入 T_t 。

10.2 我们设计这一层的灵感

我们知道矩阵状态不应该只是 key schedule 的静态查表结果。我们设计 A_t 的初始化时, 让 key material、扩展后的 32 位混合词和 LFSR 随机填充共同决定矩阵; 我们设计 UpdateState 时, 让 NLFSR 给出 bit-level 不可预测材料, 让 SDP 给出数值敏感材料。这样的设计意图是让后续轮密钥矩阵不是主密钥的简单线性展开, 而是状态机不断推动后的产物。

10.3 初始化数学

设 Word64Bit_ProcessedKey 为 $q_0, \dots, q_{m-1}$ 。矩阵填充的 key-used 分支可抽象为

$$A_{ij} \leftarrow A_{ij} - q_j \quad (j < m),$$

剩余槽由 LFSR/Bernoulli 产生的 $r_{ij} \in \mathbb{Z}/2^{64}\mathbb{Z}$ 填充:

$$A_{ij} \leftarrow A_{ij} + r_{ij}.$$

这里减法和加法均在 $\mathbb{Z}/2^{64}\mathbb{Z}$ 上执行。

10.4 IV 注入数学

设 IV expansion 产生 $v_0, \dots, v_{s-1} \in \mathbb{Z}/2^{32}\mathbb{Z}$ 。代码逆序访问 A 的槽。对当前槽 A_{ij} 和当前 word x , 令

$$\rho = \text{rotr}_{64}(x, 7),$$

则代码执行

$$A_{ij} \leftarrow A_{ij} - (x \oplus (x \& \rho)),$$

$$A_{ij} \leftarrow A_{ij} \oplus 2^x \bmod 64,$$

$$x \leftarrow x + A_{ij}, \quad A_{ij} \leftarrow A_{ij} + 2x + A_{ij}.$$

我们知道这一步的灵感是把 **IV** 当作矩阵扰动，而不是当作数据块 **XOR**。它不要求 **IV** 生成独立轮密钥，而是改变后续 A_t, T_t, G_t 的状态轨道。

10.5 NLFSR 仿射广播变异

设 NLFSR 产生

$$u = (u_0, \dots, u_{N-1}) \in \mathbb{Z}/2^{64}\mathbb{Z}^{1 \times N}, \quad v = (v_0, \dots, v_{N-1})^\top \in \mathbb{Z}/2^{64}\mathbb{Z}^{N \times 1}.$$

代码构造

$$L_{ij} = A_{ij}u_j + v_i, \quad R_{ij} = A_{ij}v_i - u_j.$$

再令

$$\alpha_{ij} = L_{ij} \oplus (A_{ij} \& T_{ij}), \quad \beta_{ij} = R_{ij} \oplus (A_{ij} | T_{ij}),$$

更新

$$A_{ij} \leftarrow A_{ij} \oplus (\text{rotr}(\alpha_{ij}, 1) + \text{rotl}(\beta_{ij}, 63)).$$

命题 10.1 (仿射广播层的非线性来源). 若只看 u, v 固定时的 A 到 L, R ，它是 $\mathbb{Z}/2^{64}\mathbb{Z}$ 上的逐元素仿射广播；但加入 $A \& T$ 、 $A | T$ 后，该层在 **bit** 级引入 Boolean 非线性。

证明. $A_{ij}u_j + v_i$ 与 $A_{ij}v_i - u_j$ 属于 $\mathbb{Z}/2^{64}\mathbb{Z}$ 上的仿射/乘法表达式。 $A \& T$ 和 $A | T$ 在 **bit** 层不是 $\mathbb{F}_2$ 线性映射；与 **XOR** 和 **rotate/addition** 组合后，输出同时依赖 **bit-level Boolean** 门和 **modular carry**。□

10.6 Kronecker 外积注入

第二段状态更新中，SDP 生成新的行向量和列向量

$$u = (u_0, \dots, u_{N-1}) \in \mathbb{Z}/2^{64}\mathbb{Z}^{1 \times N}, \quad v = (v_0, \dots, v_{N-1})^\top \in \mathbb{Z}/2^{64}\mathbb{Z}^{N \times 1}.$$

代码调用 `kroneckerProduct(u, v)`。由于第一个参数是 $1 \times N$ 行向量，第二个参数是 $N \times 1$ 列向量，**Eigen** 产生的是一个 $N \times N$ 外积形矩阵：

$$K_\otimes = vu, \quad (K_\otimes)_{ij} = v_i u_j.$$

代码随后计算内点积标量

$$\delta = \langle v, u \rangle = \sum_{i=0}^{N-1} v_i u_i \in \mathbb{Z}/2^{64}\mathbb{Z},$$

并写入

$$T \leftarrow A(K_\otimes \delta) = A(\delta vu) = \delta(Av)u.$$

这里的乘法和加法均按 $\mathbb{Z}/2^{64}\mathbb{Z}$ 的 **unsigned overflow** 语义解释；若为了说明结构而暂时嵌入域中，则该矩阵具有 **rank-at-most-one-style** 外积形态。

命题 10.2 (Kronecker 注入的外积坍塌). 设 u 为行向量， v 为列向量， $\delta = \langle v, u \rangle$ 。则

$$A(\delta vu) = \delta(Av)u.$$

在域上，该结果矩阵秩至多为 1；在 $\mathbb{Z}/2^{64}\mathbb{Z}$ 中不能直接套用域秩定理，但该映射仍然由一个列方向 Av 、一个行方向 u 和一个内点积标量 δ 完全支配。

证明. 由矩阵乘法结合律， $A(vu) = (Av)u$ 。标量 δ 可从外积中提出，得到 $A(\delta vu) = \delta A(vu) = \delta(Av)u$ 。在域上，非零列向量与非零行向量的外积秩为 1；若其中任意因子为零，则秩为 0。 $\mathbb{Z}/2^{64}\mathbb{Z}$ 存在 **zero divisors**，因此不能把域秩结论原样作为 $\mathbb{Z}/2^{64}\mathbb{Z}$ 定理，但外积支配关系不依赖可除性，仍由代码精确给出。□

我们知道这一层最容易被误读成“Kronecker 生成普通随机矩阵”。实际情况相反：它是 **collapsed / outer-product / rank-at-most-one-style** 的畸形窄映射，不是普通满熵矩阵随机化。它把 $2N$ 个 SDP word 和一个内点积压缩成 N^2 个矩阵槽，看起来扩展成一个矩阵，实际上由少数方向向量控制。这个窄映射不是实现失误，而是我们有意设计的状态注入形态。

10.7 为什么窄映射在设计中可行

我们设计 Kronecker 外积注入的动机，是在状态更新中制造一个强方向性扰动，而不是生成一个独立满熵矩阵。若目标是直接生成满熵 $N \times N$ 矩阵，外积坍塌当然不合适；但当前 Type-2 的下一步不是把 T 当作最终轮密钥，而是把 T 放入

$$G \leftarrow G + (T^\top - A)(A^\top + T)AT.$$

在这个非交换表达式中， T 同时出现在 $T^\top$ 、 $A^\top + T$ 和右端 AT 三个位置。外积方向因此会被转置路径、左乘路径和右乘路径反复重排。我们把 Kronecker 层设计成窄映射，是为了让 SDP 采样材料形成可解释的方向性注入，再由后续矩阵乘法和 CrazyTransformAssociatedWord 进一步打散。

命题 10.3 (外积方向进入后续非交换表达式的三个位置). 若 $T = \delta(Av)u$ ，则 $\Delta_G(A, T) = (T^\top - A)(A^\top + T)AT$ 中的 T 至少通过三种方向进入：

$$T^\top = \delta u^\top (Av)^\top, \quad A^\top + T = A^\top + \delta(Av)u, \quad AT = \delta A(Av)u.$$

因此同一个外积扰动不会只保留为单侧右乘滤波。

证明. 把 $T = \delta(Av)u$ 分别代入 $T^\top$ 、 $A^\top + T$ 和 AT 即得。由于这些因子在矩阵乘法中顺序不同，且一般不交换，外积方向在后续轮密钥矩阵表达式中分别进入行方向、列方向和右乘传播方向。 $\square$

我们因此把这一层称为压缩型状态注入，而不是满熵矩阵随机化。它的可行性来自职责分离：Kronecker 层负责把 SDP 材料压成方向性矩阵扰动；OPC_MatrixTransformation 负责把方向性扰动送入非交换矩阵表达式；whitening 和 BitMatrix 负责把轮密钥向量做 bit-uniform parity distribution；CrazyTransformAssociatedWord 再从当前轮密钥矩阵中做状态相关访问、masking 和模加 carry 注入。若只孤立观察 Kronecker 一层，它确实是窄映射；沿后续路径观察，才能看到窄映射并没有被当成最终安全层，而是被用作方向性注入源。

11 轮密钥矩阵、whitening 和 BitMatrix： Module_SecureRoundSubkeyGeneratation.*

11.1 我们定义的实现对象

我们把 Module_SecureRoundSubkeyGeneratation.* 视为 Type-2 轮密钥状态的第二级生成器。它接收前一阶段保存在公共状态中的矩阵 A_t 与 T_t ，生成轮密钥矩阵 G_t ，再把 G_t 展平为轮密钥向量，与前一轨道向量 whitening，最后把每 32 个 64 位 word 送入固定 BitMatrix parity mixer。该文件还包含 PHT 正逆变换和 CrazyTransformAssociatedWord；为了避免层次混乱，本节只讨论轮密钥矩阵、whitening 和 BitMatrix，PHT 与 F 函数分别在后续两节给出。

11.2 轮密钥矩阵变换：从矩阵状态到 G_t

我们在 OPC_MatrixTransformation 中定义的不是线性展开表，而是一个依赖 A_t, T_t 及其转置的 word-ring 矩阵表达式。代码先构造

$$L_t = A_t + T_t^\top, \quad R_t = T_t - A_t^\top.$$

随后计算

$$\text{Tmp}_t = R_t^\top L_t^\top = (T_t^\top - A_t)(A_t^\top + T_t), \quad Q_t = A_t T_t,$$

最后更新

$$G_t \leftarrow G_t + \text{Tmp}_t Q_t = G_t + (T_t^\top - A_t)(A_t^\top + T_t)A_t T_t.$$

所有矩阵加法、减法和乘法均在 $\mathbb{Z}/2^{64}\mathbb{Z}$ 的 unsigned overflow 语义中执行。我们不把该表达式写成 $GF(2^{64})$ 上的矩阵乘法，因为实现中没有不可约多项式约简；它是 $\mathbb{Z}/2^{64}\mathbb{Z}$ 上的矩阵乘法与 bitwise 状态机后续耦合的轮密钥生成式。

11.3 我们设计非交换矩阵表达式的灵感

我们知道前一阶段的 Kronecker 外积注入具有强方向性： $T_t = \delta(A_t v)u$ 会把一个列方向和一个行方向压入矩阵状态。如果后续只做逐元素 XOR、逐元素加法或单侧乘法，这种方向性很容易保持为可追踪结构。我们因此让 $A_t, T_t, A_t^\top, T_t^\top$ 同时进入轮密钥矩阵表达式，并把右侧传播项 $A_t T_t$ 放在最后。这样做的直觉是：行空间、列空间、转置方向和右乘传播路径必须在同一轮密钥矩阵中发生耦合。

我们把这个设计写成非交换表达式，是因为一般矩阵乘法满足

$$(T_t^\top - A_t)(A_t^\top + T_t)A_t T_t \neq A_t T_t (T_t^\top - A_t)(A_t^\top + T_t).$$

这种非交换性不等于完整安全证明，但它阻止轮密钥矩阵被误解成主密钥材料的可交换线性滤波。我们设计这一层的目的是把前面状态更新产生的方向性材料折叠进一个依赖乘法顺序的矩阵轨道。

命题 11.1 (矩阵增量的多位置依赖). 令

$$\Delta_G(A, T) = (T^\top - A)(A^\top + T)AT.$$

若 A 或 T 发生扰动，则 Δ_G 的扰动一般同时出现在左因子、中间因子和右因子中，因此该更新不是单侧线性扰动。

证明. 若 $A \leftarrow A + E$ ，则 E 分别进入 $-(A + E)$ 、 $(A + E)^\top$ 和 $(A + E)T$ ；若 $T \leftarrow T + F$ ，则 F 分别进入 $(T + F)^\top$ 、 $A^\top + (T + F)$ 和 $A(T + F)$ 。展开后出现 E 、 F 以及二次和更高交叉项。故该映射不是只在一个矩阵位置上发生的线性替换。□

11.4 轮密钥向量 whitening: key-state 合成而非数据 whitening

我们把 G_t 按 Eigen array 顺序展平为 $h_t = \text{vec}(G_t)$ 。代码随后执行

$$\gamma_t \leftarrow \gamma_{t-1} \oplus h_t.$$

这里的 **whitening** 不是对明文或密文做输入输出白化，而是对轮密钥状态做轨道合成。第一次调用时，代码先清零 γ_{t-1} 与 G_t ，再执行矩阵变换；后续调用保留前一轮生成的向量轨道。我们这样设计，是为了避免 G_t 每次覆盖旧状态，而是让新矩阵状态与旧 **round-subkey vector** 在 $\mathbb{F}_2$ 上叠加。

命题 11.2 (whitening 的差分叠加). 对两个相邻输入状态取 XOR 差分，whitening 层满足

$$\Delta\gamma_t = \Delta\gamma_{t-1} \oplus \Delta h_t.$$

证明. 由 $\gamma_t = \gamma_{t-1} \oplus h_t$ 以及 XOR 的结合律和自反消去律可直接得到。□

我们知道这个命题本身很简单，但它说明 **whitening** 的位置：该层不负责非线性，也不负责可逆性；它把旧轨道差分和新矩阵差分合并，再交给 BitMatrix 做 bit 均匀关联分布。

11.5 BitMatrix 的构造位置: hard-coded XOR ledger 是压缩表示

我们定义每 32 个 64 位 word 为一组：

$$X = (X_0, \dots, X_{31})^\top, \quad Y = (Y_0, \dots, Y_{31})^\top.$$

源码中每一行

$$\text{KeyStateY}[i] = \text{KeyStateX}[j_0] \wedge \dots \wedge \text{KeyStateX}[j_{15}]$$

都定义一个 16-of-32 的输入集合 S_i 。令

$$D_{ij} = 1 \iff j \in S_i,$$

则 word 记法只是下面 bit 线性映射的压缩显示：

$$Y_i = \bigoplus_{j \in S_i} X_j.$$

我们真正的设计对象不是一个普通 32-word 线性码，而是展平到 bit 后的矩阵乘法

$$\text{BitVectorY} = M_{\text{bit}} \text{BitVectorX}.$$

由于 uint64_t XOR 对 64 条 bit lane 并行执行，若令

$$x^{(\ell)} = (X_0[\ell], X_1[\ell], \dots, X_{31}[\ell])^\top \in \mathbb{F}_2^{32},$$

则每条 lane 都满足

$$y^{(\ell)} = D x^{(\ell)}.$$

按 word-major 顺序展平 bit vector 时，完整矩阵为

$$M_{\text{bit}} = D \otimes I_{64} \in \mathbb{F}_2^{2048 \times 2048}.$$

按 lane-major 顺序展平时，同一映射写作 $I_{64} \otimes D$ ；两种写法只差固定置换。

我们知道 GenerateDiffusionLayerPermuteIndices 的设计意义在这里最容易被误读。它不是运行时 CSPRNG，也不是把随机性隐藏在程序外部；它是离线生成 S_i 的常量构造器。ISAAC64、shuffle、16-of-32 抽样和 10223 轮扰动给出一组固定的 row supports；运行时只执行 hard-coded XOR ledger。也就是说，BitMatrix 存在于源码的赋值图中，而不是以显式二维数组保存。

11.6 我们设计 BitMatrix 的灵感：bit 均匀分布

我们设计这一层时需要解决的问题不是数据解密的可逆扩散，而是 round-subkey vector 的均匀采样和重新布线。轮密钥材料已经经历矩阵变换与 whitening；此处需要一个固定、低成本、无查表秘密、bit 级均匀的 parity distribution layer。于是每个输出 word 选 16 个输入 word，每个输入 word 在 32 行中出现 16 次。二部图语言下， D 是一个输入侧和输出侧均为 32 个节点的 (16, 16)-regular incidence matrix。

我们把该设计提升到 bit 层后，得到 64 份并行的正则关联网。每个输出 bit 从 16 个同 lane 输入 bit 得到 parity，每个输入 bit fan-out 到 16 个同 lane 输出 bit。跨 lane 的扩散不由 BitMatrix 单独完成，而是由前后的 rotate、PHT、模加 carry、 χ -like 层和 CrazyTransformAssociatedWord 完成。这个职责划分使该层可行：它只做 bit-uniform parity distribution，不伪装成 MDS 层，也不伪装成单独安全的线性码。

命题 11.3 (bit 均匀分布). 若 D 的每一行和每一列权重均为 16，则 $M_{\text{bit}} = D \otimes I_{64}$ 的每个输出 bit 依赖恰好 16 个输入 bit，每个输入 bit 影响恰好 16 个输出 bit。

证明. $D \otimes I_{64}$ 中， D 的每个 1 被替换为 64×64 单位矩阵。固定输出 word i 与 lane ℓ ， $Y_i[\ell]$ 只接收满足 $D_{ij} = 1$ 的 $X_j[\ell]$ 。行权重为 16 给出输出依赖数。固定输入 word j 与 lane ℓ ，列权重为 16 给出输入传播数。 □

命题 11.4 (单 bit 差分传播). 若 whitening 后只有一个 bit $X_j[\ell]$ 翻转，则 BitMatrix 输出中恰好有 16 个 bit 翻转，且均位于 lane ℓ 。

证明. 列 j 在 D 中出现 16 次； $D \otimes I_{64}$ 的单位块保持 lane 不变，因此该输入 bit 只传播到这些行的同一 lane。 □

11.7 当前常量的行集合与设计判据

我们给出当前硬编码 D 的行集合，是为了让正文能够直接还原 BitMatrix，而不是把该层视作省略细节。这里的行集合是 M_{bit} 的压缩 supports；它们不是 word 层 MDS 证明，也不是把 D 当作独立 word-code 的安全结论。

表 5: 当前硬编码 BitMatrix 的 32 个行集合 S_i

行	输入 word 索引集合
S_0	{24, 8, 6, 1, 9, 4, 10, 3, 26, 2, 5, 15, 17, 13, 23, 12}
S_1	{19, 11, 22, 14, 25, 31, 7, 0, 30, 21, 28, 20, 18, 27, 29, 16}
S_2	{4, 18, 10, 26, 1, 22, 30, 21, 20, 5, 23, 12, 17, 6, 3, 25}
S_3	{11, 19, 24, 16, 0, 7, 28, 13, 29, 14, 2, 15, 27, 8, 31, 9}
S_4	{21, 13, 28, 4, 7, 24, 25, 9, 16, 5, 6, 19, 23, 31, 27, 1}
S_5	{15, 3, 11, 2, 12, 20, 17, 30, 10, 22, 8, 0, 18, 26, 29, 14}
S_6	{16, 24, 21, 25, 18, 10, 30, 22, 0, 6, 27, 1, 23, 4, 28, 3}
S_7	{12, 20, 14, 31, 15, 2, 9, 8, 29, 11, 5, 19, 26, 13, 17, 7}
S_8	{7, 31, 8, 24, 2, 9, 3, 22, 14, 6, 4, 20, 27, 17, 26, 21}
S_9	{19, 23, 15, 28, 5, 0, 1, 10, 25, 30, 13, 12, 18, 16, 29, 11}
S_{10}	{25, 9, 30, 22, 14, 3, 10, 18, 12, 4, 26, 21, 27, 24, 8, 28}
S_{11}	{0, 17, 1, 19, 11, 13, 5, 7, 29, 15, 6, 20, 16, 31, 23, 2}
S_{12}	{9, 17, 13, 5, 7, 2, 28, 30, 11, 4, 24, 0, 26, 23, 16, 22}
S_{13}	{12, 20, 27, 19, 8, 6, 21, 25, 3, 10, 31, 1, 18, 14, 29, 15}
S_{14}	{7, 3, 11, 30, 28, 18, 10, 25, 1, 24, 16, 22, 26, 9, 13, 8}
S_{15}	{20, 12, 21, 23, 31, 15, 6, 2, 29, 19, 4, 0, 14, 17, 27, 5}
S_{16}	{7, 31, 8, 24, 2, 9, 3, 22, 14, 6, 4, 20, 27, 17, 26, 21}
S_{17}	{19, 23, 15, 28, 5, 0, 1, 10, 25, 30, 13, 12, 18, 16, 29, 11}
S_{18}	{25, 9, 30, 22, 14, 3, 10, 18, 12, 4, 26, 21, 27, 24, 8, 28}
S_{19}	{0, 17, 1, 19, 11, 13, 5, 7, 29, 15, 6, 20, 16, 31, 23, 2}
S_{20}	{9, 17, 13, 5, 7, 2, 28, 30, 11, 4, 24, 0, 26, 23, 16, 22}
S_{21}	{12, 20, 27, 19, 8, 6, 21, 25, 3, 10, 31, 1, 18, 14, 29, 15}
S_{22}	{7, 3, 11, 30, 28, 18, 10, 25, 1, 24, 16, 22, 26, 9, 13, 8}

行	输入 word 索引集合
S_{23}	{20, 12, 21, 23, 31, 15, 6, 2, 29, 19, 4, 0, 14, 17, 27, 5}
S_{24}	{31, 7, 23, 6, 10, 2, 5, 8, 15, 24, 9, 12, 16, 27, 14, 30}
S_{25}	{0, 4, 20, 13, 1, 22, 26, 3, 28, 25, 17, 21, 18, 11, 29, 19}
S_{26}	{18, 10, 2, 15, 8, 28, 25, 3, 21, 9, 14, 30, 16, 7, 31, 13}
S_{27}	{17, 1, 22, 27, 19, 0, 4, 5, 29, 20, 24, 12, 11, 23, 26, 6}
S_{28}	{27, 2, 4, 13, 5, 6, 17, 25, 19, 9, 7, 1, 14, 26, 11, 10}
S_{29}	{28, 12, 16, 24, 0, 31, 21, 30, 8, 3, 23, 22, 18, 15, 29, 20}
S_{30}	{13, 5, 3, 19, 25, 8, 18, 28, 22, 7, 11, 10, 14, 2, 17, 31}
S_{31}	{21, 6, 30, 12, 20, 24, 23, 26, 29, 0, 9, 1, 15, 27, 16, 4}

我们把该表作为 $D \otimes I_{64}$ 的可复现定义。该层的判据是 row degree、column degree、single-bit propagation 和与后续非线性层的接口关系；不把 word-index rank、word 层可逆性或 MDS 性质作为本文主张。

12 PHT 正逆变换与 Lai–Massey 可逆性

12.1 PHT 正向变换

ForwardTransform 对两个 32 位 word 执行

$$A = L + R, \quad B = L + 2R,$$

然后

$$B' = B \oplus \text{rotr}(A, 1), \quad A' = A \oplus \text{rotr}(B', 63).$$

由于 C++ 对 32 位 word 调用 `std::rotr` 时旋转量按 32 取模， $\text{rotr}(B', 63) = \text{rotr}(B', 31) = \text{rotr}(B', 1)$ 。

12.2 PHT 逆向变换

BackwardTransform 先撤销 XOR-rotate：

$$A = A' \oplus \text{rotr}(B', 63), \quad B = B' \oplus \text{rotr}(A, 1).$$

然后撤销 PHT：

$$R = B - A, \quad L = 2A - B.$$

命题 12.1 (PHT 层可逆). 上述 ForwardTransform 与 BackwardTransform 在 $\mathbb{Z}/2^{32}\mathbb{Z}^2$ 上互为逆映射。

证明. 正向先计算 $A = L + R$ 与 $B = L + 2R$ 。逆向末两式给出 $B - A = R$ ， $2A - B = 2(L + R) - (L + 2R) = L$ 。XOR-rotate 部分中 $B' = B \oplus \text{rotr}(A, 1)$ ，所以知道 A 后可恢复 B ； $A' = A \oplus \text{rotr}(B', 63)$ ，所以知道 B' 后可恢复 A 。逆向代码按相反顺序执行，故为逆映射。□

12.3 Lai–Massey 字级结构

LaiMasseyFramework 把 64 位 word 拆成

$$W = L \parallel R, \quad L, R \in \mathbb{Z}/2^{32}\mathbb{Z}.$$

加密时先计算

$$t = F(L \oplus R, k), \quad \tilde{L} = L \oplus t, \quad \tilde{R} = R \oplus t,$$

再输出

$$(L', R') = H(\tilde{L}, \tilde{R}).$$

解密时先执行

$$(\tilde{L}, \tilde{R}) = H^{-1}(L', R'),$$

然后用同一不变量重新计算

$$t = F(\tilde{L} \oplus \tilde{R}, k) = F(L \oplus R, k),$$

最后恢复

$$L = \tilde{L} \oplus t, \quad R = \tilde{R} \oplus t.$$

命题 12.2 (Lai-Massey 不要求 F 可逆). 只要 H 可逆, 则上述字级变换可逆, 即使 F 本身不是可逆函数。

证明. 核心不变量是

$$(L \oplus t) \oplus (R \oplus t) = L \oplus R.$$

所以解密时通过 H^{-1} 得到 $\tilde{L}, \tilde{R}$ 后, 可以计算与加密时相同的 $t = F(L \oplus R, k)$ 。再 XOR 一次 t 即恢复 L, R 。因此 F 只需可计算, 不需可逆。□

我们知道这就是 Type-2 能把极其复杂的 CrazyTransformAssociatedWord 放进数据轮的原因。复杂 F 函数可以是 state-dependent、多对一、带矩阵访问和 bit mask; 可逆性由 Lai-Massey 框架负责, 而不是由 F 自身负责。该结构与 Lai-Massey 系列设计背景相关 [13, 3, 15]。

13 状态相关 F 函数: CrazyTransformAssociatedWord

13.1 我们定义的实现对象

我们把当前实现中的函数作为本节的规范语义。输入为 associated word $a \in \mathbb{Z}/2^{32}\mathbb{Z}$ 与 64 位 word key material $k \in \mathbb{Z}/2^{64}\mathbb{Z}$ 。函数内部使用四个 32 位临时 word: A, B, C, D 。它先由 a, k 生成 pseudo-random 64 位值, 再进行 bit reorganization、key/data Boolean 注入、状态相关矩阵访问、round-subkey bit mask、双通道输出和最后的 $\mathbb{Z}/2^{32}\mathbb{Z}$ 模加。

13.2 我们设计这一层的灵感

我们知道这个函数不是普通 S-box, 也不是普通 ARX, 也不是简单查轮密钥。我们设计它时的直觉是让 F 同时依赖四种东西: 当前 Lai-Massey 的 associated word $a = L \oplus R$ 、当前 64 位 word key k 、轮密钥矩阵 G 的状态相关单元 $G_{\pi(A), \pi(B)}$ 、以及从该单元中抽取的两个 bit mask。这样一来, 攻击者不能把 F 写成只依赖 a 和固定 round key 的普通代换表。

13.3 完整数学定义

令

$$k_L = \text{hi}_{32}(k), \quad k_R = \text{lo}_{32}(k).$$

首先计算

$$P = ((k \oplus a) \ll 32) \mid (((\sim k) \oplus a) \gg 32), \quad s = k \bmod 64.$$

生成

$$C = \text{lo}_{32}((P \ll s) \gg 32), \quad D = \text{lo}_{32}(P \gg s).$$

当前函数使用 XOR 注入而不是旧式 AND/OR 赋值:

$$C \leftarrow C \oplus (a \mid k_L), \quad D \leftarrow D \oplus ((\sim a) \mid k_R).$$

然后

$$A \leftarrow A \oplus C = C, \quad B \leftarrow B \oplus D = D.$$

令 $r = P \bmod 32$, 执行

$$A \leftarrow \text{rotr}(A \oplus (k_L \mid k_R), r), \quad B \leftarrow \text{rotr}(B \oplus (k_L \& k_R), r).$$

回注 C, D :

$$\begin{aligned} D &\leftarrow D \oplus ((\sim a) \& k_L), & C &\leftarrow C \oplus (a \& k_R), \\ B &\leftarrow B \oplus C \oplus (\sim k_L), & A &\leftarrow A \oplus D \oplus (\sim k_R). \end{aligned}$$

接着用 A, B 选择矩阵单元:

$$r_0 = \pi(A \bmod N), \quad c_0 = \pi(B \bmod N), \quad q = G_{r_0, c_0}.$$

定义两个 shift 和 rotate 量:

$$\begin{aligned} \sigma_1 &= A + B, & \sigma_2 &= A + 2B, \\ \rho_1 &= c_0 - r_0, & \rho_2 &= 2r_0 - c_0. \end{aligned}$$

抽取两个 bit:

$$b_1 = (q \gg (\sigma_1 \bmod 64)) \& 1, \quad b_2 = (q \gg (\sigma_2 \bmod 64)) \& 1.$$

构造 mask:

$$m = \text{rotr}_{64}(b_1, \rho_1 \bmod 64) \oplus \text{rotr}_{64}(b_2, \rho_2 \bmod 64).$$

当前代码不再额外强制 $m \neq 0$; 它直接计算

$$\tilde{q} = q \& (\sim m).$$

把原始 q 与 masked $\tilde{q}$ 同时拆半:

$$q_L = \text{hi}_{32}(q), \quad q_R = \text{lo}_{32}(q), \quad \tilde{q}_L = \text{hi}_{32}(\tilde{q}), \quad \tilde{q}_R = \text{lo}_{32}(\tilde{q}).$$

双通道输出为

$$T_1 = \text{rotr}(A \oplus \tilde{q}_R, 16) \oplus \text{rotr}(B \oplus \tilde{q}_L, 16),$$

$$T_2 = (q_L - \text{rotr}(C \oplus q_R, 8)) \oplus \text{rotr}(D, 8).$$

最终返回

$$F(a, k) = a + (T_1 \oplus T_2) \pmod{2^{32}}.$$

13.4 为什么这个映射变态

命题 13.1 (当前 F 函数的依赖图). 当前 $F(a, k)$ 依赖于 a, k, π, G , 并且 G 以原始通道 q 和 masked 通道 $\tilde{q}$ 两种方式同时进入输出。

证明. A, B, C, D 均由 a, k, P 经 Boolean 门和 rotate 生成。 A, B 选择 r_0, c_0 , 故选择的矩阵单元 q 依赖于 a, k, π, G 。 mask m 又由 q 的两个 bit、 A, B, r_0, c_0 共同决定, 故 $\tilde{q} = q \& \sim m$ 是 q 的自引用派生值。 T_1 使用 $\tilde{q}$, T_2 使用原始 q , 最终以 XOR 后模加回注 a 。 因此输出同时依赖原始 round-subkey、 masked round-subkey 和 modular carry。 $\square$

我们知道这不是“随机写复杂”。它的畸形点有三层。第一, 行列选择由 a, k 派生, 矩阵访问不是固定位置。第二, mask 由被访问的 q 自己的两个 bit 决定, 所以 round-subkey 不是被动输入, 而是参与选择自己被删除的 bit。第三, 最终不是纯 XOR, 而是 $a + (T_1 \oplus T_2)$, carry 链会把低位扰动推向高位。这使 F 很难被写成固定 S-box 或固定线性掩码表。

说明 13.1 (关于最后一行的论文语义). 我们在本文的数学语义中把最后一行写成 $a \leftarrow a + (T_1 \oplus T_2)$, 因为当前函数中已定义的第二通道对象是 FunctionResult2。若实现文本中出现未声明的 RoundSubkeyRight2, 它在数学上对应的就是第二通道 T_2 ; 论文以可编译、可闭合的数学语义为准。

14 静态字节替换层: OaldresPuzzle_Cryptic.hpp/.cpp

14.1 我们定义的实现对象

我们知道 ByteSubstitution 不是一个口头意义上的“S-box 层”, 而是由四个公开的 8 bit permutation 和一个固定的 8 byte 位置调度组成的静态字节替换算子。当前实现文件 OaldresPuzzle_Cryptic.hpp 定义四张 256 项表:

$$S_{F0}, S_{B0}, S_{F1}, S_{B1} : \mathbb{F}_2^8 \rightarrow \mathbb{F}_2^8,$$

它们分别对应 ForwardSubstitutionBox0、BackwardSubstitutionBox0、ForwardSubstitutionBox1 和 BackwardSubstitutionBox1。当前实现文件 OaldresPuzzle_Cryptic.cpp 的 ByteSubstitution 在每个 64 bit word 的 8 个 byte 上使用固定调度, 而不是运行时生成动态 S-box, 也不是运行 ZUC 流密码状态机。

我们把一个 64 bit word 写成 byte tuple

$$W = (b_0, b_1, b_2, b_3, b_4, b_5, b_6, b_7), \quad b_i \in \mathbb{F}_2^8.$$

加密方向的 byte schedule 是

$$\Sigma_E = (S_{F1}, S_{F0}, S_{B1}, S_{B0}, S_{F0}, S_{B1}, S_{F0}, S_{B1}),$$

解密方向的 byte schedule 是

$$\Sigma_D = (S_{B1}, S_{B0}, S_{F1}, S_{F0}, S_{B0}, S_{F1}, S_{B0}, S_{F1}).$$

因此加密字节层为

$$\text{BS}_E(W) = (S_{F1}(b_0), S_{F0}(b_1), S_{B1}(b_2), S_{B0}(b_3), S_{F0}(b_4), S_{B1}(b_5), S_{F0}(b_6), S_{B1}(b_7)),$$

解密字节层为

$$\text{BS}_D(W) = (S_{B1}(b_0), S_{B0}(b_1), S_{F1}(b_2), S_{F0}(b_3), S_{B0}(b_4), S_{F1}(b_5), S_{B0}(b_6), S_{F1}(b_7)).$$

源码中若输入 byte span 长度不是 8 的倍数, ByteSubstitution 直接返回; 在 Type-2 数据轮中, 该 span 来自 64 bit word block 的 byte 视图, 因此自然满足 $8 \mid |\text{span}|$ 。

14.2 我们设计这一层的灵感

我们知道 Lai-Massey 和 CrazyTransformAssociatedWord 的主要作用发生在 32 bit 和 64 bit word 层。若每个 macro-round 只停留在 word 层，攻击者可以优先尝试保留 word 边界，构造按 word 跟踪的差分、线性或代数表达。我们在每个 macro-round 末尾加入静态 byte substitution，是为了在不破坏整体可逆性的前提下，把 word 内部的 byte 边界重新扰动，使前一层产生的 word 级扰动进入下一 macro-round 时不再保持简单的字节排列结构。

我们同时使用 S_{F0}/S_{B0} 和 S_{F1}/S_{B1} 两组表，并在同一个 64 bit word 内混用 forward 表和 inverse 表。这样做不是为了模拟 ZUC runtime core，也不是为了把 byte 层伪装成动态 S-box；我们需要的是一个低成本、公开、可逆、统计性质可审计的 byte-level nonlinear layer。当前 Type-2 core 不调用 ZUC 的 LFSR、bit reorganization、linear transform 或 keystream generator；ZUC 仅作为 S_{F1} 表的历史来源和相关工作背景出现 [16]。

14.3 我们给出的 S-box 生成公式与逆表公式

我们知道四张表不能写成黑盒常量。对 S_{F0} ，源码注释给出“AES Forward SubstitutionBox Modified”和 8 次本原多项式

$$\mu_0(X) = X^8 + X^5 + X^4 + X^3 + X^2 + X + 1.$$

于是 S_{F0} 可以写成 AES-like inversion-affine 结构。令

$$\mathbb{K}_0 = \mathbb{F}_2[X]/(\mu_0(X)).$$

把 byte x 按低位优先约定解释为 $\mathbb{K}_0$ 元素，并定义

$$\text{Inv}_0(x) = \begin{cases} 0, & x = 0, \\ x^{-1} \in \mathbb{K}_0, & x \neq 0. \end{cases}$$

若

$$\text{vec}(\text{Inv}_0(x)) = (u_0, u_1, \dots, u_7)^\top \in \mathbb{F}_2^8,$$

则当前表对应的 affine 层为

$$A_0 = \begin{pmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 0 & 1 & 1 \end{pmatrix}, \quad c_0 = (1, 1, 1, 1, 1, 1, 1, 0)^\top.$$

因此

$$S_{F0}(x) = \text{vec}^{-1}(A_0 \text{vec}(\text{Inv}_0(x)) \oplus c_0).$$

这个公式解释了为什么 ForwardSubstitutionBox0[0] 等于 0x7F：当 $x = 0$ 时， $\text{Inv}_0(0) = 0$ ，输出正是 c_0 对应的 byte。

我们把 S_{B0} 定义为 S_{F0} 的逆置换：

$$S_{B0}(y) = x \iff S_{F0}(x) = y.$$

这正是 BackwardSubstitutionBox0 的生成公式。对另一组表，当前实现把 S_{F1} 指定为 ZUC 风格的公开静态 forward S-box 表；其逆表同样由逆置换公式生成：

$$S_{B1}(y) = x \iff S_{F1}(x) = y.$$

这里的“生成”不是运行时动态生成，而是算法规范层面对静态表来源和逆表关系的数学描述。运行时只进行常量表查找。

14.4 我们如何从代码得到数学实现

源码中加密方向的核心赋值可以直接翻译为：对每个 8 byte group，执行

$$\begin{aligned} b'_0 &= S_{F1}(b_0), & b'_1 &= S_{F0}(b_1), & b'_2 &= S_{B1}(b_2), & b'_3 &= S_{B0}(b_3), \\ b'_4 &= S_{F0}(b_4), & b'_5 &= S_{B1}(b_5), & b'_6 &= S_{F0}(b_6), & b'_7 &= S_{B1}(b_7). \end{aligned}$$

解密方向执行

$$\begin{aligned} b_0 &= S_{B1}(b'_0), \quad b_1 = S_{B0}(b'_1), \quad b_2 = S_{F1}(b'_2), \quad b_3 = S_{F0}(b'_3), \\ b_4 &= S_{B0}(b'_4), \quad b_5 = S_{F1}(b'_5), \quad b_6 = S_{B0}(b'_6), \quad b_7 = S_{F1}(b'_7). \end{aligned}$$

因此 ByteSubstitution 是一个 byte-wise direct product permutation:

$$\mathbf{BS}_E = S_{F1} \times S_{F0} \times S_{B1} \times S_{B0} \times S_{F0} \times S_{B1} \times S_{F0} \times S_{B1}.$$

它不是 MDS 矩阵，也不是跨 byte 的线性扩散层；它的职责是形成 byte-local nonlinear substitution。跨 word 与跨 bit-lane 的扩散已经由前面的 Lai-Massey、BitMatrix、F 函数和 macro-round 重复调度承担。

命题 14.1 (字节替换层的可逆性). 若 $S_{B0} = S_{F0}^{-1}$ 且 $S_{B1} = S_{F1}^{-1}$ ，则对任意 64 bit word W ，有

$$\mathbf{BS}_D(\mathbf{BS}_E(W)) = W.$$

证明. 按 byte 位置逐项验证即可。位置 0 上有 $S_{B1}(S_{F1}(b_0)) = b_0$ ；位置 1 上有 $S_{B0}(S_{F0}(b_1)) = b_1$ ；位置 2 上有 $S_{F1}(S_{B1}(b_2)) = b_2$ ；位置 3 上有 $S_{F0}(S_{B0}(b_3)) = b_3$ 。其余位置同理。由于 packing/unpacking 不改变 byte 值，只改变表示方式，结论成立。□

14.5 S-box 指标的计算定义与分析范围

源码注释为四张表记录了常用 S-box 指标。我们把这些指标放入正文，是因为它们说明这些表不是随手常量；但这些指标必须先给出计算定义，否则数值表会像未验证宣称。对任意 8-bit S-box $S: \mathbb{F}_2^8 \rightarrow \mathbb{F}_2^8$ ，定义差分分布表

$$\mathbf{DDT}_S(a, b) = \#\{x \in \mathbb{F}_2^8 : S(x \oplus a) \oplus S(x) = b\},$$

其中 $a, b \in \mathbb{F}_2^8$ 。差分均匀性为

$$\delta(S) = \max_{a \neq 0, b} \mathbf{DDT}_S(a, b).$$

定义线性逼近表

$$\mathbf{LAT}_S(a, b) = \sum_{x \in \mathbb{F}_2^8} (-1)^{\langle a, x \rangle \oplus \langle b, S(x) \rangle}.$$

若 S_i 表示 S 的第 i 个坐标布尔函数，则其非线性度可写为

$$\mathbf{NL}(S_i) = 2^7 - \frac{1}{2} \max_{a \in \mathbb{F}_2^8} \left| \sum_{x \in \mathbb{F}_2^8} (-1)^{S_i(x) \oplus \langle a, x \rangle} \right|.$$

SAC、透明阶、传播特性、鲁棒性、代数次数和代数免疫度分别从单比特翻转统计、功耗相关模型、DDT/LAT 分布、布尔 ANF 最高次数以及低次数湮灭函数搜索得到。本文把这些指标作为 byte-local S-box 的局部质量说明，而不把它们提升为整个 Type-2 core 的安全证明。

表	SAC	透明阶	非线性	传播特性	差分均匀性	鲁棒性	代数次数	代数免疫度
S_{F0}	满足	7.85564	112	8	4	0.984375	7	4
S_{B0}	满足	7.85711	112	8	4	0.984375	8	4
S_{F1}	满足	7.86103	112	8	4	0.984375	8	4
S_{B1}	满足	7.86029	112	8	4	0.984375	7	4

我们使用这些指标解释 byte substitution 的局部非线性质量，并把 S-box 分析工具、AES S-box 背景、key-dependent S-box 与 ZUC S-box 相关文献放在同一条参考链中 [6, 10, 18, 1, 16]。同时，我们不把静态 S-box 层说成单独安全核心；它只在 macro-round 末尾承担 byte-local nonlinear perturbation，和 word-level Lai-Massey、state-dependent F、轮密钥矩阵、BitMatrix 共同形成完整数据路径。

14.6 这一层与当前 Type-2 core 的边界

我们知道原有材料容易把静态 S-box、ZUC path 和历史动态替换盒混在一起。当前 /OOP/BlockCipher/ Type-2 core 的边界是：运行时使用四张静态表执行 byte substitution；不执行 ZUC keystream；不执行动态材料替换盒再生成；不把 byte substitution 当作外部 KDF。这样的边界使论文中的 byte 层既能保留 S-box 生成公式和局部指标，又不会把历史路径误写成当前核心。

15 数据块轮函数：OaldresPuzzle_Cryptic.cpp

15.1 我们定义的实现对象

RoundFunction 对一个数据块

$$X = (X_0, \dots, X_{B-1}) \in (\mathbb{Z}/2^{64}\mathbb{Z})^B$$

执行。函数首先要求 $|X| = B$ ，然后调用 GenerationRoundSubkeys 生成当前 $g \in (\mathbb{Z}/2^{64}\mathbb{Z})^{N^2}$ 。加密方向每个 macro-round 中，用 g 从前向后对数据 word 做 Lai-Massey 更新；一旦 block 中所有 word 用完而 g 尚未用完，就通过 goto 继续从 X_0 开始，直到 g 用完。每个 macro-round 末尾再执行 byte substitution。总 macro-round 数为 16。

15.2 我们使用的数学定义

定义单 word update

$$\Phi_k(W) = \text{LM}_k(W),$$

其中 LM 是前一节给出的 Lai-Massey 置换。一个 macro-round 的 keyed word schedule 是

$$X_{i(t)} \leftarrow \Phi_{g_t}(X_{i(t)}), \quad i(t) = t \bmod B, \quad 0 \leq t < N^2.$$

随后执行字节替换层

$$X \leftarrow \mathcal{S}(X).$$

因此加密方向完整 round function 是

$$\mathcal{R}_g(X) = \left(\mathcal{S} \circ \prod_{t=0}^{N^2-1} \Phi_{g_t}^{(t \bmod B)} \right)^{16} (X).$$

解密方向按相反顺序执行：先 $\mathcal{S}^{-1}$ ，再以 $t = N^2 - 1, \dots, 0$ 逆向调用 $\Phi_{g_t}^{-1}$ 。

15.3 我们设计这一层的灵感

我们知道这里的重量不在“16 轮”这个数字本身，而在每个 macro-round 内部会消费整个 N^2 轮密钥向量。若 N 很大，一个数据块在一个 macro-round 中会被反复访问。我们这样设计，是为了让每个数据 word 在同一 macro-round 内多次接受不同矩阵位置的 round key，而不是像普通 SPN 那样每轮只接触一小段固定 round key。

命题 15.1 (加解密调度的一致性). 若 byte substitution 表调度互逆，且单 word Lai-Massey 置换互逆，则 RoundFunction 的解密方向是加密方向的逆序组合。

证明. 加密方向每个 macro-round 是 $\mathcal{S} \circ \Phi_{g_{N^2-1}} \circ \dots \circ \Phi_{g_0}$ ，解密方向每个 macro-round 是 $\Phi_{g_0}^{-1} \circ \dots \circ \Phi_{g_{N^2-1}}^{-1} \circ \mathcal{S}^{-1}$ 。函数复合按逆序逐项抵消。重复 16 次仍成立。□

16 Worker 层与核心范围：OPC_MainAlgorithm_Worker.*

16.1 我们定义的实现对象

OPC_MainAlgorithm_Worker 负责 byte vector 与 64 位 word vector 的打包/拆包、padding/unpadding、分块、主密钥 word vector 的分段使用、stateful counter 和 core round function 的调用。EncrypterMain 先 padding，再打包成 $\mathbb{Z}/2^{64}\mathbb{Z}$ words，调用 SplitDataBlockToEncrypt，最后拆回 bytes。DecrypterMain 反向执行并 unpadding。

16.2 设计灵感和边界

我们知道 worker 中存在处理“主密钥材料已经用完之后如何继续推动 key state”的工程逻辑。我们把外框架 KDF 视为应用层 key-material 供应问题，而不是 Type-2 core 的数学主体。因此本文不把 Scrypt/PBKDF 类代码写成 OPC Type-2 的核心安全构造。我们只把 worker 层写成：它把外部 key material 变成 $\mathbb{Z}/2^{64}\mathbb{Z}$ words，分段喂给 GenerationSubkeys，再按数据块调用 RoundFunction。

16.3 核心路径定义

本文定义的 Type-2 core 从以下输入开始：

$$K_{\text{words}} \in (\mathbb{Z}/2^{64}\mathbb{Z})^m, \quad X \in (\mathbb{Z}/2^{64}\mathbb{Z})^B, \quad (A_0, T_0, \pi_0, \text{PRNG state}).$$

其中 worker 的 padding 和 packing 不改变 core 的数学结构。core 路径是

$$K_{\text{words}} \rightarrow H_M(x) \rightarrow A_t, T_t \rightarrow G_t, g_t \rightarrow \mathcal{R}_g(X).$$

16.4 Stateful 性质

我们把一次 worker 调用视为状态转移

$$(\mathcal{S}_t, X) \mapsto (\mathcal{S}_{t+1}, Y).$$

这里 $\mathcal{S}_t$ 包含 A_t, T_t, G_t, π_t 和 PRNG 内部状态。设计意图是：同一 master key 在不同状态轨道中不应简单产生同一轮密钥序列。换句话说，Type-2 不是纯函数 $E_K(X)$ 的简单实现，而是带状态的 $E_{K, \mathcal{S}_t}(X)$ 。

17 PRNG 状态材料与内部 CSPRNG 子系统：Includes/PRNGs.hpp

17.1 本节在实现中的位置

我们在 Includes/PRNGs.hpp 中实现了多个伪随机材料源。本文只把与当前 Type-2 BlockCipher 直接相关的三类对象展开：Linear-FeedbackShiftRegister、NonlinearFeedbackShiftRegister 与 SimulateDoublePendulum。我们把它们分别记为 LFSR、NLFSR 和 SDP-CSPRNG。LFSR 给出低成本线性 bit 流；NLFSR 给出门控多项式槽位和非线性组合函数；SDP-CSPRNG 把密钥 bit 映射到双摆连续系统，再把连续轨迹抽样为 64 位整数。相关背景分别对应 LFSR/NLFSR 随机数发生器、特殊硬件 NLFSR、扩展域 LFSR 随机数和双摆伪随机数研究 [27, 20, 4, 17]。

我们知道这三个源的设计灵感并不相同。LFSR 的灵感来自“便宜、可复现、线性可控”的位流生成；NLFSR 的灵感来自“同一轮只选择部分反馈多项式槽位，使状态转移不再是一个固定线性映射”；SDP-CSPRNG 的灵感来自“把离散密钥 bit 先变成物理系统参数，再借助混沌敏感性产生非线性数值材料”。在 Type-2 中，它们不是互相替代，而是进入不同位置：LFSR 参与矩阵剩余槽填充，NLFSR 参与索引洗牌与仿射广播变异，SDP-CSPRNG 参与 SIS/Ajtai 矩阵采样和 Kronecker 外积注入。

17.2 LFSR 实现：128 位状态、右移反馈和输出累积

17.2.1 代码对应

我们在 LinearFeedbackShiftRegister 中维护两个 64 位状态字：

$$S_t = (A_t, B_t) \in \mathbb{F}_2^{128}.$$

代码中的注释把多项式写为

$$x^{128} + x^{41} + x^{39} + x + 1.$$

实现并不显式保存 128 个 bit，而是把高 64 位和低 64 位分成 NumberA 与 NumberB。每生成一个 bit，代码计算

$$f_t = (B_t \oplus (A_t \gg 23) \oplus (A_t \gg 25) \oplus (A_t \gg 63)) \& 1,$$

随后把 (A_t, B_t) 作为 128 位移位寄存器右移一位，并把 f_t 写回 A_t 的最高位。输出寄存器 answer 每轮左移一位并 XOR 当前反馈 bit。

Algorithm 2 LFSR 状态推进的数学伪代码

- 1: 输入： $A_t, B_t \in \mathbb{Z}/2^{64}\mathbb{Z}$ 与输出 bit 数 n
 - 2: $a \leftarrow 128$
 - 3: **for** $r = 0$ to $n - 1$ **do**
 - 4: $f \leftarrow (B_t \oplus (A_t \gg 23) \oplus (A_t \gg 25) \oplus (A_t \gg 63)) \& 1$
 - 5: $a \leftarrow 2a + f \pmod{2^{64}}$
 - 6: $B_t \leftarrow (B_t \gg 1) \mid ((A_t \& 1) \ll 63)$
 - 7: $A_t \leftarrow (A_t \gg 1) \mid (f \ll 63)$
 - 8: **end for**
 - 9: 输出： (a, A_t, B_t)
-

17.2.2 数学定义

我们令

$$s_t = (s_{127,t}, s_{126,t}, \dots, s_{0,t}) \in \mathbb{F}_2^{128},$$

并令 A_t 承载高 64 位、 B_t 承载低 64 位。由于代码取的是 B_t 的最低位、 A_t 右移 23、25、63 后的最低位，反馈 bit 可写为

$$f_t = s_{0,t} \oplus s_{87,t} \oplus s_{89,t} \oplus s_{127,t}.$$

状态转移为

$$s_{t+1} = (f_t, s_{127,t}, s_{126,t}, \dots, s_{1,t}).$$

输出累积为

$$a_{i+1} = 2a_i + f_i \pmod{2^{64}},$$

其中实现中初始 $a_0 = 128$ 。

17.2.3 设计灵感

我们需要一个便宜、确定、可重现、能在矩阵初始化阶段快速补齐剩余 word 的 bit 流。LFSR 的优势正好在这里：它的状态推进只需要 shift 和 XOR，序列可复现，成本低，而且线性结构在这里不是主安全假设。我们没有让 LFSR 单独承担安全性，而是把它放在 SIS/Ajtai hardening、32 位混合和矩阵更新之后的填充位置。

17.2.4 可证明性质

命题 17.1 (LFSR 层是 $\mathbb{F}_2$ 上的线性状态转移)。忽略输出累积寄存器，`LinearFeedbackShiftRegister::generate_bits` 对 (A_t, B_t) 的一次 bit 推进是 $\mathbb{F}_2^{128}$ 上的线性映射。

证明. 反馈 $f_t = s_{0,t} \oplus s_{87,t} \oplus s_{89,t} \oplus s_{127,t}$ 是状态 bit 的线性函数；右移操作只是坐标置换；把 f_t 写入最高位仍是线性坐标组合。因此整个状态推进可写成 $s_{t+1} = Ls_t$ ，其中 $L \in \mathbb{F}_2^{128 \times 128}$ 。□

说明 17.1. 我们知道这条命题同时说明 LFSR 的优点和边界。优点是实现轻、可证明、可复现；边界是线性结构不应被单独当成密码安全来源。因此在本文的安全叙述中，LFSR 只被定义为状态材料补充源。

17.3 NLFSR 实现：九个反馈多项式槽位、四路门控推进和两级组合函数

17.3.1 代码对应

我们在 `NonlinearFeedbackShiftRegister` 中维护四个 64 位状态字

$$S_t = (S_t^0, S_t^1, S_t^2, S_t^3) \in (\mathbb{F}_2^{64})^4.$$

代码给出九个反馈掩码槽位 `kFeedbackMasks[0..8]`，每一轮先从当前状态派生一个 9-bit gate：

$$\gamma_t = (S_t^0 \oplus \text{rotr}(S_t^1, 7) \oplus \text{rotl}(S_t^2, 19) \oplus (S_t^3 \gg 3) \oplus (S_t^0 \gg 41)) \& 0x1FF.$$

若 $\gamma_t = 0$ ，代码以 `constant-time` 方式补最低位；随后用 9-bit rotate 调整 gate，并在固定 9 次循环中选择最先出现的四个 1 的位置 i_0, i_1, i_2, i_3 。四个状态字分别用所选反馈多项式槽位推进，再取四个低位 u_1, u_2, u_3, u_4 输入非线性组合函数。

Algorithm 3 NLFSR 九槽位反馈选择的数学伪代码

- 1: 输入：四个状态字 $S_t^0, S_t^1, S_t^2, S_t^3$ 与九个公开反馈掩码 $m_0, \dots, m_8$
 - 2: $\gamma \leftarrow (S_t^0 \oplus \text{rotr}(S_t^1, 7) \oplus \text{rotl}(S_t^2, 19) \oplus (S_t^3 \gg 3) \oplus (S_t^0 \gg 41)) \& 511$
 - 3: **if** $\gamma = 0$ **then**
 - 4: $\gamma \leftarrow \gamma \mid 1$
 - 5: **end if**
 - 6: 从 γ 的九个 bit 中按固定顺序选择前四个 1 的位置 i_0, i_1, i_2, i_3
 - 7: **for** $r = 0$ to 3 **do**
 - 8: $b_r \leftarrow S_t^r \& 1$
 - 9: $S_{t+1}^r \leftarrow (S_t^r \gg 1) \oplus ((-b_r) \& m_{i_r}) \oplus b_r$
 - 10: **end for**
 - 11: 输出： S_{t+1} 与四个低位 u_1, u_2, u_3, u_4
-

17.3.2 数学定义：反馈槽位

每个反馈掩码 m_j 定义一个条件 Galois LFSR 步进

$$L_{m_j}(s) = (s \gg 1) \oplus ((-b) \& m_j), \quad b = s \& 1.$$

代码中的 `random_bits(state, j, bit)` 先执行 L_{m_j} ，再 XOR 一个外部 bit：

$$R_j(s, b) = L_{m_j}(s) \oplus b.$$

因此一轮 NLFSR 的四路状态推进为

$$S_{t+1}^r = R_{i_r(t)}(S_t^r, b_r(t)), \quad r = 0, 1, 2, 3,$$

其中 $b_r(t) = S_t^r \& 1$ ，而 $i_0(t), \dots, i_3(t)$ 是从 γ_t 的 9-bit gate 中选出的槽位编号。

17.3.3 数学定义：组合函数

代码定义的一级组合函数是 4 变量 ANF：

$$f(u_1, u_2, u_3, u_4) = u_1 \oplus u_1 u_2 \oplus u_3 \oplus u_4.$$

`operator()` 每生成 64 位输出时，重复 64 次取

$$o_t = f(u_1, u_2, u_3, u_4), \quad A_{k+1} = 2A_k + o_t \pmod{2^{64}}.$$

`unpredictable_bits` 还使用二级组合函数

$$\begin{aligned} f_2 &= f_1 \oplus u_1 u_3 \oplus u_2 u_4 \oplus u_3 u_4 \oplus u_1 e \oplus u_4 e \\ &\quad \oplus f_1 u_3 u_4 \oplus f_1 u_1 e, \end{aligned}$$

其中 e 来自输出历史 bit 和当前状态观察 bit：

$$e = (A_k \& 1) \& (u_1 \oplus u_3).$$

定义 17.1 (NLFSR 两级组合函数). 我们把代码中的一级组合函数写成

$$f_1(u_1, u_2, u_3, u_4) = u_1 \oplus u_1 u_2 \oplus u_3 \oplus u_4.$$

二级组合函数写成

$$\begin{aligned} f_2 &= f_1 \oplus u_1 u_3 \oplus u_2 u_4 \oplus u_3 u_4 \oplus u_1 e \oplus u_4 e \\ &\quad \oplus f_1 u_3 u_4 \oplus f_1 u_1 e, \end{aligned}$$

其中 $e = (a \& 1) \& (u_1 \oplus u_3)$ 。该定义保留代码中的全部乘积项，不把 NLFSR 的非线性组合压成黑盒。

17.3.4 设计灵感

我们不希望保存九份完整 NLFSR 状态，因为那会把状态材料体积和实现复杂度都放大。我们采用“九个反馈多项式槽位，但只有四个状态寄存器”的结构：每一轮由当前状态自己生成 gate，再从九个槽位选出四个槽位驱动四个寄存器。这样做的核心动机是让反馈多项式选择依赖状态，而不是让整个 NLFSR 永远执行一个固定反馈函数。我们又把选择循环写成固定 9 次并用 constant-time 掩码赋值，是为了避免把 gate 选择直接写成 secret-dependent branch。

17.3.5 可证明性质

命题 17.2 (固定 gate 条件下的仿射性). 若一轮中的槽位 i_0, i_1, i_2, i_3 固定，则四个状态字的推进在 $\mathbb{F}_2$ 上是仿射映射；若 gate 由当前状态导出，则全局状态转移是分段仿射、状态选择的非线性映射。

证明. 对固定反馈掩码 m_j ， L_{m_j} 是 Galois LFSR 形式，属于 $\mathbb{F}_2^{64}$ 上的线性映射；再 XOR 外部 bit b 是仿射项。因此固定 i_r 时每路推进是仿射。可是 i_r 本身由 γ_t 的 bit pattern 和“前四个 1”选择规则决定，而 γ_t 是当前四个状态字经过 XOR、rotate、shift 后的函数；不同状态会选择不同线性槽位，所以全局映射是分段仿射并具有状态依赖的非线性选择。□

命题 17.3 (一级组合函数的代数次数). $f(u_1, u_2, u_3, u_4) = u_1 \oplus u_1 u_2 \oplus u_3 \oplus u_4$ 的代数次数为 2；二级组合函数 f_2 的代数次数为 3。

证明. f 的 ANF 中最高次数项是 $u_1 u_2$ ，所以次数为 2。 f_2 的 ANF 中包含 $f_1 u_3 u_4$ 和 $f_1 u_1 e$ 这样的三元乘积项，且没有把所有三次项抵消，所以次数为 3。□

17.4 SDP-CSPRNG 实现：双摆初始化、连续系统推进和 64 位抽样

17.4.1 代码对应

我们在 `SimulateDoublePendulum` 中维护十个长双精度变量：

$$(l_1, l_2, m_1, m_2, \theta_1, \theta_2, r, n, \omega_1, \omega_2).$$

其中 l_1, l_2 是两段摆长， m_1, m_2 是质量， θ_1, θ_2 是角度， ω_1, ω_2 是角速度， r 和 n 用于初始化推进步数。代码先把二进制密钥切成四段 K_0, K_1, K_2, K_3 ，再构造七组派生 bit：

$$K_0 \oplus K_1, K_0 \oplus K_2, K_0 \oplus K_3, K_1 \oplus K_2, K_1 \oplus K_3, K_2 \oplus K_3, K_0.$$

前六组写入 $(l_1, l_2, m_1, m_2, \theta_1, \theta_2)$ 的小数二进制展开，第七组写入半径/推进因子 r 。

Algorithm 4 SDP-CSPRNG 双摆显式步进的数学伪代码

```
1: 输入：  $(l_1, l_2, m_1, m_2, \theta_1, \theta_2, \omega_1, \omega_2)$  与步数  $n$ 
2: for  $t = 0$  to  $n - 1$  do
3:    $D \leftarrow 2m_1 + m_2 - m_2 \cos(2\theta_1 - 2\theta_2)$ 
4:   按下文给出的双摆加速度公式计算  $\alpha_1, \alpha_2$ 
5:    $\omega_1 \leftarrow \omega_1 + h\alpha_1, \omega_2 \leftarrow \omega_2 + h\alpha_2$ 
6:    $\theta_1 \leftarrow \theta_1 + h\omega_1, \theta_2 \leftarrow \theta_2 + h\omega_2$ 
7: end for
8: 输出：更新后的双摆连续状态
```

17.4.2 数学定义：初始化映射

令输入密钥 bit 串为 $K \in \mathbb{F}_2^L$ ，分为四段 $K^{(0)}, K^{(1)}, K^{(2)}, K^{(3)}$ 。代码构造

$$\begin{aligned} P_0 &= K^{(0)} \oplus K^{(1)}, & P_1 &= K^{(0)} \oplus K^{(2)}, & P_2 &= K^{(0)} \oplus K^{(3)}, \\ P_3 &= K^{(1)} \oplus K^{(2)}, & P_4 &= K^{(1)} \oplus K^{(3)}, & P_5 &= K^{(2)} \oplus K^{(3)}, \\ P_6 &= K^{(0)}. \end{aligned}$$

对 $0 \leq j < 6$ ，系统参数写为

$$X_j = \sum_{i=0}^{63} P_j[i] 2^{-i},$$

而半径因子写为

$$r = \sum_{i=0}^{63} P_6[i] 2^{4-i}.$$

初始化推进步数为

$$\tau_0 = \text{round}(rL).$$

执行 `run_system(true,  $\tau_0$ )` 后，代码备份 $(\theta_1, \theta_2, \omega_1, \omega_2)$ ，使 `reset()` 能把 SDP-CSPRNG 恢复到同一个密钥定义的起始轨道。

17.4.3 数学定义：双摆动力系统

代码实现的离散步进可写成

$$\begin{aligned} \omega_{1,t+1} &= \omega_{1,t} + h\alpha_1(\theta_{1,t}, \theta_{2,t}, \omega_{1,t}, \omega_{2,t}), \\ \omega_{2,t+1} &= \omega_{2,t} + h\alpha_2(\theta_{1,t}, \theta_{2,t}, \omega_{1,t}, \omega_{2,t}), \\ \theta_{1,t+1} &= \theta_{1,t} + h\omega_{1,t+1}, & \theta_{2,t+1} &= \theta_{2,t} + h\omega_{2,t+1}, \end{aligned}$$

其中 $h = 0.002$ ， $g = 9.8$ ，且

$$\begin{aligned} D &= 2m_1 + m_2 - m_2 \cos(2\theta_1 - 2\theta_2), \\ \alpha_1 &= \frac{-g(2m_1 + m_2) \sin \theta_1 - m_2 g \sin(\theta_1 - 2\theta_2) - 2 \sin(\theta_1 - \theta_2) m_2 (\omega_2^2 l_2 + \omega_1^2 l_1 \cos(\theta_1 - \theta_2))}{l_1 D}, \\ \alpha_2 &= \frac{2 \sin(\theta_1 - \theta_2) (\omega_1^2 l_1 (m_1 + m_2) + g(m_1 + m_2) \cos \theta_1 + \omega_2^2 l_2 m_2 \cos(\theta_1 - \theta_2))}{l_2 D}. \end{aligned}$$

17.4.4 数学定义：输出抽样

每次生成 64 位输出时，代码推进一步，然后计算

$$x_t = l_1 \sin \theta_{1,t} + l_2 \sin \theta_{2,t}, \quad \bar{x}_t = -l_1 \sin \theta_{1,t} - l_2 \sin \theta_{2,t}.$$

之后取

$$L_t = \lfloor \text{frac}(1000x_t)2^{32} \rfloor, \quad R_t = \lfloor \text{frac}(1000\bar{x}_t)2^{32} \rfloor,$$

并用 Morton interleave 把两个 32 位整数拼成 64 位：

$$O_t = \text{interleave}_{1\text{-bit}}(L_t, R_t).$$

其中 interleave 操作把 L_t 的 bit 放在偶数位，把 R_t 的 bit 放在奇数位。代码中的 `operator() (min,max)` 再对 O_t 做区间映射。

Algorithm 5 SDP-CSPRNG 64 位输出抽样的数学伪代码

- 1: 推进双摆状态一步
- 2: $x_t \leftarrow l_1 \sin \theta_{1,t} + l_2 \sin \theta_{2,t}$
- 3: $\bar{x}_t \leftarrow -l_1 \sin \theta_{1,t} - l_2 \sin \theta_{2,t}$
- 4: $L_t \leftarrow \lfloor \text{frac}(1000x_t)2^{32} \rfloor$
- 5: $R_t \leftarrow \lfloor \text{frac}(1000\bar{x}_t)2^{32} \rfloor$
- 6: $O_t \leftarrow \text{interleave}_{1\text{-bit}}(L_t, R_t)$
- 7: 输出: $O_t \in \mathbb{Z}/2^{64}\mathbb{Z}$

17.4.5 设计灵感

我们把 SDP-CSPRNG 接入 Type-2，不是为了说“混沌系统自动等于密码随机数”，而是为了引入一种和 LFSR/NLFSR 完全不同的状态材料来源。LFSR/NLFSR 仍然是离散布尔系统；SDP-CSPRNG 则把 key bit 转换成连续动力系统参数，再通过三角函数、分母耦合项、显式时间步进和 fractional extraction 得到 64 位材料。这个材料随后用于两个关键位置：主子密钥硬化时采样 SIS/Ajtai 矩阵 M ，以及 UpdateState 中生成 Kronecker 外积注入所需的向量 u, v 。

17.4.6 可证明性质

命题 17.4 (SDP 初始化映射的密钥相关性). 若两个密钥 bit 串在四段 XOR 派生 $P_0, \dots, P_6$ 上不同，则它们给 SDP-CSPRNG 的初始参数或初始化推进步数至少有一个不同。

证明. 系统参数 $X_0, \dots, X_5$ 由 $P_0, \dots, P_5$ 的 64 位二进制小数直接定义，推进因子 r 由 P_6 直接定义。若某个 P_j 不同，则对应二进制展开或 r 的值不同；若 r 不同，则 `round( $rL$ )` 也可能不同。故 SDP 的初始连续状态或 warm-up 轨道由密钥决定。□

命题 17.5 (SDP 输出映射的非线性来源). SDP-CSPRNG 的输出 O_t 不是输入参数关于 $\mathbb{F}_2$ 的线性函数。

证明. 状态推进包含 sin, cos、乘法、除法和 long double 舍入，输出还包含 frac、floor 与 bit interleave。这些操作均不能表示为输入 bit 上的 $\mathbb{F}_2$ 线性映射。因此输出映射不是 $\mathbb{F}_2$ 线性函数。□

说明 17.2. 我们在论文中把该模块称为 SDP-CSPRNG，是因为代码把它作为内部伪随机材料源使用，并且其初始化和输出路径都由密钥驱动。我们能由代码直接证明的是“密钥相关、可复现、非线性、连续轨道抽样”；完整密码随机性仍需外部统计测试和密码分析。但在 Type-2 中，它并不是单独暴露的 keystream，而是进入矩阵采样、Kronecker 注入和后续非线性 key schedule。

17.5 PRNG 三源在 Type-2 中的分工

表 6: LFSR、NLFSR 与 SDP-CSPRNG 的 Type-2 分工

源	实现对象	进入的算法位置	我们的设计意图
LFSR	两个 64 位 word 组成的 128 位线性反馈寄存器	矩阵初始化剩余槽、低成本 bit 材料	便宜、可复现、快速补齐；不单独承担安全性。
NLFSR	四个 64 位状态 + 九个反馈槽位 + gate 选择 + f, f_2	仿射广播变异、索引洗牌、unpredictable bits	状态选择多项式槽位，让转移不再是固定线性映射。

源	实现对象	进入的算法位置	我们的设计意图
SDP-CSPRNG	双摆连续状态 + key-dependent 参数 + 64 位 interleave 输出	SIS/Ajtai 矩阵采样、Kronecker 外积注入	引入与 bit 寄存器不同的数值敏感材料源。
ISAAC64	离线和辅助随机源	海绵内部 index/move、BitMatrix 常量生成器	生成固定常量或局部扰动索引，不作为 Type-2 数据流核心。

17.6 为什么这不是把安全性推给 PRNG

我们知道该边界必须直接写入正文：PRNG 三源不被用作直接加密流；它们只产生状态材料。Type-2 的主路径还包含 SIS/Ajtai 素域哈希、海绵 hardening、32 位门级 expansion、矩阵状态变换、round-subkey matrix transformation、BitMatrix 投影、CrazyTransform 和 Lai-Massey。PRNG 的作用是提高状态轨道的多样性，而不是替代分组密码结构本身。

18 核心子程序的详细 breakablealgorithm 伪代码

我们在本节集中给出核心子程序的论文伪代码。后续章节继续给出对应数学定义、设计灵感和证明；本节的作用是先给出完整执行链，而不是在分散段落中拼接算法。

18.1 主子密钥硬化

Algorithm 6 SIS/Ajtai-form 主子密钥硬化

Require: key words K , 素数 $p = 2^{64} - 59$, 公开矩阵 $M \in \mathbb{F}_p^{N \times N}$
Ensure: hardened vector $z \in \mathbb{F}_p^N$
1: 将 K packing 为 $x \in \mathbb{F}_p^N$ 。
2: $y \leftarrow Mx \bmod p$ 。
3: $h \leftarrow \text{TDOM-Sponge}_h(y)$, 其中 TDOM-Sponge_h 为白盒海绵。
4: **for** $i = 0$ to $N - 1$ **do**
5: $z_i \leftarrow y_i + h_i \bmod p$ 。
6: **end for**
7: **return** z 。

18.2 白盒海绵

Algorithm 7 CustomSecureHash 的吸收-置换-挤出框架

Require: byte string M , rate r , capacity c , state word 数 s
Ensure: digest words H
1: 初始化 state $S \leftarrow 0^s$ 。
2: 按当前实现 padding 规则把 M 补齐为 rate blocks $B_0, \dots, B_{u-1}$ 。
3: **for** $i = 0$ to $\nu - 1$ **do**
4: 将 B_i XOR 到 S 的 rate 区域。
5: $S \leftarrow \text{TransformState}(S)$ 。
6: **end for**
7: **while** 输出长度不足 **do**
8: 从 S 的 rate 区域读出输出 word。
9: 若仍需更多输出, 则 $S \leftarrow \text{TransformState}(S)$ 。
10: **end while**
11: **return** H 。

Algorithm 8 TransformState 的结构化白盒置换

Require: state words $S_0, \dots, S_{s-1}$, 公开轮常量 RC , 公开 index/rotation schedules
Ensure: transformed state S'
1: 执行相邻 word mixing, 使 S_i 同时依赖 S_{i-1}, S_i, S_{i+1} 。
2: 对前后半状态执行旋转扩散, 打破单半区局部性。
3: 使用固定 index schedule 选择若干 state positions 并应用公开 rotation schedule。

- 4: 对每个局部窗口执行 χ -like 二次布尔更新: $x_i \leftarrow x_i \oplus ((\neg x_{i+1}) \& x_{i+2})$ 的同型变体。
- 5: 在双端注入 RC , 破坏全零、平移和短周期对称。
- 6: **return** S' 。

18.3 矩阵状态推进与 Kronecker 外积注入

Algorithm 9 UpdateState 的矩阵状态推进

Require: 当前矩阵 A_t, T_t , NLFSR 材料, SDP 材料

Ensure: 更新后的 A_{t+1}, T_{t+1}

- 1: 从 NLFSR 生成 row vector u 与 column vector v 。
- 2: **for** 每个矩阵位置 (i, j) **do**
- 3: $L_{ij} \leftarrow A_{ij} \cdot u_j + v_i \pmod{2^{64}}$ 。
- 4: $R_{ij} \leftarrow A_{ij} \cdot v_i - u_j \pmod{2^{64}}$ 。
- 5: $\alpha_{ij} \leftarrow L_{ij} \oplus (A_{ij} \& T_{ij})$ 。
- 6: $\beta_{ij} \leftarrow R_{ij} \oplus (A_{ij} | T_{ij})$ 。
- 7: $A_{ij} \leftarrow A_{ij} \oplus (\text{rotr}(\alpha_{ij}, 1) + \text{rotl}(\beta_{ij}, 63))$ 。
- 8: **end for**
- 9: 从 SDP-CSPRNG 生成 u', v' 。
- 10: $\delta \leftarrow \langle v', u' \rangle$ 。
- 11: $T_{t+1} \leftarrow A_{t+1}(\delta v' u') = \delta(A_{t+1} v') u'$ 。
- 12: **return** A_{t+1}, T_{t+1} 。

18.4 轮密钥生成

Algorithm 10 轮密钥矩阵、whitening 与 BitMatrix 投影

Require: A_t, T_t, G_t 与 previous vector g_{t-1}

Ensure: round-subkey vector g_t

- 1: $L \leftarrow A_t^\top + T_t$ 。
- 2: $R \leftarrow T_t^\top - A_t$ 。
- 3: $G_t \leftarrow G_{t-1} + R L A_t T_t$ 。
- 4: $w \leftarrow \text{vec}(G_t)$ 。
- 5: **for** 每个 index i **do**
- 6: $w_i \leftarrow w_i \oplus g_{t-1, i}$ 。
- 7: **end for**
- 8: **for** 每个连续 32-word block X of w **do**
- 9: 将 X 展平为 $\text{bit}(X) \in \mathbb{F}_2^{2048}$ 。
- 10: $\text{bit}(Y) \leftarrow (D \otimes I_{64}) \text{bit}(X)$ 。
- 11: 将 Y 写回 transformed round-subkey vector。
- 12: **end for**
- 13: **return** g_t 。

18.5 状态相关 F 函数

Algorithm 11 CrazyTransformAssociatedWord 的论文伪代码

Require: associated word $a \in \mathbb{Z}/2^{32}\mathbb{Z}$, word key $k \in \mathbb{Z}/2^{64}\mathbb{Z}$, 矩阵 G_t , shuffle π_t

Ensure: $F(a, k) \in \mathbb{Z}/2^{32}\mathbb{Z}$

- 1: $k_L \leftarrow \text{hi}_{32}(k)$, $k_R \leftarrow \text{lo}_{32}(k)$ 。
- 2: $P \leftarrow ((k \oplus a) \ll 32) | ((\sim k \oplus a) \gg 32)$ 。
- 3: $C \leftarrow ((P \ll (k \bmod 64)) \gg 32) \bmod 2^{32}$ 。
- 4: $D \leftarrow (P \gg (k \bmod 64)) \bmod 2^{32}$ 。
- 5: $C \leftarrow C \oplus (a | k_L)$, $D \leftarrow D \oplus ((\sim a) | k_R)$ 。
- 6: $A \leftarrow C$, $B \leftarrow D$ 。
- 7: $A \leftarrow \text{rotl}(A \oplus (k_L | k_R), P \bmod 32)$ 。
- 8: $B \leftarrow \text{rotr}(B \oplus (k_L \& k_R), P \bmod 32)$ 。
- 9: $D \leftarrow D \oplus ((\sim a) \& k_L)$, $C \leftarrow C \oplus (a \& k_R)$ 。
- 10: $B \leftarrow B \oplus C \oplus \sim k_L$, $A \leftarrow A \oplus D \oplus \sim k_R$ 。

```

11:  $r \leftarrow \pi_t[A \bmod |\pi_t|], c \leftarrow \pi_t[B \bmod |\pi_t|]$ 。
12:  $q \leftarrow G_t[r, c]$ 。
13: 从  $q$  中按  $(A + B) \bmod 64$  和  $(A + 2B) \bmod 64$  抽取两个 bit, 并旋转形成 mask  $m$ 。
14:  $\tilde{q} \leftarrow q \& \sim m$ 。
15:  $T_1 \leftarrow \text{rotl}(A \oplus \text{lo}_{32}(\tilde{q}), 16) \oplus \text{rotr}(B \oplus \text{hi}_{32}(\tilde{q}), 16)$ 。
16:  $T_2 \leftarrow \text{hi}_{32}(q) - \text{rotl}(C \oplus \text{lo}_{32}(q), 8) \oplus \text{rotr}(D, 8)$ 。
17: return  $a + (T_1 \oplus T_2) \pmod{2^{32}}$ 。

```

18.6 数据轮与解密逆序

Algorithm 12 一个 block 的加密 macro-round 调度

Require: block $X = (x_0, \dots, x_{B-1})$, round-subkey vector $g = (g_0, \dots, g_{N^2-1})$

Ensure: encrypted block Y

```

1: for  $r = 0$  to  $15$  do
2:   for  $i = 0$  to  $N^2 - 1$  do
3:      $j \leftarrow i \bmod B$ 。
4:      $x_j \leftarrow \text{LMEncrypt}(x_j, g_i)$ 。
5:   end for
6:    $X \leftarrow \text{ByteSubstitution}(X)$ 。
7: end for
8: return  $X$ 。

```

Algorithm 13 一个 block 的解密 macro-round 逆序调度

Require: ciphertext block Y , 同一状态下重建出的 $g = (g_0, \dots, g_{N^2-1})$

Ensure: plaintext block X

```

1: for  $r = 15$  down to  $0$  do
2:    $Y \leftarrow \text{InverseByteSubstitution}(Y)$ 。
3:   for  $i = N^2 - 1$  down to  $0$  do
4:      $j \leftarrow i \bmod B$ 。
5:      $y_j \leftarrow \text{LMDecrypt}(y_j, g_i)$ 。
6:   end for
7: end for
8: return  $Y$ 。

```

19 模块级 breakablealgorithm 伪代码完整展开

我们在本节把前述总路径继续展开为模块级论文伪代码。这里的伪代码不替代数学定义, 也不替代源码; 它的作用是把 /OOP/Block-Cipher/ 中隐藏在函数边界、循环边界、状态对象和查表路径里的执行顺序显式写入正文。我们采用“粗路径已经给出、细路径继续下钻”的写法: 每个算法只处理一个可审计子对象, 避免把多个逻辑层压成一个黑盒步骤。

19.1 参数、状态与数据表示

我们首先把参数检查、状态布局、字节打包和填充过程写成伪代码。这样做是必要的, 因为 Type-2 core 不是单纯函数 $E_K(P)$; 它在运行过程中持续推进 A, T, G, π 和 PRNG 子状态。若不先写明表示层, 后续轮函数、矩阵状态和 worker 边界都会被误读成普通静态分组密码。

Algorithm 14 尺寸参数与 Type-2 core 状态对象构造

Require: 数据块 word 数 B , 主密钥块 word 数 K , initial-vector word 数 V

Ensure: 公共状态 $\mathcal{S} = (B, K, N, A, T, G, \pi, \text{PRNG})$

```

1: 要求  $B \geq 2$  且  $B$  为偶数。
2: 要求  $K \geq 4$ ,  $K \equiv 0 \pmod{4}$ ,  $K > B$ , 且  $B \mid K$ 。
3: 令  $N \leftarrow 2K$ , 并建立  $N \times N$  的 word 矩阵空间。
4: 分配  $A, T, G \in \mathbb{Z}/2^{64}\mathbb{Z}^{N \times N}$ 。
5: 初始化  $\pi \leftarrow (0, 1, \dots, N - 1)$  并在状态更新后允许重新 shuffle。
6: 构造 LFSR、NLFSR、SDP-CSPRNG 指针, 作为状态材料发生器。

```

7: 将 B, K, N, A, T, G, π 与 PRNG 对象存入 `CommonStateData`。

Algorithm 15 字节消息到 64 位 word block 的打包路径

Require: byte string $M = (m_0, \dots, m_{\ell-1})$

Ensure: word blocks $X^{(0)}, \dots, X^{(q-1)} \in (\mathbb{Z}/2^{64}\mathbb{Z})^B$

- 1: 按实现端序约定把每 8 个 byte packing 为一个 64 位 word。
 - 2: 若 byte 长度不是 $8B$ 的倍数, 则先执行 padding 过程。
 - 3: 将 packing 后的 word 序列按长度 B 切分为连续 block。
 - 4: **return** $X^{(0)}, \dots, X^{(q-1)}$ 。
-

Algorithm 16 worker 层 padding 与 unpadding 语义

Require: byte string M , block byte size $8B$

Ensure: padded string M' 或 unpadded string M

- 1: 令 $r \leftarrow |M| \bmod 8B$ 。
 - 2: **if** $r = 0$ **then**
 - 3: 可按 no-padding interface 保持 M 不变; 按 padding interface 则追加完整 padding block。
 - 4: **else**
 - 5: $c \leftarrow 8B - r$ 。
 - 6: 追加 $c - 1$ 个填充 byte, 再追加一个记录 c 的尾 byte。
 - 7: **end if**
 - 8: unpadding 时读取最后一个 byte c , 并删除末尾 c 个 padding byte。
 - 9: padding 属于 worker 表示层; 它不参与 round function 的密码学贡献。
-

19.2 主子密钥硬化与白盒海绵

我们把 SIS/Ajtai 主子密钥硬化拆成四个算法: 素域入域、公开矩阵生成、裸核乘法、白盒海绵扰动。这样拆开后, Mx 的线性结构、 $\text{TDOM-Sponge}_h(Mx)$ 的白盒非线性结构以及最终 $H_M(x)$ 的差分公式不会互相遮蔽。

Algorithm 17 主密钥 word 到 $\mathbb{F}_p$ 向量的入域

Require: key word span $K = (K_0, \dots, K_{s-1}) \in (\mathbb{Z}/2^{64}\mathbb{Z})^s$, 维度 N

Ensure: $x \in \mathbb{F}_p^N$

- 1: **for** $i = 0$ to $N - 1$ **do**
 - 2: $x_i \leftarrow K_{i \bmod s} \bmod p$ 。
 - 3: **end for**
 - 4: **return** x 。
-

Algorithm 18 SDP rejection sampling 生成 SIS/Ajtai 矩阵 M

Require: SDP-CSPRNG SDP, 素数 $p = 2^{64} - 59$, 阈值 τ

Ensure: $M \in \mathbb{F}_p^{N \times N}$

- 1: **for** $i = 0$ to $N^2 - 1$ **do**
 - 2: **repeat**
 - 3: $r \leftarrow \text{SDP}(0, 2^{64} - 1)$ 。
 - 4: **until** $r < \tau$
 - 5: $M_i \leftarrow r \bmod p$ 。
 - 6: **end for**
 - 7: 将线性数组 M_i 视作 $N \times N$ 矩阵。
 - 8: **return** M 。
-

Algorithm 19 Montgomery 素域 GEMV 与出域

Require: $M \in \mathbb{F}_p^{N \times N}$, $x \in \mathbb{F}_p^N$, Montgomery context $\mathcal{C}_p$

Ensure: $y = Mx \bmod p$

- 1: 将 M 的每个元素映射为 Montgomery 表示 $\widehat{M}_{ij}$ 。
 - 2: 将 x 的每个元素映射为 Montgomery 表示 $\widehat{x}_i$ 。
 - 3: 在 Montgomery 表示中计算 $\widehat{y}_i \leftarrow \sum_j \widehat{M}_{ij} \widehat{x}_j$ 。
 - 4: 将每个 $\widehat{y}_i$ 出域为标准余数 $y_i \in [0, p)$ 。
 - 5: **return** y 。
-

Algorithm 20 SIS/Ajtai-form hardening 的完整计算

Require: key-derived vector $x \in \mathbb{F}_p^N$, matrix $M \in \mathbb{F}_p^{N \times N}$, 白盒海绵 TDOM-Sponge _{h}

Ensure: hardened vector $z \in \mathbb{F}_p^N$

- 1: $y \leftarrow Mx \bmod p$ 。
 - 2: 将 y 以 64-bit word 序列形式送入 TDOM-Sponge _{h} 。
 - 3: $h \leftarrow \text{TDOM-Sponge}_h(y)$, 其中 $h \in \mathbb{F}_p^N$ 按 word 截断和取模解释。
 - 4: **for** $i = 0$ to $N - 1$ **do**
 - 5: $z_i \leftarrow y_i + h_i \bmod p$ 。
 - 6: **end for**
 - 7: **return** z 。
-

Algorithm 21 白盒海绵 padding 与 rate block 切分

Require: 输入 byte/word 序列 M , rate byte size r

Ensure: rate blocks $B_0, \dots, B_{q-1}$

- 1: 将输入复制到 buffer P 。
 - 2: **if** $|P| \bmod r \neq 0$ **then**
 - 3: 追加一个 padding marker byte/word。
 - 4: 继续追加零, 直到 $|P|$ 是 r 的倍数。
 - 5: **end if**
 - 6: 将 P 切分为长度 r 的连续 rate blocks。
 - 7: **return** $B_0, \dots, B_{q-1}$ 。
-

Algorithm 22 白盒海绵吸收阶段

Require: state $S \in (\mathbb{Z}/2^{64}\mathbb{Z})^w$, rate blocks $B_0, \dots, B_{q-1}$

Ensure: absorbed state S

- 1: **for** $i = 0$ to $q - 1$ **do**
 - 2: 将 B_i packing 为 rate words $b_{i,0}, \dots, b_{i,r_w-1}$ 。
 - 3: **for** $j = 0$ to $r_w - 1$ **do**
 - 4: $S_j \leftarrow S_j \oplus b_{i,j}$ 。
 - 5: **end for**
 - 6: $S \leftarrow \text{TransformState}(S)$ 。
 - 7: **end for**
 - 8: **return** S 。
-

Algorithm 23 白盒海绵挤出阶段

Require: absorbed state S , 输出 word 数 L , rate word 数 r_w

Ensure: output words $H_0, \dots, H_{L-1}$

- 1: $t \leftarrow 0$ 。
 - 2: **while** $t < L$ **do**
 - 3: **for** $j = 0$ to $r_w - 1$ **do**
 - 4: **if** $t < L$ **then**
 - 5: $H_t \leftarrow S_j$; $t \leftarrow t + 1$ 。
 - 6: **end if**
 - 7: **end for**
 - 8: **if** $t < L$ **then**
 - 9: $S \leftarrow \text{TransformState}(S)$ 。
 - 10: **end if**
 - 11: **end while**
 - 12: **return** H 。
-

Algorithm 24 海绵公开 index/rotation schedule 生成

Require: 固定种子 ISAAC64, 半状态长度 $w/2$

Ensure: rotation counts ρ 与 state indices σ

- 1: 构造集合 $\{1, 2, \dots, 63\}$, 由 ISAAC64 shuffle 得到 ρ 。
- 2: 构造集合 $\{0, 1, \dots, w/2 - 1\}$, 由 ISAAC64 shuffle 得到 σ 。

3: 构造左旋索引表 σ_L 与右旋索引表 σ_R 。

4: 这些表公开固定，不依赖 secret key。

5: **return** $\rho, \sigma, \sigma_L, \sigma_R$ 。

Algorithm 25 TransformState 的逐相位置换

Require: state $S = (S_0, \dots, S_{w-1})$, round constants RC , 公开 schedule ρ, σ

Ensure: transformed state S'

1: 复制 S 到三个临时 buffer U, V, W 。

2: **for** 每个 state index i **do**

3: $U_i \leftarrow S_i \oplus \text{rotr}(S_{i-1}, \rho_i) \oplus \text{rotr}(S_{i+1}, \rho_{i+1})$ 。

4: **end for**

5: **for** 每个半状态 index i **do**

6: $V_i \leftarrow U_i \oplus \text{rotr}(U_{\sigma_L(i)}, \rho_i)$ 。

7: $V_{i+w/2} \leftarrow U_{i+w/2} \oplus \text{rotr}(U_{\sigma_R(i)+w/2}, \rho_i)$ 。

8: **end for**

9: **for** 每个局部窗口 (a, b, c) **do**

10: $W_a \leftarrow V_a \oplus ((\sim V_b) \& V_c)$ 。

11: **end for**

12: 将 RC 分别注入 state 两端和/或轮常量指定位置。

13: **return** W 。

19.3 32 位扩展与矩阵初始化

我们接着把 32 位扩展工具写成独立伪代码。这里不能只写“扩展密钥”，因为该层含有 bit pair swap、半字/nibble 分割、六变量 χ -like 层、PHT 扩散和 12-word 输出结构。

Algorithm 26 SwapBits 与 WordBitRestruct

Require: word $w \in \mathbb{Z}/2^{32}\mathbb{Z}$, 固定交换表 $(s_0, t_0), \dots, (s_{15}, t_{15})$

Ensure: restructured word w'

1: **for** $i = 0$ to 15 **do**

2: $b \leftarrow ((w \gg s_i) \& 1) \oplus ((w \gg t_i) \& 1)$ 。

3: $m \leftarrow (b \ll s_i) \mid (b \ll t_i)$ 。

4: $w \leftarrow w \oplus m$ 。

5: **end for**

6: **return** w 。

Algorithm 27 单个 32-bit word 到六个扩散寄存器

Require: $w \in \mathbb{Z}/2^{32}\mathbb{Z}$

Ensure: $a, b, c, d, e, f \in \mathbb{Z}/2^{32}\mathbb{Z}$

1: $r \leftarrow \text{WordBitRestruct}(w)$ 。

2: $U \leftarrow r \gg 16$; $D \leftarrow r \& (2^{16} - 1)$ 。

3: $L \leftarrow$ 从 r 的高 nibble 位置抽取并左移对齐的 word。

4: $R \leftarrow$ 从 r 的低 nibble 位置抽取并左移对齐的 word。

5: $a \leftarrow U \oplus D$; $b \leftarrow L \oplus R$; $c \leftarrow U \oplus L$ 。

6: $d \leftarrow D \oplus R$; $e \leftarrow U \oplus R$; $f \leftarrow D \oplus L$ 。

7: **return** a, b, c, d, e, f 。

Algorithm 28 六变量 χ -like + PHT 双轮扩展

Require: $a, b, c, d, e, f \in \mathbb{Z}/2^{32}\mathbb{Z}$

Ensure: a, b, c, d, e, f 的扩散后状态

1: **for** $r = 0$ to 1 **do**

2: $(a_0, b_0, c_0, d_0, e_0, f_0) \leftarrow (a, b, c, d, e, f)$ 。

3: $a_0 \leftarrow a_0 \oplus ((\sim b_0) \& c_0)$; $d_0 \leftarrow d_0 \oplus ((\sim e_0) \& f_0)$ 。

4: $b_0 \leftarrow b_0 \oplus ((\sim c_0) \& d_0)$; $e_0 \leftarrow e_0 \oplus ((\sim f_0) \& a_0)$ 。

5: $c_0 \leftarrow c_0 \oplus ((\sim d_0) \& e_0)$; $f_0 \leftarrow f_0 \oplus ((\sim a_0) \& b_0)$ 。

6: $a \leftarrow \text{rotr}(a_0, 5)$; $b \leftarrow \text{rotr}(b_0, 11)$; $c \leftarrow \text{rotr}(c_0, 17)$ 。

```

7:    $d \leftarrow \text{rotr}(d_0, 7); e \leftarrow \text{rotr}(e_0, 13); f \leftarrow \text{rotr}(f_0, 19)。$ 
8:    $(a, b) \leftarrow (a + 2b, b + a + 2b) \bmod 2^{32}。$ 
9:    $(c, d) \leftarrow (c + 2d, d + c + 2d) \bmod 2^{32}。$ 
10:   $(e, f) \leftarrow (e + 2f, f + e + 2f) \bmod 2^{32}。$ 
11: end for
12: return  $a, b, c, d, e, f。$ 

```

Algorithm 29 Word32Bit_ExpandKey 的 12-word 输出

Require: word sequence $W = (W_0, \dots, W_{n-1})$

Ensure: expanded sequence $E \in (\mathbb{Z}/2^{32}\mathbb{Z})^{12n}$

```

1: for  $i = 0$  to  $n - 1$  do
2:    $(a, b, c, d, e, f) \leftarrow \text{Split6}(W_i)。$ 
3:    $(a, b, c, d, e, f) \leftarrow \text{ChiPHT2}(a, b, c, d, e, f)。$ 
4:    $E_{12i+0} \leftarrow a; E_{12i+1} \leftarrow b; E_{12i+2} \leftarrow c; E_{12i+3} \leftarrow d。$ 
5:    $E_{12i+4} \leftarrow e; E_{12i+5} \leftarrow f。$ 
6:    $E_{12i+6} \leftarrow a \oplus c; E_{12i+7} \leftarrow b \oplus d。$ 
7:    $E_{12i+8} \leftarrow c \oplus e; E_{12i+9} \leftarrow d \oplus f。$ 
8:    $E_{12i+10} \leftarrow e \oplus a; E_{12i+11} \leftarrow f \oplus b。$ 
9: end for
10: return  $E。$ 

```

Algorithm 30 Word32Bit_KeyWithFunction 的状态更新

Require: four words R_0, R_1, R_2, R_3 , state registers s_0, s_1

Ensure: feedback word o 与更新后 s_0, s_1

```

1:  $o \leftarrow \text{NAND}(R_0 \oplus s_0, s_1)。$ 
2:  $u \leftarrow \text{NOR}(s_0, R_1); v \leftarrow s_1 \oplus R_2。$ 
3:  $A \leftarrow (u \ll 16) \mid (v \gg 16); B \leftarrow (v \ll 16) \mid (u \gg 16)。$ 
4:  $t_0 \leftarrow \text{NAND}(s_0, s_1); t_1 \leftarrow \text{NOR}(s_0, \text{rotr}(s_1, 1))。$ 
5:  $t_2 \leftarrow \text{NAND}(\text{rotr}(s_0, 5), t_1)。$ 
6:  $s_0 \leftarrow (s_0 \oplus t_0) \oplus \text{rotr}(t_2, 3)。$ 
7:  $t_3 \leftarrow \text{NAND}(s_1, \text{rotr}(s_0, 2)); t_4 \leftarrow \text{NOR}(s_1, \text{rotr}(s_0, 7))。$ 
8:  $s_1 \leftarrow (s_1 \oplus t_3) \oplus \text{rotr}(t_4, 1)。$ 
9:  $s_0 \leftarrow A \oplus \text{rotl}(A, 2) \oplus \text{rotl}(A, 10) \oplus \text{rotl}(A, 18) \oplus \text{rotl}(A, 24)。$ 
10:  $s_1 \leftarrow B \oplus \text{rotl}(B, 8) \oplus \text{rotl}(B, 14) \oplus \text{rotl}(B, 22) \oplus \text{rotl}(B, 30)。$ 
11: return  $o。$ 

```

Algorithm 31 矩阵初始化 InitializationState

Require: hardened key words z , state matrices A, T , PRNG state

Ensure: initialized A, T, π

```

1: 将  $z$  unpack 为 byte string, 再 packing 为 32-bit words。
2:  $E \leftarrow \text{Word32Bit\_ExpandKey}(z_{32})。$ 
3: 调用 Word32Bit_Initialize 初始化两个 32-bit state registers。
4: for 每四个连续 expanded words  $E_{4i}, \dots, E_{4i+3}$  do
5:    $r_i \leftarrow \text{Word32Bit\_KeyWithFunction}(E_{4i..4i+3}) \oplus E_{4i+3}。$ 
6: end for
7: 将  $r_i$  packing 为 64-bit words, 按行填充  $A。$ 
8: 若材料不足, 则由 LFSR/NLFSR/SDP 补足剩余矩阵位置。
9: 初始化或刷新  $\pi$  为矩阵行列偏移 shuffle。
10: return  $A, T, \pi。$ 

```

Algorithm 32 Initial Vector 的 bit-level 状态注入

Require: initial vector words IV , matrix $A \in \mathbb{Z}/2^{64}\mathbb{Z}^{N \times N}$

Ensure: injected matrix A

```

1:  $E \leftarrow \text{Word32Bit\_ExpandKey}(IV)。$ 
2: for 每个 expanded word  $e \in E$  do
3:   从  $e$  派生 row index  $i$ 、column index  $j$  与 bit index  $b。$ 
4:    $A_{ij} \leftarrow A_{ij} \oplus (1 \ll b)。$ 

```

```
5: end for
6: return  $A$ 。
```

19.4 状态材料发生器

我们把 LFSR、NLFSR 和 SDP-CSPRNG 写成三个不同职责的伪代码。LFSR 是线性低成本状态源，NLFSR 是门控非线性状态源，SDP-CSPRNG 是连续系统采样状态源。三者不是把安全性外包给 PRNG，而是向矩阵状态轨道注入不同代数类型的材料。

Algorithm 33 LFSR 128-bit 状态推进与输出累积

Require: state $(s_0, s_1) \in (\mathbb{Z}/2^{64}\mathbb{Z})^2$, 输出 bit 数 n

Ensure: output word o

```
1:  $o \leftarrow 0$ 。
2: for  $t = 0$  to  $n - 1$  do
3:    $b \leftarrow s_{127} \oplus s_{40} \oplus s_{38} \oplus s_0$ 
4:    $(s_0, s_1) \leftarrow$  左移一位并把  $b$  写入最低位。
5:    $o \leftarrow (o \ll 1) \mid b$ 。
6: end for
7: return  $o$  与更新后 state。
```

Algorithm 34 NLFSR 九槽位反馈门控

Require: state (s_0, s_1, s_2, s_3) , gate bits $g \in \mathbb{F}_2^9$

Ensure: nonlinear feedback f

```
1: 预定义九个 feedback candidates  $F_0, \dots, F_8$ , 每个候选由 XOR、AND、OR、NOT 和 rotation 组成。
2:  $f \leftarrow 0$ 。
3: for  $i = 0$  to 8 do
4:   if  $g_i = 1$  then
5:      $f \leftarrow f \oplus F_i(s_0, s_1, s_2, s_3)$ 。
6:   end if
7: end for
8: return  $f$ 。
```

Algorithm 35 NLFSR 状态推进与 word 生成

Require: state (s_0, s_1, s_2, s_3)

Ensure: output word o 与新状态

```
1: 从当前 state 派生 gate bits  $g$ 。
2:  $f \leftarrow \text{NLFeedback}(s_0, s_1, s_2, s_3, g)$ 。
3:  $o \leftarrow f \oplus \text{rotr}(s_0, 13) \oplus \text{rotr}(s_1, 7) \oplus (s_2 \& \sim s_3)$ 。
4:  $(s_0, s_1, s_2, s_3) \leftarrow (s_1, s_2, s_3, o)$ 。
5: return  $o$ 。
```

Algorithm 36 SDP-CSPRNG key material 到双摆初值

Require: binary key/state material K_b

Ensure: double-pendulum state $(\theta_1, \theta_2, \omega_1, \omega_2)$

```
1: 将  $K_b$  切分为四段并映射为  $[0, 1)$  上的实数  $u_1, u_2, u_3, u_4$ 。
2:  $\theta_1 \leftarrow \pi(2u_1 - 1)$ ;  $\theta_2 \leftarrow \pi(2u_2 - 1)$ 。
3:  $\omega_1 \leftarrow 2u_3 - 1$ ;  $\omega_2 \leftarrow 2u_4 - 1$ 。
4: return  $(\theta_1, \theta_2, \omega_1, \omega_2)$ 。
```

Algorithm 37 SDP-CSPRNG 双摆显式时间步进

Require: state $(\theta_1, \theta_2, \omega_1, \omega_2)$, 步长 h , 步数 T

Ensure: updated state

```
1: for  $t = 0$  to  $T - 1$  do
2:   计算  $\Delta \leftarrow \theta_1 - \theta_2$ 。
3:   按双摆方程计算角加速度  $\alpha_1, \alpha_2$ 。
4:    $\omega_1 \leftarrow \omega_1 + h\alpha_1$ ;  $\omega_2 \leftarrow \omega_2 + h\alpha_2$ 。
5:    $\theta_1 \leftarrow \theta_1 + h\omega_1$ ;  $\theta_2 \leftarrow \theta_2 + h\omega_2$ 。
```

```

6:   对角度执行周期规约，使其回到实现选定区间。
7: end for
8: return updated state。

```

Algorithm 38 SDP-CSPRNG fractional extraction 与 Morton 合成

Require: updated double-pendulum state

Ensure: 64-bit word r

```

1: 提取  $\theta_1, \theta_2, \omega_1, \omega_2$  的小数部分并放大为若干 32-bit 整数。
2:  $u \leftarrow$  来自第一组变量的 32-bit extraction。
3:  $v \leftarrow$  来自第二组变量的 32-bit extraction。
4:  $r \leftarrow \text{MortonInterleave}_{32,32}(u, v)$ 。
5: return  $r$ 。

```

19.5 矩阵状态、轮密钥与 BitMatrix

Algorithm 39 NLFSR affine-broadcast 状态变异

Require: $A, T \in \mathbb{Z}/2^{64}\mathbb{Z}^{N \times N}$, row material $u \in \mathbb{Z}/2^{64}\mathbb{Z}^{1 \times N}$, column material $v \in \mathbb{Z}/2^{64}\mathbb{Z}^{N \times 1}$

Ensure: mutated A

```

1: for  $i = 0$  to  $N - 1$  do
2:   for  $j = 0$  to  $N - 1$  do
3:      $L_{ij} \leftarrow A_{ij}u_j + v_i \pmod{2^{64}}$ 。
4:      $R_{ij} \leftarrow A_{ij}v_i - u_j \pmod{2^{64}}$ 。
5:      $\alpha_{ij} \leftarrow L_{ij} \oplus (A_{ij} \& T_{ij})$ 。
6:      $\beta_{ij} \leftarrow R_{ij} \oplus (A_{ij} \mid T_{ij})$ 。
7:      $A_{ij} \leftarrow A_{ij} \oplus (\text{rotr}(\alpha_{ij}, 1) + \text{rotl}(\beta_{ij}, 63))$ 。
8:   end for
9: end for
10: return  $A$ 。

```

Algorithm 40 Kronecker 外积窄注入

Require: $A \in \mathbb{Z}/2^{64}\mathbb{Z}^{N \times N}$, row vector $u \in \mathbb{Z}/2^{64}\mathbb{Z}^{1 \times N}$, column vector $v \in \mathbb{Z}/2^{64}\mathbb{Z}^{N \times 1}$

Ensure: transformed matrix T

```

1:  $K \leftarrow vu$ , 即  $K_{ij} = v_i u_j$ 。
2:  $\delta \leftarrow vu$  的内点积标量  $\langle v, u \rangle = \sum_i v_i u_i$ 。
3:  $T \leftarrow A(\delta K)$ 。
4: 等价写成  $T = \delta(Av)u$ , 显示 collapsed outer-product structure。
5: return  $T$ 。

```

Algorithm 41 OPC 轮密钥矩阵变换

Require: $A, T, G \in \mathbb{Z}/2^{64}\mathbb{Z}^{N \times N}$

Ensure: updated G

```

1:  $L \leftarrow A^\top + T$ 。
2:  $R \leftarrow T^\top - A$ 。
3:  $Q \leftarrow RL$ 。
4:  $P \leftarrow AT$ 。
5:  $G \leftarrow G + QP$ 。
6: return  $G$ 。

```

Algorithm 42 key-state whitening

Require: previous vector g , matrix G

Ensure: whitened vector w

```

1:  $v \leftarrow \text{vec}(G)$ 。
2: for  $i = 0$  to  $N^2 - 1$  do
3:    $w_i \leftarrow g_i \oplus v_i$ 
4: end for
5: return  $w$ 。

```

Algorithm 43 BitMatrix 常量 D 的离线构造思想

Require: base permutation array P , ISAAC64 CSPRNG

Ensure: incidence matrix $D \in \mathbb{F}_2^{32 \times 32}$

```
1: for 多个离线搜索 round do
2:   for 每个输出行  $i$  do
3:     从  $P$  的 shuffle stream 中无放回抽取 16 个输入 index。
4:     令这些 index 对应的  $D_{ij} \leftarrow 1$ , 其余为 0。
5:     若当前 pool 为空, 则重新 shuffle  $P$  并继续。
6:   end for
7: end for
8: 固化最终  $D$  为源码中的 XOR ledger。
9: return  $D$ 。
```

Algorithm 44 BitMatrix $D \otimes I_{64}$ 的运行时投影

Require: 32-word block $X = (X_0, \dots, X_{31})$

Ensure: 32-word block $Y = (Y_0, \dots, Y_{31})$

```
1: for  $i = 0$  to 31 do
2:    $Y_i \leftarrow 0$ 。
3:   for  $j = 0$  to 31 do
4:     if  $D_{ij} = 1$  then
5:        $Y_i \leftarrow Y_i \oplus X_j$ 。
6:     end if
7:   end for
8: end for
9: 该 word-level XOR 等价于  $\text{bit}(Y) = (D \otimes I_{64}) \text{bit}(X)$ 。
10: return  $Y$ 。
```

Algorithm 45 GenerationRoundSubkeys 总流程

Require: current matrices A, T, G , previous vector g

Ensure: new round-subkey vector g'

```
1: if matrix transformation counter 为 0 then
2:   清零  $G$  与 previous vector。
3: end if
4:  $G \leftarrow \text{OPCMatrixTransformation}(A, T, G)$ 。
5:  $w \leftarrow \text{KeyWhitening}(g, G)$ 。
6: for 每个连续 32-word block  $X$  of  $w$  do
7:    $Y \leftarrow \text{BitMatrixApply}(D, X)$ 。
8:   将  $Y$  写入输出 vector 对应位置。
9: end for
10:  $g' \leftarrow$  transformed vector。
11: 递增 matrix transformation counter。
12: return  $g'$ 。
```

19.6 数据轮函数与可逆层

Algorithm 46 PHT forward transform H

Require: $L, R \in \mathbb{Z}/2^{32}\mathbb{Z}$

Ensure: L', R'

```
1:  $A \leftarrow L + R \pmod{2^{32}}$ 。
2:  $B \leftarrow L + 2R \pmod{2^{32}}$ 。
3:  $B \leftarrow B \oplus \text{rotr}(A, 1)$ 。
4:  $A \leftarrow A \oplus \text{rotr}(B, 63)$ 。
5: return  $(A, B)$ 。
```

Algorithm 47 PHT backward transform H^{-1}

Require: $A, B \in \mathbb{Z}/2^{32}\mathbb{Z}$

Ensure: L, R

- 1: $A \leftarrow A \oplus \text{rotr}(B, 63)_{\circ}$
- 2: $B \leftarrow B \oplus \text{rotl}(A, 1)_{\circ}$
- 3: $R \leftarrow B - A \pmod{2^{32}}_{\circ}$
- 4: $L \leftarrow 2A - B \pmod{2^{32}}_{\circ}$
- 5: **return** $(L, R)_{\circ}$

Algorithm 48 Lai–Massey word encryption

Require: word $W \in \mathbb{Z}/2^{64}\mathbb{Z}$, round key $k \in \mathbb{Z}/2^{64}\mathbb{Z}$

Ensure: word W'

- 1: $L \leftarrow \text{hi}_{32}(W)$, $R \leftarrow \text{lo}_{32}(W)_{\circ}$
- 2: $a \leftarrow L \oplus R_{\circ}$
- 3: $t \leftarrow \text{CrazyTransformAssociatedWord}(a, k)_{\circ}$
- 4: $L \leftarrow L \oplus t$; $R \leftarrow R \oplus t_{\circ}$
- 5: $(L, R) \leftarrow H(L, R)_{\circ}$
- 6: **return** $(L \ll 32) \mid R_{\circ}$

Algorithm 49 Lai–Massey word decryption

Require: word $W' \in \mathbb{Z}/2^{64}\mathbb{Z}$, round key $k \in \mathbb{Z}/2^{64}\mathbb{Z}$

Ensure: word W

- 1: $L \leftarrow \text{hi}_{32}(W')$, $R \leftarrow \text{lo}_{32}(W')_{\circ}$
- 2: $(L, R) \leftarrow H^{-1}(L, R)_{\circ}$
- 3: $a \leftarrow L \oplus R_{\circ}$
- 4: $t \leftarrow \text{CrazyTransformAssociatedWord}(a, k)_{\circ}$
- 5: $L \leftarrow L \oplus t$; $R \leftarrow R \oplus t_{\circ}$
- 6: **return** $(L \ll 32) \mid R_{\circ}$

Algorithm 50 CrazyTransform 的矩阵选址与 mask 子过程

Require: $A, B, C, D \in \mathbb{Z}/2^{32}\mathbb{Z}$, round-subkey matrix G , shuffle π

Ensure: $q, \tilde{q} \in \mathbb{Z}/2^{64}\mathbb{Z}$

- 1: $r \leftarrow \pi[A \bmod |\pi|]$, $c \leftarrow \pi[B \bmod |\pi|]_{\circ}$
- 2: $q \leftarrow G_{r, c_{\circ}}$
- 3: $s_1 \leftarrow (A + B) \bmod 64$; $s_2 \leftarrow (A + 2B) \bmod 64_{\circ}$
- 4: $b_1 \leftarrow (q \gg s_1) \& 1$; $b_2 \leftarrow (q \gg s_2) \& 1_{\circ}$
- 5: $m_L \leftarrow \text{rotl}(b_1, (c - r) \bmod 64)_{\circ}$
- 6: $m_R \leftarrow \text{rotr}(b_2, (2r - c) \bmod 64)_{\circ}$
- 7: $m \leftarrow m_L \oplus m_R_{\circ}$
- 8: $\tilde{q} \leftarrow q \& \sim m_{\circ}$
- 9: **return** $q, \tilde{q}_{\circ}$

Algorithm 51 CrazyTransform 的最终双通道输出

Require: $a, A, B, C, D \in \mathbb{Z}/2^{32}\mathbb{Z}$, $q, \tilde{q} \in \mathbb{Z}/2^{64}\mathbb{Z}$

Ensure: $F(a, k)$

- 1: $q_L \leftarrow \text{hi}_{32}(q)$, $q_R \leftarrow \text{lo}_{32}(q)_{\circ}$
- 2: $\tilde{q}_L \leftarrow \text{hi}_{32}(\tilde{q})$, $\tilde{q}_R \leftarrow \text{lo}_{32}(\tilde{q})_{\circ}$
- 3: $T_1 \leftarrow \text{rotl}(A \oplus \tilde{q}_R, 16) \oplus \text{rotr}(B \oplus \tilde{q}_L, 16)_{\circ}$
- 4: $T_2 \leftarrow (q_L - \text{rotl}(C \oplus q_R, 8)) \oplus \text{rotr}(D, 8)_{\circ}$
- 5: **return** $a + (T_1 \oplus T_2) \pmod{2^{32}}_{\circ}$

Algorithm 52 静态 S-box 生成公式与逆表构造

Require: primitive polynomial $\mu_0(X) = X^8 + X^5 + X^4 + X^3 + X^2 + X + 1$; affine matrix A_0 ; constant c_0 ; ZUC-style static forward table S_{F1}

Ensure: $S_{F0}, S_{B0}, S_{F1}, S_{B1}$

- 1: **for** $x = 0$ to 255 **do**
- 2: Interpret x as an element of $\mathbb{K}_0 = \mathbb{F}_2[X]/(\mu_0)$.


```

3:   if  $x = 0$  then
4:        $u \leftarrow 0$ .
5:   else
6:        $u \leftarrow x^{-1}$  in  $\mathbb{K}_0$ .
7:   end if
8:    $S_{F0}[x] \leftarrow \text{vec}^{-1}(A_0 \text{vec}(u) \oplus c_0)$ .
9: end for
10: for  $x = 0$  to 255 do
11:      $S_{B0}[S_{F0}[x]] \leftarrow x$ .
12:      $S_{B1}[S_{F1}[x]] \leftarrow x$ .
13: end for
14: return  $S_{F0}, S_{B0}, S_{F1}, S_{B1}$ .

```

Algorithm 53 ByteSubstitution forward schedule

Require: byte span B with $8 \mid |B|$; tables $S_{F0}, S_{B0}, S_{F1}, S_{B1}$

Ensure: substituted byte span B

```

1: for  $i = 0$  to  $|B| - 1$  step 8 do
2:    $B[i + 0] \leftarrow S_{F1}[B[i + 0]]$ .
3:    $B[i + 1] \leftarrow S_{F0}[B[i + 1]]$ .
4:    $B[i + 2] \leftarrow S_{B1}[B[i + 2]]$ .
5:    $B[i + 3] \leftarrow S_{B0}[B[i + 3]]$ .
6:    $B[i + 4] \leftarrow S_{F0}[B[i + 4]]$ .
7:    $B[i + 5] \leftarrow S_{B1}[B[i + 5]]$ .
8:    $B[i + 6] \leftarrow S_{F0}[B[i + 6]]$ .
9:    $B[i + 7] \leftarrow S_{B1}[B[i + 7]]$ .
10: end for
11: return  $B$ .

```

Algorithm 54 ByteSubstitution inverse schedule

Require: byte span B with $8 \mid |B|$; tables $S_{F0}, S_{B0}, S_{F1}, S_{B1}$

Ensure: restored byte span B

```

1: for  $i = 0$  to  $|B| - 1$  step 8 do
2:    $B[i + 0] \leftarrow S_{B1}[B[i + 0]]$ .
3:    $B[i + 1] \leftarrow S_{B0}[B[i + 1]]$ .
4:    $B[i + 2] \leftarrow S_{F1}[B[i + 2]]$ .
5:    $B[i + 3] \leftarrow S_{F0}[B[i + 3]]$ .
6:    $B[i + 4] \leftarrow S_{B0}[B[i + 4]]$ .
7:    $B[i + 5] \leftarrow S_{F1}[B[i + 5]]$ .
8:    $B[i + 6] \leftarrow S_{B0}[B[i + 6]]$ .
9:    $B[i + 7] \leftarrow S_{F1}[B[i + 7]]$ .
10: end for
11: return  $B$ .

```

Algorithm 55 RoundFunction 加密的完整嵌套循环

Require: block $X \in (\mathbb{Z}/2^{64}\mathbb{Z})^B$, round-subkey vector $g \in (\mathbb{Z}/2^{64}\mathbb{Z})^{N^2}$

Ensure: encrypted block X

```

1: for  $r = 0$  to 15 do
2:   for  $i = 0$  to  $N^2 - 1$  do
3:      $j \leftarrow i \bmod B$ .
4:      $X_j \leftarrow \text{LMEnc}(X_j, g_i)$ .
5:   end for
6:    $X \leftarrow \text{ByteSubstitution}(X)$ .
7: end for
8: return  $X$ .

```

Algorithm 56 RoundFunction 解密的完整逆序循环

Require: block $X \in (\mathbb{Z}/2^{64}\mathbb{Z})^B$, round-subkey vector $g \in (\mathbb{Z}/2^{64}\mathbb{Z})^{N^2}$

Ensure: decrypted block X

```
1: for  $r = 15$  down to  $0$  do
2:    $X \leftarrow \text{InverseByteSubstitution}(X)$ 。
3:   for  $i = N^2 - 1$  down to  $0$  do
4:      $j \leftarrow i \bmod B$ 。
5:      $X_j \leftarrow \text{LMDec}(X_j, g_i)$ 。
6:   end for
7: end for
8: return  $X$ 。
```

19.7 worker 级状态化调度

Algorithm 57 worker 加密调度与状态推进

Require: plaintext blocks $P^{(0)}, \dots, P^{(q-1)}$, key words K

Ensure: ciphertext blocks $C^{(0)}, \dots, C^{(q-1)}$

```
1: 初始化或继续公共状态  $\mathcal{S}$ 。
2: for 每个 key block 或 state epoch do
3:   调用 GenerationSubkeys 吸收 key words 并推进  $A, T$ 。
4:   for 该 epoch 内每个 plaintext block  $P^{(i)}$  do
5:      $g \leftarrow \text{GenerationRoundSubkeys}(\mathcal{S})$ 。
6:      $C^{(i)} \leftarrow \text{RoundFunctionEncrypt}(P^{(i)}, g)$ 。
7:     推进或保留实现规定的 state counter。
8:   end for
9: end for
10: return ciphertext blocks。
```

Algorithm 58 worker 解密调度与状态重建要求

Require: ciphertext blocks $C^{(0)}, \dots, C^{(q-1)}$, 同一 key words K , 等价初始状态

Ensure: plaintext blocks $P^{(0)}, \dots, P^{(q-1)}$

```
1: 从相同 key words、IV 与初始参数重建公共状态  $\mathcal{S}$ 。
2: for 每个 key block 或 state epoch do
3:   以与加密相同顺序调用 GenerationSubkeys。
4:   for 该 epoch 内每个 ciphertext block  $C^{(i)}$  do
5:      $g \leftarrow \text{GenerationRoundSubkeys}(\mathcal{S})$ 。
6:      $P^{(i)} \leftarrow \text{RoundFunctionDecrypt}(C^{(i)}, g)$ 。
7:   end for
8: end for
9: return plaintext blocks。
```

20 端到端正确性与实现–数学对应关系

我们把前文分散的实现对象收束为一条端到端路径。Type-2 core 的初始化阶段先把 key words 解释为 $\mathbb{F}_p$ 上的输入材料，经过

$$H_M(x) = Mx + \text{TDOM-Sponge}_h(Mx) \pmod{p}$$

得到主子密钥硬化向量；随后 32-bit expansion 将该向量转成适合矩阵填充的门级材料；矩阵初始化、IV 注入、NLFSR 仿射广播和 SDP–Kronecker 外积注入共同生成 A_t, T_t ；轮密钥阶段由

$$G \leftarrow G + (T^\top - A)(A^\top + T)AT$$

生成状态相关轮密钥矩阵，再经过 key-state whitening 与 $M_{\text{bit}} = D \otimes I_{64}$ 的 bit-uniform 投影得到轮密钥向量；数据轮阶段使用 g_t 驱动 Lai–Massey word permutation，并在每个 macro-round 末尾执行静态 byte substitution。

实现模块	数学对象	主要可证明性质
SecureSubkeyGeneratation	$H_M(x) = Mx + \text{TDOM-Sponge}_h(Mx)$	SIS/Ajtai 裸核约束与 twist 差分方程
CustomSecureHash	白盒海绵 TDOM-Sponge _h	padding/absorb/permutation/squeeze 可展开
MixTransformationUtil	E_{32} 扩展映射	bit 重排、PHT、 χ -like 依赖扩张
SubkeyMatrixOperation	A_t, T_t 状态轨道	状态确定性与外积窄注入公式
SecureRoundSubkeyGeneratation	G_t 、whitening、 $D \otimes I_{64}$	轮密钥状态合成与 bit fan-out
OaldresPuzzle_Cryptic	LM、F、byte substitution	数据路径正逆可闭合
OPC_MainAlgorithm_Worker	分块、padding、stateful 调度	封装层不改变核心置换语义

命题 20.1 (端到端可逆性). 若加密和解密使用相同的初始化参数、相同的状态推进轨道和相同的轮密钥向量序列，则 Type-2 的单分组数据变换可逆。

证明. 主子密钥硬化、矩阵状态更新和轮密钥生成不需要在数据解密阶段反向求逆；它们只需要在相同初始条件下重建同一轮密钥轨道。给定轮密钥轨道，数据路径由三类可逆层组成：PHT/rotation-xor 层具有显式逆，Lai–Massey 结构只要求 F 可重算而不要求 F 自身可逆，byte substitution 使用正逆表。解密按照 byte 层逆变换、Lai–Massey 逆调度和 macro-round 逆序执行，因此恢复原分组。□

我们这样收束实现–数学对应关系，是为了避免重复解释同一模块。每个实现文件只在其主体章节中完整定义一次；本节只给出端到端路径和正确性闭合。

21 设计动机、分析边界与后续工作

我们设计 Type-2 OPC 的核心目标，不是把复杂度当作安全证明，而是让复杂度分解为可审计的数学对象。SIS/Ajtai 层给出高维离散格骨架；TDOM 白盒海绵给出确定非线性位移场；矩阵状态层给出 stateful key schedule；Kronecker 外积注入把 SDP 材料压入方向性状态轨道；轮密钥矩阵和 BitMatrix 把状态材料投影到 bit-level parity 网络；CrazyTransform 把数据相关矩阵访问、mask 删除、双通道子密钥和 carry 链合在 Lai–Massey 的可逆框架中。

我们不声明完整 IND-CPA/PRP 归约，也不声明整个分组密码归约到 SIS 或 LWE。我们声明并证明的是更具体的结构事实：裸 SIS/Ajtai 核的碰撞、差分和线性掩码可以写成清楚的代数对象；TDOM hardening 后这些对象被扭成基点相关的非齐次移动目标与海绵掩码耦合；BitMatrix 的 row/column degree 和 single-bit fan-out 可以直接证明；PHT、Lai–Massey 与 byte substitution 的正逆路径可以闭合。后续分析应继续围绕差分 hull、线性 hull、相关密钥模型、状态恢复、矩阵轨道周期、TDOM 状态碰撞和量子查询模型展开。

22 结论

我们给出了 OaldresPuzzle_Cryptic Type-2 状态化分组密码的当前技术论文形态。该构造以 1024-bit 默认数据分组、2048-bit 默认主密钥块和 64×64 轮密钥矩阵为主体，把 SIS/Ajtai 形式素域矩阵像、TDOM 白盒海绵、32-bit 门级扩展、 $\mathbb{Z}/2^{64}\mathbb{Z}$ 矩阵状态轨道、Kronecker 外积窄注入、轮密钥矩阵变换、key-state whitening、BitMatrix parity 投影、状态相关 F 函数、Lai–Massey word permutation 和静态 byte substitution 组合为一条端到端可逆数据路径。

我们把设计动机落实为可审计的数学对象。SIS/Ajtai–TDOM 层形成 $\tau_h(\text{Im}(M))$ 与 $\Gamma_{M,h}$ 这样的 twist 图像空间；Kronecker 层形成 $T = \delta(Av)u$ 这样的 collapsed outer-product 状态注入；BitMatrix 层形成 $D \otimes I_{64}$ 这样的 bit-uniform parity network；CrazyTransform 则把状态相关矩阵访问、mask 和 carry 注入 Lai–Massey 所允许的不可逆 F 函数位置。传统分析者必须同时处理素域格像、白盒海绵差分、word overflow 矩阵轨道、bit-level parity、数据相关查表和模加 carry；量子攻击者若要构造完整 oracle，则必须把这些路径 coherent 地写成可逆门电路，并承担反算、ancilla、Toffoli/T-count、宽 CNOT 网络和受控选择的资源账本。

本文给出的结论保持边界清晰：我们不把实现复杂度本身当作安全证明，不声明整个分组密码已经归约到 SIS/LWE，也不声明抵抗所有理想化无限资源量子模型。我们声明的是当前构造的规格、设计理由、数学对象、局部可证明性质和后续分析入口。后续工作应继续以本文定义为基准，系统开展差分、线性、代数、相关密钥、状态恢复和量子 oracle 成本分析。

参考文献

[1] Fırat Artuğer and Fatih Özkaynak. A method for generation of substitution box based on random selection. *Egyptian Informatics Journal*, 23(1):127–135, 2022.

- [2] Fabio Borges, Paulo Ricardo Reis, and Diogo Pereira. A comparison of security and its performance for key agreements in post-quantum cryptography. *IEEE Access*, 8:142413–142422, 2020.
- [3] Amit Kumar Chauhan and Somitra Sanadhya. Quantum security of fox construction based on lai-massey scheme. Cryptology ePrint Archive, Paper 2022/1001, 2022. <https://eprint.iacr.org/2022/1001>.
- [4] Guillermo Cotrina, Alberto Peinado, and Andrés Ortiz. Gaussian pseudorandom number generator using linear feedback shift registers in extended fields. *Mathematics*, 9(5):556, 2021.
- [5] Aarti Dadheech. Post-quantum lattice-based cryptography: A quantum-resistant cryptosystem. In *Limitations and Future Applications of Quantum Cryptography*, pages 102–123. IGI Global, 2021.
- [6] Joan Daemen and Vincent Rijmen. *The design of Rijndael*, volume 2. Springer, 2002.
- [7] Muhammed Fethullah Esgin. *Practice-oriented techniques in lattice-based cryptography*. PhD thesis, Monash University, 2020.
- [8] Oded Goldreich, Shafi Goldwasser, and Shai Halevi. Collision-free hashing from lattice problems. *IACR Cryptol. ePrint Arch.*, 1996:9, 1996.
- [9] Marc Kaplan, Gaëtan Leurent, Anthony Leverrier, and María Naya-Plasencia. Breaking symmetric cryptosystems using quantum period finding. In *Advances in Cryptology – CRYPTO 2016*, pages 207–237. Springer Berlin Heidelberg, 2016.
- [10] Kazys Kazlauskas and Jaunius Kazlauskas. Key-dependent s-box generation in aes block cipher system. *Informatica, Lith. Acad. Sci.*, 20:23–34, 01 2009.
- [11] Hidenori Kuwakado and Masakatu Morii. Quantum distinguisher between the 3-round feistel cipher and the random permutation. In *2010 IEEE International Symposium on Information Theory*, pages 2682–2685, 2010.
- [12] Hidenori Kuwakado and Masakatu Morii. Security on the quantum-type even-mansour cipher. In *2012 International Symposium on Information Theory and its Applications*, pages 312–316, 2012.
- [13] Yiyuan Luo, Xuejia Lai, and Yujie Zhou. Generic attacks on the lai-massey scheme. *Designs, Codes and Cryptography*, 83, 05 2017.
- [14] Ashwini Kumar Malviya, Namita Tiwari, and Meenu Chawla. Quantum cryptanalytic attacks of symmetric ciphers: A review. *Computers and Electrical Engineering*, 101:108122, 2022.
- [15] Shuping Mao, Tingting Guo, Peng Wang, and Lei Hu. Quantum attacks on lai-massey structure. Cryptology ePrint Archive, Paper 2022/986, 2022. <https://eprint.iacr.org/2022/986>.
- [16] Chandra Sekhar Mukherjee, Dibyendu Roy, and Subhamoy Maitra. Design specification of zuc stream cipher. In *Design and Cryptanalysis of ZUC*, pages 43–62. Springer, 2021.
- [17] Chokri Nouar and Z. Guennoun. A pseudo-random number generator using double pendulum. *Applied Mathematics & Information Sciences*, 14:977–984, 11 2020.
- [18] Stjepan Picek, Lejla Batina, Domagoj Jakobović, Bariş Ege, and Marin Golub. S-box, SET, Match: A Toolbox for S-box Analysis. In David Naccache and Damien Sauveron, editors, *8th IFIP International Workshop on Information Security Theory and Practice (WISTP)*, volume LNCS-8501 of *Information Security Theory and Practice. Securing the Internet of Things*, pages 140–149, Heraklion, Crete, Greece, June 2014. Springer. Part 5: Short Papers.
- [19] Richard Preston. Applying grover’s algorithm to hash functions: A software perspective, 2022.
- [20] Tomasz Rachwalik, Janusz Szmidt, Robert Wicik, and Janusz Zablocki. Generation of nonlinear feedback shift registers with special-purpose hardware. Cryptology ePrint Archive, Paper 2012/314, 2012. <https://eprint.iacr.org/2012/314>.
- [21] Maximilian Richter, Magdalena Bertram, Jasper Seidensticker, and Alexander Tschache. A mathematical perspective on post-quantum cryptography. *Mathematics*, 10(15), 2022.
- [22] Ronald L Rivest, Matthew JB Robshaw, Ray Sidney, and Yiqun Lisa Yin. The rc6tm block cipher. In *First advanced encryption standard (AES) conference*, page 16, 1998.
- [23] M. R. Mirzaee Shamsabad and S. M. Dehnavi. Lai-massey scheme revisited. Cryptology ePrint Archive, Paper 2020/005, 2020. <https://eprint.iacr.org/2020/005>.

- [24] D.R. Simon. On the power of quantum computation. In *Proceedings 35th Annual Symposium on Foundations of Computer Science*, pages 116–123, 1994.
- [25] Nigel P Smart and Emmanuel Thomé. History of Cryptographic Key Sizes. In Joppe Bos and Martijn Stam, editors, *Computational Cryptography*, volume 469 of *London Mathematical Society Lecture Note Series*. Cambridge University Press, December 2021.
- [26] Han SUI and Wenling WU. Research status and development trend of post-quantum symmetric cryptography. *Journal of Electronics & Information Technology*, 42(190667):287, 2020.
- [27] Kethan Vooke, Nithin Kumar Toramamidi, Kalyan Kumar Thodeti, and Sangeeta Singh. Design of pseudo-random number generator using non-linear feedback shift register. In *2022 First International Conference on Electrical, Electronics, Information and Communication Technologies (ICEEICT)*, pages 1–5, 2022.
- [28] Zeguo Wang, Shijie Wei, Gui Long, and L. Hanzo. Variational quantum attacks threaten advanced encryption standard based symmetric cryptography. *Science China Information Sciences*, 65:200503, 10 2022.
- [29] Aaram Yun, Je Hong Park, and Jooyoung Lee. Lai-massey scheme and quasi-feistel networks. Cryptology ePrint Archive, Paper 2007/347, 2007. <https://eprint.iacr.org/2007/347>.